Smarter Regular Expression Matching

From Theory to Tools

Ilian Kohl

November 3, 2026

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Problem Statement . . . 4
- 1.3 Contributions . . . 4
- 1.4 Thesis Outline . . . 5

2 Regular Expressions **6**
- 2.1 Syntax . . . 6
- 2.2 Semantics . . . 7
- 2.3 Rust Representation . . . 8

3 Derivatives and Partial Derivatives **9**
- 3.1 Naive Matching . . . 9
- 3.2 Brzozowski Derivatives . . . 11
- 3.3 Antimirov Partial Derivatives . . . 17

4 Matching and Parsing **21**
- 4.1 Matching: A Boolean Answer . . . 21
- 4.2 Parsing: Parse Trees as Matching Evidence . . . 21

5 Derivative-Based Parser **24**
- 5.1 The Standard Derivative-Based Parser . . . 24
  - 5.1.1 Building the Empty-Word Tree . . . 25
  - 5.1.2 Reinserting Characters via Injection . . . 26
  - 5.1.3 Parser . . . 28
  - 5.1.4 Worked Example: Applying the Standard Parser . . . 29
- 5.2 The Bit-Coded Derivative-Based Parser . . . 30
  - 5.2.1 Annotated Expressions . . . 31
  - 5.2.2 Fusing Bit-Lists . . . 32
  - 5.2.3 Internalizing Expressions . . . 33
  - 5.2.4 Extracting Bits . . . 34
  - 5.2.5 Annotated Derivatives . . . 35
  - 5.2.6 Decoding Bits . . . 37
  - 5.2.7 Parser . . . 39
  - 5.2.8 Worked Example: Applying the Bitcoded Parser . . . 40

6 Partial-Derivative-Based Parser **41**
- 6.1 The Standard Partial-Derivative Parser . . . 41

6.1.1 Partial Derivatives with Injections . . . 42
6.1.2 State Sets and Selection . . . 46
6.1.3 Parser . . . 48
6.1.4 Worked Example: Standard Partial-Derivative Parser . . . 48
6.2 The Bit-Coded Partial-Derivative Parser . . . 50
6.2.1 Bit-Coded Partial Derivatives . . . 50
6.2.2 Extracting Bits . . . 53
6.2.3 State Sets and Selection . . . 54
6.2.4 Parser . . . 55
6.2.5 Worked Example: Ambiguous Expression . . . 55

7 Translation **57**
7.1 Parsing: String to ExtPat . . . 57
7.2 Translate: ExtPat to Regex . . . 57
7.3 The Surface Syntax . . . 58
7.3.1 Accepted Constructs . . . 59
7.3.2 Rejected Constructs . . . 59
7.4 Translation Sizes . . . 60

8 Evaluation **61**
8.1 Candidates . . . 61
8.2 Evaluation Criteria . . . 62
8.2.1 Measurement Setup . . . 62
8.2.2 Timing . . . 62
8.2.3 Memory . . . 63
8.2.4 Translation . . . 63
8.2.5 Stack Depth . . . 64
8.2.6 Parse Agreement . . . 64
8.3 Dataset Preparation . . . 65
8.3.1 S2ELab . . . 65
8.3.2 Synthetic Stress Set . . . 66
8.4 Performance Analysis . . . 67
8.4.1 Stack Depth . . . 67
8.4.2 Timing . . . 68
8.4.3 Memory . . . 73
8.4.4 Translation . . . 77
8.4.5 Parse Agreement . . . 79
8.5 Discussion of Results . . . 81
8.5.1 Cost of the standard construction . . . 81
8.5.2 Cost of bit-coding . . . 82
8.5.3 Cost of partial derivatives . . . 82
8.5.4 Threats to validity . . . 82

9 Related Work **84**

10 Conclusion **85**

1 Introduction

1.1 Motivation

Regular expressions are a fundamental formalism for describing regular languages and are among the most widely used abstractions in computer science. They underpin applications including lexical analysis, input validation, log processing, search tools, and network security. Across these applications, the problem is regular expression matching. Given a regular expression and an input string, determine whether the string belongs to the language denoted by the expression. Formally, this is the *membership problem*.

For example, the expression `(ab|a)*` matches `"aba"`, as it decomposes into `"ab"` followed by `"a"`, but does not match `"abb"`, since no sequence of subexpressions can produce it.

For many applications, a Boolean answer is enough. An input validator may only need to accept or reject a password or URL. Others require more. For instance, a lexer must determine how input decomposes into tokens. Capturing groups, schema validation, and information extraction require identifying which input parts correspond to parts of the expression. Recognition alone is not enough. The matcher must also recover the *structure*.

Efficiently solving these problems remains an active research area. The choice of algorithm influences performance, memory consumption, and security. Once matching becomes parsing, the way in which match structure is recovered also influences correctness and raises the question of how to handle ambiguity when multiple matches are possible.

Existing approaches include backtracking and finite-automata-based techniques, such as nondeterministic (NFA) and deterministic (DFA) automata. These approaches have different trade-offs in implementation complexity, memory consumption, and worst-case performance. Backtracking engines are flexible and widely used, but may exhibit exponential running time on constructed inputs, leading to regular expression denial-of-service (ReDoS) vulnerabilities [Cox, 2007]. Automata-based approaches provide stronger worst-case guarantees but may require preprocessing or suffer from state-space explosion.

Derivative-based algorithms provide an alternative by transforming the regular expression as input symbols are consumed. Each derivative represents the language remaining after an input symbol has been consumed. By maintaining a correspondence between an expression and its successive residuals, derivative-based methods provide an elegant foundation for regular expression matching and, with suitable extensions, structured parsing.

However, derivative-based techniques are less common in practical regular expression tooling. Their use for practical matching, parsing, and diagnostic tools motivates this thesis: "From Theory to Tools".

1.2 Problem Statement

Derivative-based approaches provide a principled foundation for regular expression matching and parsing. Brzozowski introduced regular expression derivatives as a way of transforming a regular expression after consuming an input symbol [Brzozowski, 1964]. Antimirov later introduced partial derivatives, which represent sets of residual expressions and are closely related to nondeterministic automata [Antimirov, 1996].

Building on Brzozowski's derivatives, Sulzmann and Lu developed a derivative-based algorithm for POSIX regular expression parsing [Sulzmann and Lu, 2014]. Their work also presents a bit-coded variant that represents parse information as a compact bit sequence, which can be decoded after a successful match rather than maintaining an explicit parse tree throughout the computation. A Haskell implementation of the POSIX parser is available in Sulzmann's *Programming Paradigms* teaching material [Sulzmann, 2026].

Sulzmann and Lu also studied partial derivatives in the context of regular expression sub-matching [Sulzmann and Lu, 2012]. While sub-matching is not the focus of this thesis, their work provides relevant background for the partial-derivative approach. The *Programming Paradigms* teaching material also presents a bit-coded partial-derivative-based parser in Haskell.

These constructions have primarily been presented in a theoretical and functional programming setting, leaving open practical questions about execution time, memory consumption, and data representation.

This thesis investigates these questions through implementations in Rust. As a systems programming language, Rust provides fine-grained control over memory allocation and data representation, allowing the derivative-based and partial-derivative-based parsers, including their bit-coded variants, to be evaluated in practice.

The implementation also provides traced versions of the parsing algorithms that record intermediate derivative computations during matching. These traces show the transformations performed by the algorithms and help explain successful and unsuccessful matches.

1.3 Contributions

The following contributions are being made:

- **Rust implementation of derivative-based parsers.** The thesis implements derivative-based and partial-derivative-based parsing constructions in Rust, including direct and bit-coded variants. The implementations are based on the constructions described in the literature and the corresponding Haskell material, resulting in four parser variants in total.
- **Empirical evaluation of the parser variants.** The four implementations are evaluated with respect to execution time and memory consumption using synthetic benchmarks and a corpus of real-world regular expressions from Snort/Suricata, SpamAssassin, and RegexLib.

1.4 Thesis Outline

The remainder of this thesis follows the parsing constructions from their formal concepts through their Rust implementation to their evaluation. Theory and implementation are presented together.

- **Chapter 2 - Regular Expressions.** This chapter introduces the syntax and semantics of regular expressions and the Rust representation used throughout the implementation.
- **Chapter 3 - Derivatives.** This chapter introduces Brzozowski's derivatives and Antimirov's partial derivatives, together with their basic matching algorithms and Rust implementations.
- **Chapter 4 - Matching vs. Parsing.** This chapter distinguishes regular expression matching from parsing and introduces the disambiguation problem.
- **Chapter 5 - Derivative Parser.** This chapter presents the derivative-based POSIX parsing approach of Sulzmann and Lu, along with a Rust implementation developed in this work, in both direct and bit-coded variants.
- **Chapter 6 - Partial-Derivative Parser.** This chapter presents the partial-derivative-based parsing approach of Sulzmann and Lu, along with a Rust implementation developed in this work, in both direct and bit-coded variants.
- **Chapter 7 - External versus Internal Syntax.** This chapter describes the parsing of regular expression strings into an external AST and their translation into the internal representation used by the parsers.
- **Chapter 8 - Evaluation.** This chapter evaluates the four parser variants with respect to execution time, memory consumption, and correctness. The evaluation uses synthetic benchmarks and a corpus of real-world regular expressions. It also examines optimization challenges and the insights gained from the evaluation.
- **Chapter 9 - Related Work.**
- **Chapter 10 - Conclusion.** Finally, this chapter summarizes the findings of the thesis and discusses the practical implications of the results and directions for future work.

2 Regular Expressions

2.1 Syntax

Let Σ be an alphabet. Regular expressions over Σ are defined inductively by the grammar:

$$r, s ::= x \mid \varepsilon \mid \emptyset \mid r + s \mid r \cdot s \mid r^* \qquad (x \in \Sigma). \tag{2.1}$$

In the grammar, r and s range over regular expressions, while x ranges over the alphabet Σ and denotes an arbitrary symbol, such as a, b, or c. The constants ε and $\emptyset$ represent the *empty-word* and *empty-language* expressions. The operators $+$, $\cdot$, and * denote alternation, concatenation, and Kleene star, respectively.

Concatenation is written implicitly, so rs denotes $r \cdot s$. Given this notation, we adopt the precedence and associativity conventions. Unless parentheses are shown explicitly, Kleene star binds most tightly, followed by concatenation, followed by alternation. For example,

$$ab^*c = (a \cdot (b^*)) \cdot c,$$
$$a + bc^* = a + (b \cdot (c^*)).$$

For repeated operators we use left association, so

$$a + b + c = (a + b) + c,$$
$$abc = (ab)c.$$

The grammar above is the core syntax. Most regular expression libraries also offer one-or-more, optional, and bounded repetition (r^+, $r?$, $r^{\{m,n\}}$), but these add no expressive power since each can be defined by the core constructs: $r^+ = r \cdot r^*$, $r? = r + \varepsilon$, and

$$r^{\{m,n\}} = \underbrace{r \cdot \ldots \cdot r}_{m \text{ times}} \cdot \underbrace{(r + \varepsilon) \cdot \ldots \cdot (r + \varepsilon)}_{n-m \text{ times}}.$$

In the implementation, expressions using such operators are translated into the core constructs before they reach the parsing algorithms.

For example, consider the regular expression r_{01} over $\Sigma = \{0, 1\}$

$$r_{01} = (0 + 1)^* \cdot 0 \cdot 1. \tag{2.2}$$

By the conventions above, its syntax tree is $((0 + 1)^* \cdot 0) \cdot 1$. This tree represents the hierarchical structure of the expression: a Kleene star applied to the alternation $0 + 1$, followed by concatenation with the literals 0 and 1.

The syntactic structure must be given a semantic interpretation.

2.2 Semantics

A semantic interpretation assigns a language to each regular expression. A *word* over Σ is a finite sequence of symbols from Σ. The set of all words over Σ, including the empty word ε, is denoted by Σ^*. A *language* over Σ is a subset of Σ^*.

For a regular expression r, we write $L(r)$ for the language it denotes, where $L(r) \subseteq \Sigma^*$. The definition follows the structure of the syntax tree by *structural recursion*. Each case corresponds to one production of the grammar in (2.1):

$$\begin{aligned} L(x) &= \{x\}, \\ L(\varepsilon) &= \{\varepsilon\}, \\ L(\emptyset) &= \emptyset, \\ L(r + s) &= L(r) \cup L(s), \\ L(r \cdot s) &= \{vw \mid v \in L(r),\ w \in L(s)\}, \\ L(r^*) &= \{\varepsilon\} \cup \{w_1 \cdots w_n \mid n \geq 1,\ w_1, \ldots, w_n \in L(r)\}. \end{aligned} \tag{2.3}$$

Table 2.1 applies these definitions to small expressions. The literal a denotes the language containing only the word a. The expression ε denotes the language containing only the empty word, while $\emptyset$ denotes the language containing no words. The alternation $a + b$ combines the languages of a and b to give $\{a, b\}$, and concatenation ab combines words from the languages of a and b to give $\{ab\}$. The Kleene star a^* allows zero or more concatenations of words from the language of a, giving $\{\varepsilon, a, aa, aaa, \ldots\}$.

Expression r	Language $L(r)$
a	$\{a\}$
ε	$\{\varepsilon\}$
$\emptyset$	$\emptyset$
$a + b$	$\{a, b\}$
ab	$\{ab\}$
a^*	$\{\varepsilon, a, aa, aaa, \ldots\}$

Table 2.1: Languages denoted by basic expressions.

For example $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ from (2.2), we compute its language step by step:

$$\begin{aligned} L(0) &= \{0\}, \\ L(1) &= \{1\}, \\ L(0 + 1) &= L(0) \cup L(1) = \{0, 1\}, \\ L((0 + 1)^*) &= \{0, 1\}^*, \\ L(r_{01}) &= L((0 + 1)^*) \cdot L(0) \cdot L(1) = \{\, w \cdot 0 \cdot 1 \mid w \in \{0, 1\}^* \,\}. \end{aligned} \tag{2.4}$$

This is the set of binary words ending in 01. For example, the words 01, 001, and 1101 belong to $L(r_{01})$, whereas ε, 0, and 010 do not.

This is the question of whether a word *matches* a regular expression: w *matches* r if $w \in L(r)$. Determining this is the *membership problem*.

2.3 Rust Representation

The grammar from (2.1) is represented directly as a Rust enum with one variant for each syntactic construct, as shown in Figure 2.1:

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    Phi, Eps, Lit(char),
    Seq(Box<Regex>, Box<Regex>),  // r . s
    Alt(Box<Regex>, Box<Regex>),  // r + s
    Star(Box<Regex>),              // r*
}

impl Regex {
    pub fn seq(r: Regex, s: Regex) -> Regex {
        Regex::Seq(Box::new(r), Box::new(s))
    }
    pub fn alt(r: Regex, s: Regex) -> Regex {
        Regex::Alt(Box::new(r), Box::new(s))
    }
    pub fn star(r: Regex) -> Regex { Regex::Star(Box::new(r)) }
    pub fn lit(c: char) -> Regex { Regex::Lit(c) }
}
```

Figure 2.1: The `Regex` enum and constructors. Derived traits support operations needed by parsing: `Clone` for duplication, `PartialEq`/`Eq` for comparison, `Hash` for collections.

The type carries no precedence or associativity information, as these are determined by the syntax tree. Rust requires explicit indirection because enum sizes must be known at compile time. Recursive variants therefore use `Box<Regex>`. A direct definition such as `Seq(Regex, Regex)` would have infinite size, whereas `Box` stores recursive values separately, providing finite size [The Rust Project Developers, 2026a,b]. This differs from Haskell's heap-allocated data types.

The example $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ from (2.2) is represented by the syntax tree:

$$\mathrm{Seq}\Big(\mathrm{Seq}(\mathrm{Star}(\mathrm{Alt}(0, 1)), 0), 1\Big). \tag{2.5}$$

In Rust, this is represented as shown in Figure 2.2, with the outermost `Seq` containing `Lit('1')` and `Seq(Star(Alt(0,1)), 0)` as its right and left children.

```
Regex::seq(
    Regex::seq(
        Regex::star(Regex::alt(Regex::lit('0'), Regex::lit('1'))),
        Regex::lit('0')),
    Regex::lit('1'))
```

Figure 2.2: Rust representation of r_{01}.

3 Derivatives and Partial Derivatives

Chapter 2 introduced the syntax and semantics of regular expressions, and defined the *membership problem*: Decide whether a word w belongs to $L(r)$. This chapter introduces three algorithms for solving it, each returning a Boolean answer: match or no match.

The first is a naive recursive matcher that follows the structure of the regular-expression semantics. The other two transform the regular expression as input symbols are consumed. Brzozowski derivatives transform it into a single residual expression, while Antimirov's partial derivatives represent the possible residual expressions as a set.

3.1 Naive Matching

A direct way to decide whether $w \in L(r)$ is to interpret the regular-expression semantics as recursive matching rules, by following the structure of the expression and searching for possible decompositions of the input. Let $w[i:j]$ denote the substring from position i to j. A recursive procedure $\mathrm{match}(w, i, j, r)$ determines whether $w[i:j] \in L(r)$:

$$\begin{aligned}
\mathrm{match}(w,i,j,\varepsilon) &\iff i = j,\\
\mathrm{match}(w,i,j,x) &\iff j = i+1 \wedge w[i] = x,\\
\mathrm{match}(w,i,j,r+s) &\iff \mathrm{match}(w,i,j,r) \vee \mathrm{match}(w,i,j,s),\\
\mathrm{match}(w,i,j,r\cdot s) &\iff \exists k \in [i,j].\ \mathrm{match}(w,i,k,r) \wedge \mathrm{match}(w,k,j,s),\\
\mathrm{match}(w,i,j,r^*) &\iff i = j \vee \exists k \in (i,j].\ \mathrm{match}(w,i,k,r) \wedge \mathrm{match}(w,k,j,r^*).
\end{aligned} \tag{3.1}$$

Each rule corresponds directly to the semantics. Concatenation does not specify where to split, so the matcher must consider all possible split points k.

For matching $(a \cdot a) \cdot a$ against `aaa`, the split points $k \in \{0,1,2,3\}$ are:

$$\begin{aligned}
&\underbrace{\mathrm{match}(w,0,0,a\cdot a)}_{a\cdot a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\mathrm{match}(w,0,3,a)}_{a \text{ against } w[0:3]=\texttt{aaa}}\\
\vee\ &\underbrace{\mathrm{match}(w,0,1,a\cdot a)}_{a\cdot a \text{ against } w[0:1]=\texttt{a}} \wedge \underbrace{\mathrm{match}(w,1,3,a)}_{a \text{ against } w[1:3]=\texttt{aa}}\\
\vee\ &\underbrace{\mathrm{match}(w,0,2,a\cdot a)}_{a\cdot a \text{ against } w[0:2]=\texttt{aa}} \wedge \underbrace{\mathrm{match}(w,2,3,a)}_{a \text{ against } w[2:3]=\texttt{a}}\\
\vee\ &\underbrace{\mathrm{match}(w,0,3,a\cdot a)}_{a\cdot a \text{ against } w[0:3]=\texttt{aaa}} \wedge \underbrace{\mathrm{match}(w,3,3,a)}_{a \text{ against } w[3:3]=\epsilon}.
\end{aligned} \tag{3.2}$$

At $k = 2$, both operands match, triggering a recursive search on $(a \cdot a)$ against `aa` with split points $k \in \{0, 1, 2\}$, where $k = 1$ succeeds:

$$\begin{aligned} & \underbrace{\text{match}(w, 0, 0, a)}_{a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\texttt{aa}} \\ \vee\ & \underbrace{\text{match}(w, 0, 1, a)}_{a \text{ against } w[0:1]=\texttt{a}} \wedge \underbrace{\text{match}(w, 1, 2, a)}_{a \text{ against } w[1:2]=\texttt{a}} \\ \vee\ & \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\texttt{aa}} \wedge \underbrace{\text{match}(w, 2, 2, a)}_{a \text{ against } w[2:2]=\epsilon}. \end{aligned} \tag{3.3}$$

This completes the original match through the nested split points $k = 2$ and $k = 1$.

In Rust. The recursive rules translate directly into the corresponding Rust match arms, shown in Figure 3.1. Alternation tries both branches on the same input, while concatenation and star search possible split points. The star arm clones the body to construct the recursive `Star` expression. This is necessary because the recursive call needs to own the newly constructed expression. The range-based search short-circuits at the first successful split. Nested concatenations repeatedly search smaller substrings, exploring multiple paths before succeeding. This leads to exponential backtracking in the worst case.

```
Regex::Alt(r, s) => {
    match_naive_range(word, i, j, r)
        || match_naive_range(word, i, j, s)
}
Regex::Seq(r, s) => {
    (i..=j).any(|k| {
        match_naive_range(word, i, k, r)
            && match_naive_range(word, k, j, s)
    })
}
Regex::Star(r) => {
    i == j || (i + 1..=j).any(|k| {
        match_naive_range(word, i, k, r)
            && match_naive_range(word, k, j,
                &Regex::star(*r.clone()))
    })
}
```

Figure 3.1: Alternation, concatenation, and star arms of the naive matcher.

Although inefficient, this matcher is useful as a reference implementation because it follows the regular-expression semantics directly without any intermediate transformation. The cost is visible in the worked examples: nested concatenations repeatedly search smaller substrings, each exploring all possible split points before finding a match.

The derivative approach takes a different approach. Rather than searching for a decomposition of w while keeping r fixed, it transforms r as each character of w is consumed. The resulting expression describes the language that can still match the remaining suffix.

3.2 Brzozowski Derivatives

Brzozowski's derivative of r with respect to a symbol x, written $r\backslash x$, is the expression describing exactly what remains of r's language after consuming the leading symbol x of a word $w \in L(r)$ [Brzozowski, 1964]:

$$L(r\backslash x) \;=\; \{\, w \mid xw \in L(r) \,\}. \tag{3.4}$$

For $w = x_1 x_2 \cdots x_n$, consume one symbol at a time by repeatedly taking derivatives:

$$r, \quad r\backslash x_1, \quad (r\backslash x_1)\backslash x_2, \quad \ldots$$

Nullability. After all symbols have been consumed, the remaining input is ε. Thus, the resulting expression must accept ε for the original word to belong to $L(r)$. This is the question of whether the expression is *nullable*. Let $n(r)$ be true exactly when $\varepsilon \in L(r)$:

$$\begin{array}{llll} n(\emptyset) = \text{false} & (\mathsf{N}_\emptyset) & n(r+s) = n(r) \vee n(s) & (\mathsf{N}_+) \\ n(\varepsilon) = \text{true} & (\mathsf{N}_\varepsilon) & n(r \cdot s) = n(r) \wedge n(s) & (\mathsf{N}_{\text{seq}}) \\ n(x) = \text{false} & (\mathsf{N}_{\text{lit}}) & n(r^*) = \text{true} & (\mathsf{N}_*) \end{array} \tag{3.5}$$

The empty language $\emptyset$ is not nullable because it contains no words, including ε, while a literal cannot match the empty word. ε and r^* always match ε. An alternation is nullable when either branch is nullable, and a concatenation only when both components are.

In Rust. The definition is transcribed one `Regex` variant at a time, using the enum of Figure 2.1. The case split of (3.5) becomes the arms of a `match`, shown in Figure 3.2.

```
pub fn nullable(r: &Regex) -> bool {
    match r {
        Regex::Phi => false,
        Regex::Eps => true,
        Regex::Lit(_) => false,
        Regex::Alt(r, s) => nullable(r) || nullable(s),
        Regex::Seq(r, s) => nullable(r) && nullable(s),
        Regex::Star(_) => true,
    }
}
```

Figure 3.2: `nullable` implementation and its six cases: $\mathsf{N}_\emptyset$, N_ε, N_{lit}, N_+, N_{seq} and N_*.

The expression is passed by reference because `nullable` only inspects r and does not modify or consume it. The underscores in N_{lit} and N_* indicate that the corresponding operands are irrelevant. A literal is never nullable, regardless of its symbol, while a Kleene-star expression is always nullable, regardless of its body. The short-circuiting operators `||` and `&&` avoid evaluating the second operand when the result is already determined, while preserving the corresponding logical semantics.

Derivative Rules. The rules specify the expression that remains after consuming x:

$$\begin{aligned}
&\emptyset\backslash x = \emptyset && (\mathsf{D}_\emptyset) && (r+s)\backslash x = (r\backslash x) + (s\backslash x) && (\mathsf{D}_+)\\
&\varepsilon\backslash x = \emptyset && (\mathsf{D}_\varepsilon) && r^*\backslash x = (r\backslash x)\cdot r^* && (\mathsf{D}_*)\\
&y\backslash x = \begin{cases} \varepsilon & x = y \\ \emptyset & x \neq y \end{cases} && (\mathsf{D}_{\text{lit}}) && (r\cdot s)\backslash x = \begin{cases} (r\backslash x)\cdot s + s\backslash x & n(r) = \text{true} \\ (r\backslash x)\cdot s & n(r) = \text{false} \end{cases} && \begin{matrix}(\mathsf{D}^n_{\text{seq}})\\(\mathsf{D}_{\text{seq}})\end{matrix}
\end{aligned} \tag{3.6}$$

The empty language and ε cannot consume x. A literal y succeeds exactly when $x = y$, leaving ε. Alternation retains both derivatives and for r^*, one occurrence of r consumes x while further occurrences remain available.

Concatenation is the only rule that depends on nullability. If r is not nullable, r must consume the leading x, leaving $r\backslash x$ as the remainder of r while s remains untouched. If r is nullable, it may either consume x giving $(r\backslash x)\cdot s$, or r matches ε and s consumes x leaving $s\backslash x$. Hence the nullable case produces the alternation $(r\backslash x)\cdot s + s\backslash x$.

In Rust. Figure 3.3 implements (3.6) similarly, one variant at a time.

```
pub fn deriv(r: &Regex, x: char) -> Regex {
    match r {
        Regex::Phi => Regex::Phi,
        Regex::Eps => Regex::Phi,
        Regex::Lit(c) => if *c == x { Regex::Eps } else { Regex::Phi },
        Regex::Alt(r1, r2) =>
            Regex::alt(deriv(r1, x), deriv(r2, x)),
        Regex::Seq(r1, r2) => {
            let dr1 = Regex::seq(deriv(r1, x), *r2.clone());
            if nullable(r1) {
                Regex::alt(dr1, deriv(r2, x))
            } else { dr1 }
        }
        Regex::Star(r1) =>
            Regex::seq(deriv(r1, x), Regex::star(*r1.clone())),
    }
}
```

Figure 3.3: `deriv` implementation with six cases.

`deriv` differs from `nullable` in that it borrows its input but returns an owned `Regex`, since a derivative is a new expression. The `if nullable(r1)` implements the case split of (3.6), with branches in order $\mathsf{D}^n_{\text{seq}}$ then D_{seq}. This is the only place where a derivative rule consults nullability and where the output can be larger than the input, since the nullable branch produces an alternation while the other produces a concatenation.

The cloning in `*r2.clone()` is necessary because s appears untouched in both branches of the concatenation case, and the star arm must preserve r^*'s body both derived and underived. This copying mechanism drives expression growth, which justifies simplification after every derivative step.

Example: Growth without simplification. Consider $(x+y)^*$ against the word $\mathtt{xx}$: The expression is a star, so D_* applies with body $r = x + y$:

$$\begin{aligned} (x+y)^*\backslash\mathtt{x} &= ((x+y)\backslash\mathtt{x}) \cdot (x+y)^* && \mathsf{D}_*,\ r = x+y \\ &= (\varepsilon + \emptyset) \cdot (x+y)^* && \mathsf{D}_+,\ \mathsf{D}_{\text{lit}} \end{aligned} \tag{3.7}$$

The expression is now the concatenation $(\varepsilon + \emptyset) \cdot (x+y)^*$, and deriving it consumes the second $\mathtt{x}$. The result is an alternation because $\mathsf{D}^n_{\text{seq}}$ applies with $r = \varepsilon + \emptyset$ and $s = (x+y)^*$, its nullable case being selected since $n(r) = n(\varepsilon + \emptyset) = \text{true}$ by N_+ and N_ε:

$$\begin{aligned} &((\varepsilon + \emptyset) \cdot (x+y)^*)\backslash\mathtt{x} \\ &= ((\varepsilon + \emptyset)\backslash\mathtt{x}) \cdot (x+y)^* + (x+y)^*\backslash\mathtt{x} && \mathsf{D}^n_{\text{seq}} \\ &= (\emptyset + \emptyset) \cdot (x+y)^* + (x+y)^*\backslash\mathtt{x} && \mathsf{D}_\varepsilon \\ &= (\emptyset + \emptyset) \cdot (x+y)^* + (\varepsilon + \emptyset) \cdot (x+y)^* && \mathsf{D}_*,\ \mathsf{D}_+,\ \mathsf{D}_{\text{lit}} \end{aligned} \tag{3.8}$$

After both symbols are consumed, the result is checked for nullability:

$$\begin{aligned} &n((\emptyset + \emptyset) \cdot (x+y)^* + (\varepsilon + \emptyset) \cdot (x+y)^*) \\ &= n((\emptyset + \emptyset) \cdot (x+y)^*) \vee n((\varepsilon + \emptyset) \cdot (x+y)^*) && \mathsf{N}_+ \\ &= (n(\emptyset + \emptyset) \wedge n((x+y)^*)) \vee (n(\varepsilon + \emptyset) \wedge n((x+y)^*)) && \mathsf{N}_{\text{seq}} \\ &= (\text{false} \wedge \text{true}) \vee (\text{true} \wedge \text{true}) && \mathsf{N}_+,\ \mathsf{N}_\emptyset,\ \mathsf{N}_\varepsilon,\ \mathsf{N}_* \\ &= \text{true}. \end{aligned} \tag{3.9}$$

Thus $(x+y)^*$ matches $\mathtt{xx}$.

Simplification. Derivatives can grow unboundedly. Repeatedly deriving $(x+y)^*$ by the same symbol produces expressions of increasing size. This growth results from the nullable branch of $\mathsf{D}^n_{\text{seq}}$, which introduces an alternation as a redundant structure that is carried into subsequent derivatives. ε and $\emptyset$ accumulate without bound:

$$\begin{aligned} & (x+y)^* && 4 \\ \xrightarrow{\mathtt{x}}\ & (\varepsilon + \emptyset) \cdot (x+y)^* && 8 \\ \xrightarrow{\mathtt{x}}\ & (\emptyset + \emptyset) \cdot (x+y)^* \ +\ (\varepsilon + \emptyset) \cdot (x+y)^* && 17 \\ \xrightarrow{\mathtt{x}}\ & (\emptyset + \emptyset) \cdot (x+y)^* \ +\ \left((\emptyset + \emptyset) \cdot (x+y)^* \ +\ (\varepsilon + \emptyset) \cdot (x+y)^*\right) && 26 \end{aligned} \tag{3.10}$$

To counter the growth, expressions are simplified after each derivative step, using identities that preserve the denoted language:

$$\begin{aligned} \emptyset \cdot r &= \emptyset && (\mathsf{S}_1) \\ \varepsilon \cdot r &= r && (\mathsf{S}_2) \\ \emptyset + r &= r && (\mathsf{S}_3) \end{aligned} \tag{3.11}$$

S_1 eliminates dead concatenations with $\emptyset$, S_2 eliminates identity concatenations with ε, and S_3 eliminates $\emptyset$ from alternations. Each holds equally with the unit on the right.

Applying them to (3.8), after each step, returns the expression back to $(x+y)^*$, keeping the size bounded regardless of input length:

$$\begin{aligned}
&(\emptyset + \emptyset) \cdot (x+y)^* + (\varepsilon + \emptyset) \cdot (x+y)^* \\
&= \emptyset \cdot (x+y)^* + \varepsilon \cdot (x+y)^* && \mathsf{S}_3,\ \text{twice} \\
&= \emptyset + (x+y)^* && \mathsf{S}_1,\ \mathsf{S}_2 \\
&= (x+y)^* && \mathsf{S}_3
\end{aligned} \tag{3.12}$$

In Rust. The identities are implemented recursively, applying the rules bottom-up over the expression tree. Each arm of the inner match implements an identity from (3.11).

`simplify` takes ownership of its input and returns simplified subexpressions directly, unlike `deriv`, which borrows and constructs. This model is visible where the two compose:

```
current = simplify(deriv(&current, c));
```

`deriv` borrows `current`, constructs a new expression, and `simplify` takes ownership of that result to return it without additional copying. The recursive calls ensure that all dead code is eliminated before an expression is returned, keeping size bounded at each step. Together, derive-then-simplify forms the core loop of the matcher, transforming and cleaning the expression at every input symbol.

```
pub fn simplify(r: Regex) -> Regex {
    match r {
        Regex::Seq(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) | (_, Regex::Phi) => Regex::Phi,
                (Regex::Eps, _) => s,
                (_, Regex::Eps) => r,
                _ => Regex::seq(r, s),
            }
        }
        Regex::Alt(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) => s,
                (_, Regex::Phi) => r,
                _ if r == s => r,
                _ => Regex::alt(r, s),
            }
        }
        Regex::Star(r) => match simplify(*r) {
            Regex::Eps | Regex::Phi => Regex::Eps,
            r => Regex::star(r),
        },
        other => other,
    }
}
```

Figure 3.4: Simplification rules: S_1 and S_2 in concatenation, S_3 in both branches.

The matcher. The implementation can be summarized by the sequence of residual expressions produced while consuming the input. For $w = x_1 x_2 \cdots x_n$, the initial residual is $r_0 = r$, and each subsequent residual is obtained by deriving with respect to the next symbol and simplifying the result:

$$r_0 = r, \qquad r_i = \text{simplify}(r_{i-1} \backslash x_i) \quad (1 \le i \le n). \tag{3.13}$$

The residual r_i denotes exactly the set of suffixes that complete the consumed prefix $x_1 \cdots x_i$ to a word in $L(r)$. Since simplification preserves the represented language, each r_i is equivalent to the unsimplified derivative after consuming that prefix. After the final symbol has been consumed, the matcher accepts when the remaining residual is nullable:

$$w \in L(r) \iff n(r_n) = \text{true}.$$

Each step depends only on the current residual and the next symbol, so the matcher never revisits a prefix and never backtracks into an earlier residual. The cost of one step is the cost of `deriv` on the current residual, which is proportional to its size.

So $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \cdots \xrightarrow{x_n} r_n$ can be viewed as a path through a lazily constructed DFA. Each residual expression represents a state, and $\xrightarrow{x}$ denotes the transition obtained by taking the derivative with respect to x. Unlike an explicitly constructed DFA, only the states reached by the input are constructed. A residual is an accepting state exactly when it is nullable, so acceptance is determined by whether the final residual is nullable.

In Rust. Figure 3.5 implements (3.13) by composing `deriv`, `simplify`, and `nullable`. The implementation follows the mathematical definition directly. The current residual is transformed at each step, and acceptance is determined after the complete input has been consumed.

```
pub fn match_deriv(input: &str, r: &Regex) -> bool {
    let mut current = r.clone();
    for c in input.chars() {
        current = simplify(deriv(&current, c));
    }
    nullable(&current)
}
```

Figure 3.5: `match_deriv`: the core matcher loop. For each input character, derive the current residual expression and simplify it in place. Accept if the final residual is nullable.

The loop implements the residual sequence. At each iteration, `current` contains the current residual, which is then derived and simplified before processing the next input symbol. Only the current residual is retained, since the previous residual is dropped when `current` is reassigned. In contrast to parsing, where intermediate results may be required to reconstruct a parse tree, Boolean matching requires only the final residual.

The derivative-based matcher avoids the exponential backtracking of naive matching by transforming the expression rather than searching for decompositions. Simplification keeps the resulting expressions compact, preventing the growth shown in (3.10).

Worked Example: Applying the derivative-based matcher Apply `match_deriv` to $r_0 = (a + ab)(b + \varepsilon)$ and $w = \mathtt{ab}$ with the expression written as:

$$r_0 = \mathit{Seq}(s,\, t), \qquad s = a + ab, \qquad t = b + \varepsilon,$$

Derive by `a`. The first factor s is not nullable, so D_{seq} applies to the sequence r_0:

$$r_1 = r_0 \backslash \mathtt{a} = (s \backslash \mathtt{a}) \cdot t \qquad (\mathsf{D}_{\text{seq}}).$$

Since s cannot match the empty word, the second factor t is not touched by this derivative and appears unchanged. To derive s, the alternation rule D_{+} is applied to give:

$$s \backslash \mathtt{a} = (a \backslash \mathtt{a}) + (ab \backslash \mathtt{a}) \qquad (\mathsf{D}_{+}).$$

The two summands are derived separately, each by its own rule:

$$\begin{aligned} a \backslash \mathtt{a} &= \varepsilon && (\mathsf{D}_{\text{lit}}), \\ ab \backslash \mathtt{a} &= (a \backslash \mathtt{a}) \cdot b = \varepsilon \cdot b && (\mathsf{D}_{\text{seq}},\ \mathsf{D}_{\text{lit}}). \end{aligned}$$

Substituting both summands into the alternation and the result into the outer sequence:

$$\begin{aligned} s \backslash \mathtt{a} &= \varepsilon + \varepsilon \cdot b && (\mathsf{D}_{+}), \\ r_0 \backslash \mathtt{a} &= (\varepsilon + \varepsilon \cdot b) \cdot t && (\mathsf{D}_{\text{seq}}). \end{aligned}$$

Then, applying the simplification rule S_2 to the occurrences of $\varepsilon \cdot b$, gives the residual:

$$r_1 = (\varepsilon + b) \cdot t \qquad (\mathsf{S}_2).$$

Derive by `b`. The residual r_1 is again a sequence, but now the first factor $\varepsilon + b$ is nullable, so the nullable sequence rule $\mathsf{D}^{n}_{\text{seq}}$ applies when deriving by b:

$$r_2 = r_1 \backslash \mathtt{b} = \Big((\varepsilon + b) \backslash \mathtt{b}\Big) \cdot t + (t \backslash \mathtt{b}) \qquad (\mathsf{D}^{n}_{\text{seq}}).$$

Since the first factor $\varepsilon + b$ is nullable, it can either consume the character itself, leaving t untouched, or match ε and let t consume it. Both are possible, so $\mathsf{D}^{n}_{\text{seq}}$ produces one summand for each, and D_{+} derives the first factor and t in their respective summands:

$$\begin{aligned} (\varepsilon + b) \backslash \mathtt{b} &= (\varepsilon \backslash \mathtt{b}) + (b \backslash \mathtt{b}) = \emptyset + \varepsilon && (\mathsf{D}_{+},\ \mathsf{D}_{\varepsilon},\ \mathsf{D}_{\text{lit}}), \\ t \backslash \mathtt{b} = (b + \varepsilon) \backslash \mathtt{b} &= (b \backslash \mathtt{b}) + (\varepsilon \backslash \mathtt{b}) = \varepsilon + \emptyset && (\mathsf{D}_{+},\ \mathsf{D}_{\text{lit}},\ \mathsf{D}_{\varepsilon}). \end{aligned}$$

Substituting both summands into the two-summand expression:

$$r_1 \backslash \mathtt{b} = (\emptyset + \varepsilon) \cdot t + (\varepsilon + \emptyset).$$

Applying the simplification rules S_3 and S_2 to the first summand and S_3 to the second, gives the final residual after deriving the second character:

$$r_2 = t + \varepsilon \qquad (\mathsf{S}_3,\ \mathsf{S}_2,\ \mathsf{S}_3).$$

Acceptance. The residual $r_2 = (b + \varepsilon) + \varepsilon$ is nullable by N_{+} and N_{ε}, so $\mathtt{ab} \in L(r_0)$.

3.3 Antimirov Partial Derivatives

The derivative of Section 3.2 combines alternative residuals into a single expression using $+$. Antimirov's partial derivatives instead keep them as a *set* of expressions, one per strand of nondeterminism [Antimirov, 1996]. Writing $pd(r, x)$ for this set:

$$
\begin{aligned}
pd(\emptyset, x) &= \{\} && && (\mathsf{P}_\emptyset)\\
pd(\varepsilon, x) &= \{\} && && (\mathsf{P}_\varepsilon)\\
pd(y, x) &= \begin{cases} \{\varepsilon\} & x = y \\ \{\} & x \neq y \end{cases} && && (\mathsf{P}_{\text{lit}})\\
pd(r + s, x) &= pd(r, x) \cup pd(s, x) && && (\mathsf{P}_+)\\
pd(r^*, x) &= \{\, smartC(r', r^*) \mid r' \in pd(r, x) \,\} && && (\mathsf{P}_*)\\
pd(r \cdot s, x) &= \begin{cases} \{\, smartC(r', s) \mid r' \in pd(r, x) \,\} \cup pd(s, x) & n(r) = \text{true} & (\mathsf{P}^n_{\text{seq}}) \\ \{\, smartC(r', s) \mid r' \in pd(r, x) \,\} & n(r) = \text{false} & (\mathsf{P}_{\text{seq}}) \end{cases}
\end{aligned}
\tag{3.14}
$$

The empty language and ε produce the empty set, since neither has a residual with respect to x: where $\mathsf{D}_\emptyset$ and D_ε returned $\emptyset$ as a node in the result, $\mathsf{P}_\emptyset$ and P_ε return no residual at all. A literal y gives $\{\varepsilon\}$ when $x = y$ and $\{\}$ otherwise. Alternation unions the two residual sets, and for r^* one iteration of r consumes x, with each resulting residual placed in front of r^* so the star may continue. None of these alternatives are combined with $+$; each stays a separate element of the set.

Concatenation depends on nullability for the same reason as D_{seq}. If r is not nullable it must consume x, and each of its residuals is concatenated with the untouched s. If r is nullable, r may instead match ε and let s consume x, in which case $pd(s, x)$ joins the set. Where $\mathsf{D}^n_{\text{seq}}$ combined the two cases with $+$ inside one expression, $\mathsf{P}^n_{\text{seq}}$ keeps them as two elements of one set.

The smart constructor. The concatenation in P_{seq} uses an auxiliary constructor *smartC*. Since each element of $pd(r, x)$ is a separate residual rather than a branch of a sum, *smartC* is applied to each independently. The constructor avoids building $\varepsilon \cdot s$:

$$
smartC(r', s) = \begin{cases} s & r' = \varepsilon \\ r' \cdot s & \text{otherwise.} \end{cases}
\tag{3.15}
$$

In Rust. Figure 3.6 implements *smartC* directly as `smart_seq`. Matching on `r` implements the case split of (3.15): the `Eps` arm returns a clone of `s` and never builds $\varepsilon \cdot s$, while the second arm wraps `r` in front of a cloned `s`

```
pub fn smart_seq(r: Regex, s: &Regex) -> Regex {
    match r {
        Regex::Eps => s.clone(),
        r => Regex::seq(r, s.clone()),
    }
}
```

Figure 3.6: `smart_seq`, the smart constructor *smartC*

Figure 3.7 presents the implementation of (3.14), with each `match` arm corresponding to one case. The `Phi`, `Eps`, and `Lit` variants implement $\mathsf{P}_{\emptyset}$, P_{ε}, and P_{lit} directly, returning `Vec::new()` where the equation gives $\{\}$. `Alt` concatenates the two child vectors, implementing the union of P_{+}.

`Star` maps `smart_seq` over r's own residuals, matching P_{*}. The star's body is derived, and each resulting residual is placed in front of a fresh `Regex::star` so the star may continue. `Regex::star(*r.clone())` clones because `Regex::star` needs an owned `Regex` and `r` is only borrowed, once per residual since the closure runs once for each.

`Seq` implements the case split directly. The `map` applies `smart_seq` to each residual of the left operand, matching P_{seq}, and the guarded `out.extend(pderiv(s, x))` adds $pd(s, x)$ for $\mathsf{P}^{n}_{\text{seq}}$. Although the mathematical definition uses sets, the implementation uses `Vec<Regex>` throughout the recursion and defers deduplication to a single `HashSet` insertion at the top, rather than deduplicating at every `Alt` or `Seq` node on the way down.

Every union in `pderiv` is therefore a vector `extend`, not a set operation. Duplicate residuals reached by different strands are only merged once inserted into the `HashSet` used by the matcher below to keep the residual generation separate from deduplication

```
fn pderiv(r: &Regex, x: char) -> Vec<Regex> {
    match r {
        Regex::Phi => Vec::new(),
        Regex::Eps => Vec::new(),
        Regex::Lit(c) => {
            if *c == x { vec![Regex::Eps] } else { Vec::new() }
        }
        Regex::Alt(r, s) => {
            let mut out = pderiv(r, x);
            out.extend(pderiv(s, x));
            out
        }
        Regex::Seq(r, s) => {
            let mut out: Vec<Regex> = pderiv(r, x)
                .into_iter()
                .map(|r_prime| smart_seq(r_prime, s))
                .collect();
            if nullable(r) {
                out.extend(pderiv(s, x));
            }
            out
        }
        Regex::Star(r) => {
            pderiv(r, x)
                .into_iter()
                .map(|r_prime| smart_seq(r_prime, &Regex::star(*r.clone())))
                .collect()
        }
    }
}
```

Figure 3.7: `pderiv`, implementing (3.14) one arm per case of the equation.

Example: No growth under *pd*. Consider $(x+y)^*$ against $\mathtt{xx}$ under pd.

P_* applies with star body $r = x+y$:

$$\begin{aligned} pd((x+y)^*, \mathtt{x}) &= \{\, smartC(r', (x+y)^*) \mid r' \in pd(x+y, \mathtt{x}) \,\} && \mathsf{P}_* \\ &= \{\, smartC(r', (x+y)^*) \mid r' \in \{\varepsilon\} \,\} && \mathsf{P}_+,\ \mathsf{P}_{\text{lit}} \text{ twice} \\ &= \{(x+y)^*\} && \mathsf{SC},\ r' = \varepsilon. \end{aligned} \tag{3.16}$$

The set is $\{(x+y)^*\}$, so deriving by the second $\mathtt{x}$ repeats the computation, and the final partial derivative accepts ε since $(x+y)^*$ is nullable by N_*. So $\mathtt{xx} \in L((x+y)^*)$.

Equation (3.17) traces the derivative and the partial derivative on the same expression:

$$\begin{aligned} \text{Derivative (unsimplified):} \quad & \underbrace{(x+y)^*}_{4} \xrightarrow{\mathtt{x}} \underbrace{(\varepsilon+\emptyset)(x+y)^*}_{8} \xrightarrow{\mathtt{x}} \underbrace{(\emptyset+\emptyset)(x+y)^*+(\varepsilon+\emptyset)(x+y)^*}_{17} \\ \text{Partial derivative:} \quad & \underbrace{\{(x+y)^*\}}_{4} \xrightarrow{\mathtt{x}} \underbrace{\{(x+y)^*\}}_{4} \xrightarrow{\mathtt{x}} \underbrace{\{(x+y)^*\}}_{4} \end{aligned} \tag{3.17}$$

The unsimplified derivative grows at every step, while the partial derivative stays at one element because $\mathsf{P}_\emptyset$ and P_ε contribute no residual, so there is nothing to simplify.

Finiteness. For a regular expression r, the number of distinct partial derivatives reachable by any input is bounded by $\|r\| + 1$ [Antimirov, 1996, Theorem 3.4, p. 309]. This is a ceiling, not a guarantee that all $\|r\| + 1$ are reached. For $(x+y)^*$, $\|r\| = 2$ the ceiling is 3, yet only $(x+y)^*$ is reachable, since deriving by either $\mathtt{x}$ or $\mathtt{y}$ returns it unchanged.

Ordinary derivatives need equivalence classes to be finite [Brzozowski, 1964, Theorem 4.3], giving $d_R \leq 2^{d_P} - 1$. Simplification removes units and redundant $\emptyset$, but not all equivalent forms. In contrast the partial-derivative bound is linear without simplification.

The matcher. The implementation can be summarized by the sequence of state sets produced while consuming the input. For $w = x_1 x_2 \cdots x_n$, the initial state set is $S_0 = \{r\}$, and each subsequent set is obtained by taking pd of every residual in the previous set and unioning the results:

$$S_0 = \{r\}, \qquad S_i = \bigcup_{r' \in S_{i-1}} pd(r', x_i) \quad (1 \leq i \leq n). \tag{3.18}$$

The state set S_i collects every residual reachable after consuming the prefix $x_1 \cdots x_i$. Unlike the derivative matcher's single residual r_i, no element of S_i grows with the input. The number of elements is bounded by $\|r\| + 1$ as shown above. After the final symbol has been consumed, the matcher accepts when some residual in S_n is nullable:

$$w \in L(r) \iff \exists r_n \in S_n.\ n(r_n) = \text{true}.$$

So $S_0 \xrightarrow{x_1} S_1 \xrightarrow{x_2} \cdots \xrightarrow{x_n} S_n$ can be viewed as a path through a lazily constructed NFA. Each state set represents the set of NFA states reachable after reading the corresponding prefix, and $\xrightarrow{x}$ denotes the transition obtained by taking pd with respect to x and unioning. Unlike an explicitly constructed NFA, only the states reached by the input are constructed. A state set is accepting exactly when it contains at least one nullable residual, so acceptance is determined by whether the final state set contains a nullable residual.

In Rust. Figure 3.8 implements (3.18) with a `HashSet<Regex>`, whose `Hash` and `Eq` implementations collapse residuals reached by different paths. The early return when the set empties follows from construction: an empty set has no states to derive. The final `any(nullable)` checks whether some residual is nullable. Where the derivative matcher holds one growing `Regex`, this holds a set of small residuals that never need simplification.

```
pub fn match_pderiv(input: &str, r: &Regex) -> bool {
    let mut states: HashSet<Regex> = HashSet::new();
    states.insert(r.clone());
    for c in input.chars() {
        let mut next: HashSet<Regex> = HashSet::new();
        for state in &states {
            next.extend(pderiv(state, c));
        }
        if next.is_empty() { return false; }
        states = next;
    }
    states.iter().any(nullable)
}
```

Figure 3.8: The partial-derivative matcher.

Worked Example: Applying the partial-derivative-based matcher Applying `match_pderiv` to $r_0 = (a + ab)(b + \varepsilon)$ and $w = \mathtt{ab}$ with the expression written as:

$$r_0 = \mathit{Seq}(s,\, t), \qquad s = a + ab, \qquad t = b + \varepsilon.$$

Derive by `a`. Since s is not nullable, t is untouched and P_{seq} applies to r_0:

$$S_1 = \{\, \mathit{smartC}(r', t) \mid r' \in \mathit{pd}(s, \mathtt{a}) \,\} \qquad (\mathsf{P}_{\text{seq}}).$$

The alternation rule gives the two branches of s separately:

$$\begin{aligned} \mathit{pd}(a, \mathtt{a}) &= \{\varepsilon\} && (\mathsf{P}_{\text{lit}}),\\ \mathit{pd}(ab, \mathtt{a}) &= \{\mathit{smartC}(\varepsilon, b)\} = \{b\} && (\mathsf{P}_{\text{seq}}), \end{aligned}$$

So $\mathit{pd}(s, \mathtt{a}) = \{\varepsilon, b\}$. Applying smartC to each element against t prepends that element to t, with ε absorbed so that $\varepsilon \cdot t$ is never built:

$$S_1 = \{\, t,\ b \cdot t \,\} = \{\, b + \varepsilon,\ \ b\,(b + \varepsilon) \,\}.$$

Derive by `b`. Each state in S_1 is derived separately and the results unioned.

For the first state $t = b + \varepsilon$:

$$\mathit{pd}(b + \varepsilon, \mathtt{b}) = \mathit{pd}(b, \mathtt{b}) \cup \mathit{pd}(\varepsilon, \mathtt{b}) = \{\varepsilon\} \cup \{\} = \{\varepsilon\} \qquad (\mathsf{P}_{+},\ \mathsf{P}_{\text{lit}},\ \mathsf{P}_{\varepsilon}).$$

For the second state $b \cdot t = b\,(b + \varepsilon)$:

$$\mathit{pd}\big(b\,(b + \varepsilon), \mathtt{b}\big) = \{\, \mathit{smartC}(\varepsilon,\, b + \varepsilon) \,\} = \{b + \varepsilon\} \qquad (\mathsf{P}_{\text{seq}}).$$

Only the left factor b is derived, as it is a non-nullable literal. The union gives:

$$S_2 = \{\varepsilon\} \cup \{b + \varepsilon\} = \{\, \varepsilon,\ \ b + \varepsilon \,\}.$$

Acceptance. Both states in S_2 are nullable, ε by N_{ε} and $b + \varepsilon$ by N_{+} and N_{ε}, so `match_pderiv("ab",` r_0`)` returns `true` and $\mathtt{ab} \in L(r_0)$, agreeing with Section 3.2.

4 Matching and Parsing

Chapter 3 established matching with a Boolean answer: $w \in L(r)$. A matcher only answers whether the input is accepted by the regular expression. Parsing identifies the evidence for the match and *parse trees* act as their structured witnesses.

4.1 Matching: A Boolean Answer

Consider $r = (a + ab)(b + \varepsilon)$ and $w = \texttt{ab}$. The matcher can determine $w \in L(r)$ using the algorithms of Chapter 3. There are, however, two ways in which r can match w:

$$\underbrace{a}_{a+ab}\ \underbrace{b}_{b+\varepsilon} \quad \text{and} \quad \underbrace{\texttt{ab}}_{a+ab}\ \underbrace{\varepsilon}_{b+\varepsilon} \tag{4.1}$$

Both choices establish the same membership result, but assign the input differently to the subexpressions of r. The Boolean result alone cannot distinguish these cases, so the matcher cannot reveal which decomposition was chosen.

Parsing makes this additional structure explicit, by showing which alternatives were chosen and which parts of the input were consumed by the corresponding subexpressions.

4.2 Parsing: Parse Trees as Matching Evidence

Parse Tree. A *parse tree* provides a structured witness for such a derivation. Sulzmann and Lu view regular expressions as types and *parse trees* as their values [Sulzmann and Lu, 2014]. The judgment $\vdash v : r$ states that v is a valid parse tree for r, recording which alternatives were chosen and how each part of the input was consumed during matching:

$$\frac{}{\vdash () : \varepsilon}\ (\mathsf{T}_\varepsilon) \qquad \frac{\vdash v_1 : r_1}{\vdash \mathit{Left}\ v_1 : r_1 + r_2}\ (\mathsf{T}^L_+)$$

$$\frac{l \in \Sigma}{\vdash l : l}\ (\mathsf{T}_{\text{lit}}) \qquad \frac{\vdash v_2 : r_2}{\vdash \mathit{Right}\ v_2 : r_1 + r_2}\ (\mathsf{T}^R_+) \tag{4.2}$$

$$\frac{\vdash v_1 : r_1 \qquad \vdash v_2 : r_2}{\vdash (v_1, v_2) : r_1 r_2}\ (\mathsf{T}_{\text{seq}}) \qquad \frac{}{\vdash [\,] : r^*}\ (\mathsf{T}^0_*)$$

$$\frac{\vdash v : r \qquad \vdash vs : r^*}{\vdash (v : vs) : r^*}\ (\mathsf{T}_*)$$

In Rust. Figure 4.1 shows the parse-tree type, with one variant for each constructor.

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ParseTree {
    Empty,                                  // ()
    Char(char),                             // l
    Pair(Box<ParseTree>, Box<ParseTree>),   // (v1, v2)
    Left(Box<ParseTree>),                   // Left v
    Right(Box<ParseTree>),                  // Right v
    Star(Vec<ParseTree>),                   // [v1, ..., vn]
}
```

Figure 4.1: The parse tree type, representing the structure of a parse.

The definitions above translate directly to Rust, except for the star case. $\mathsf{T}\varepsilon$ corresponds to `Empty` for the empty parse, T_{lit} to `Char` for a literal, T_{seq} to `Pair` for a concatenation of parse trees, and T_{+}^{L} and T_{+}^{R} to `Left` and `Right`, recording the chosen branch of an alternation. Both star rules are represented by `Star(vec![])`.

Flattening. The typing judgment describes the structure of a match, but a parse tree must also be related to the word that it matches. For this purpose, they define a *flattening* function $|v|$ that maps a parse tree to the word represented by its structure:

$$\begin{aligned} |()| &= \varepsilon, & |(v_1, v_2)| &= |v_1||v_2|, \\ |l| &= l, & |\mathit{Left}\ v| &= |v|, \\ |[]| &= \varepsilon & |\mathit{Right}\ v| &= |v|, \\ & & |v : vs| &= |v||vs|. \end{aligned} \tag{4.3}$$

The flattening function directly connects parse trees to $L(r)$, and thus validates that a parse tree witnesses membership in $L(r)$, serving as an correctness check:

$$L(r) = \{\, |v| \mid\!\vdash v : r \,\}. \tag{4.4}$$

In Rust. Figure 4.2 shows how `flatten` recursively traverses the parse tree and concatenates its leaf characters. The function borrows the tree to allow subsequent inspection and comparison. The `Pair` case concatenates the results of flattening both children, while `Left` and `Right` delegate to their child, and `Star` concatenates the results for each element.

```
pub fn flatten(v: &ParseTree) -> String {
    match v {
        ParseTree::Empty        => String::new(),
        ParseTree::Char(c)      => c.to_string(),
        ParseTree::Pair(v1, v2) => format!("{}{}", flatten(v1), flatten(v2)),
        ParseTree::Left(v)      => flatten(v),
        ParseTree::Right(v)     => flatten(v),
        ParseTree::Star(vs)     => vs.iter().map(flatten).collect(),
    }
}
```

Figure 4.2: The `flatten` function extracts the matched word from a parse tree.

Unambiguous example. Consider $r = ab$ and $w = \texttt{ab}$. Since each subexpression is a literal, the rule T_{lit} gives each literal a unique witness. Then T_{seq} requires the parse tree for a to be followed by the parse tree for b, combining them into a parse tree for ab:

$$\frac{a \in \Sigma}{\vdash a : a}\,(\mathsf{T}_{\text{lit}}) \qquad \frac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}}) \qquad \frac{\vdash a : a \qquad \vdash b : b}{\vdash (a, b) : ab}\,(\mathsf{T}_{\text{seq}}) \tag{4.5}$$

The parse tree $v = (a, b)$ flattens to $|(a, b)| = \texttt{ab}$.

Ambiguous Example. Recall $r = (a + ab)(b + \varepsilon)$ and $w = \texttt{ab}$ from Section 4.1. The word admits two distinct decompositions, corresponding to different parse trees.

The first decomposition uses the right alternatives of both subexpressions:

$$\frac{a \in \Sigma}{\vdash a : a}\,(\mathsf{T}_{\text{lit}}) \qquad \frac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}})$$

$$\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}\,(\mathsf{T}_{\text{seq}})$$

$$\underbrace{\frac{\vdash (a, b) : ab}{\vdash \mathit{Right}(a, b) : a + ab}\,(\mathsf{T}^R_+)}_{a + ab} \qquad \underbrace{\frac{\frac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}}) \qquad \vdash () : \varepsilon\,(\mathsf{T}_\varepsilon)}{\vdash \mathit{Right}() : b + \varepsilon}\,(\mathsf{T}^R_+)}_{b + \varepsilon}$$

By T_{seq}, these derivations combine into the parse tree:

$$\frac{\dfrac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \mathit{Right}(a, b) : a + ab} \qquad \frac{\vdash () : \varepsilon}{\vdash \mathit{Right}() : b + \varepsilon} \tag{4.6}$$

$$v_1 = \Big(\mathit{Right}(a, b), \mathit{Right}()\Big), \qquad |v_1| = \texttt{ab}.$$

The second decomposition uses the left alternatives of both subexpressions:

$$\frac{a \in \Sigma}{\vdash a : a}\,(\mathsf{T}_{\text{lit}}) \qquad \frac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}})$$

$$\underbrace{\frac{\vdash a : a}{\vdash \mathit{Left}\, a : a + ab}\,(\mathsf{T}^L_+)}_{a + ab} \qquad \underbrace{\frac{\vdash b : b}{\vdash \mathit{Left}\, b : b + \varepsilon}\,(\mathsf{T}^L_+)}_{b + \varepsilon}$$

By T_{seq}, these derivations combine into the second parse tree:

$$\frac{\vdash a : a}{\vdash \mathit{Left}\, a : a + ab} \qquad \frac{\vdash b : b}{\vdash \mathit{Left}\, b : b + \varepsilon} \tag{4.7}$$

$$v_2 = \Big(\mathit{Left}\, a, \mathit{Left}\, b\Big), \qquad |v_2| = \texttt{ab}.$$

Both trees are valid parse trees for the same expression and word, yet they differ in structure. The typing judgment alone does not determine which parse tree should be selected. A *disambiguation policy* specifies this choice.

5 Derivative-Based Parser

Chapter 3 showed how a Brzozowski-derivative matcher determines whether $w \in L(r)$ and Chapter 4 showed that a matching word may admit multiple valid parse trees. A parser must therefore select a single parse tree according to a disambiguation policy.

Following Sulzmann and Lu [Sulzmann and Lu, 2014], this chapter presents two derivative-based methods for constructing POSIX parse trees. The direct method performs a forward derivative pass followed by a backward pass for tree reconstruction. The bit-coded method records parse-tree information during the forward pass, avoiding a separate backward pass.

5.1 The Standard Derivative-Based Parser

Consider the example $r = (a + ab)(b + \varepsilon)$ with input $w = \texttt{ab}$. Chapter 4 showed that it is ambiguous, as the word admits two parse trees. The question here is whether a parse tree can be recovered from the final derivative:

$$\begin{aligned} r &\xrightarrow{a} (\varepsilon + \varepsilon b)(b + \varepsilon) \\ &\xrightarrow{b} (\emptyset + (\emptyset b + \varepsilon))(b + \varepsilon) + (\varepsilon + \emptyset). \end{aligned} \tag{5.1}$$

The derivation was already computed in Section 3.2, shown here unsimplified. The final derivative is nullable, so $w \in L(r)$ and both disjuncts accept the empty word. Its first disjunct, $(\emptyset + (\emptyset b + \varepsilon))(b + \varepsilon)$, reflects $ab \cdot \varepsilon$, and its second, $(\varepsilon + \emptyset)$, reflects $a \cdot b$.

Since a derivative step only records the remaining input language and not how consumed characters were matched, the final derivative alone does not show which decomposition was selected, and cannot be used to recover the parse tree. However, the derivation sequence itself contains this information. Each step consumed one character and is invertible given that character, so reversing the steps recovers the tree, as shown in Figure 5.1:

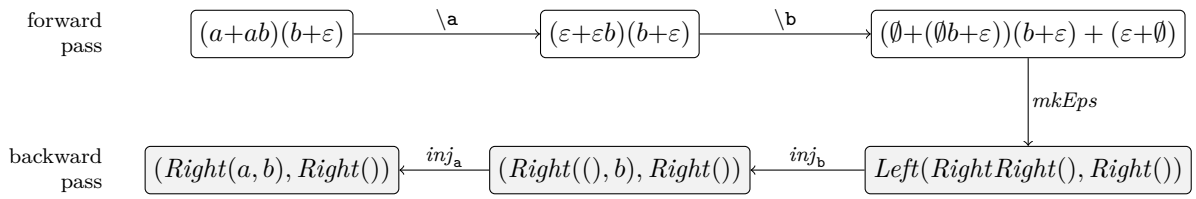


Figure 5.1: Overview of the standard derivative-based parser on the ambiguous example. The forward pass computes derivatives, while the backward pass uses *mkEps* to construct a parse tree for the final nullable derivative and *inj* to reconstruct the original parse tree.

The two-pass pipeline. The direct construction uses two passes. The forward pass applies `deriv` (Figure 3.3) and `nullable` (Figure 3.2) to process the input.

The backward pass applies two new functions to walk the derivation sequence in reverse and reconstruct the parse tree. First *mkEps* builds a parse tree for the empty word from the final nullable expression. Then injection steps are applied, with *inj* reinserting one consumed character into the parse tree, moving in w from right to left. Each injection step takes the current parse tree, the corresponding derivative step, and its consumed character, and returns the parse tree for the preceding expression.

As shown in Figure 5.1 after the first injection of b, the tree is $(\mathit{Right}((), b), \mathit{Right}())$, where $()$ marks the position still to be filled. After the second injection of a, that position is replaced by the subtree $\mathit{Right}(a, b)$, giving the complete parse tree $(\mathit{Right}(a, b), \mathit{Right}())$.

In the following the operations for the standard derivative parser are shown in more detail.

5.1.1 Building the Empty-Word Tree

The backward pass begins with *mkEps*, which reads a parse tree for the empty word off the final nullable expression. The function follows [Sulzmann and Lu, 2014, Fig. 3] and is defined recursively:

$$\begin{aligned}
\mathit{mkEps}_{\varepsilon} &= () && (\mathsf{E}_{\varepsilon}) \\
\mathit{mkEps}_{r^*} &= [\,] && (\mathsf{E}_{*}) \\
\mathit{mkEps}_{r_1 r_2} &= (\mathit{mkEps}_{r_1}, \mathit{mkEps}_{r_2}) && (\mathsf{E}_{\mathrm{seq}}) \\
\mathit{mkEps}_{r_1+r_2} &= \begin{cases} \mathit{Left}\ \mathit{mkEps}_{r_1} & \text{if } n(r_1) \quad (\mathsf{E}^L_+) \\ \mathit{Right}\ \mathit{mkEps}_{r_2} & \text{otherwise} \quad (\mathsf{E}^R_+) \end{cases}
\end{aligned} \tag{5.2}$$

The first three rules follow from the structure of the expression: $(\mathsf{E}_{\varepsilon})$ gives ε an empty parse tree, (E_{*}) treats a star matching the empty word as zero iterations, and $(\mathsf{E}_{\mathrm{seq}})$ matches a concatenation against ε only when both factors do. No rules are needed for $\emptyset$ or literals, since neither can match the empty word. The alternation rule is where disambiguation occurs. If exactly one of r_1 and r_2 is nullable, that branch is the only possible witness. If both are, (E^L_+) breaks the tie by preferring the left one, following POSIX's leftmost-longest policy [Sulzmann and Lu, 2014]. In the example, the final expression is:

$$(\emptyset + (\emptyset b + \varepsilon))(b + \varepsilon) + (\varepsilon + \emptyset).$$

At top-level $+$, both summands are nullable, so (E^L_+) applies and picks the left one. Within the left summand's first factor $\emptyset + (\emptyset b + \varepsilon)$, only the right alternative is nullable, so (E^R_+) applies, and within it only the right sub-alternative ε is nullable, so (E^R_+) applies again, contributing $\mathit{RightRight}()$. The second factor $b + \varepsilon$ contributes $\mathit{Right}()$ via (E^R_+), giving:

$$\mathit{Left}(\mathit{RightRight}(), \mathit{Right}()).$$

Since *mkEps* handles disambiguation, the forward pass cannot simplify. It must keep the branch that consumed a, or injection cannot recover $\mathit{Right}(a, b)$. Simplifying gives:

$$r \xrightarrow{a} (\varepsilon + \varepsilon b)(b + \varepsilon) \xrightarrow{\text{simp}} (\varepsilon + b)(b + \varepsilon) \xrightarrow{b} (\emptyset + \varepsilon)(b + \varepsilon) + (\varepsilon + \emptyset) \xrightarrow{\text{simp}} (b + \varepsilon) + \varepsilon,$$

This still witnesses $w \in L(r)$, but $(\emptyset + \varepsilon) \to \varepsilon$ and $(\varepsilon + \emptyset) \to \varepsilon$ erase the pair structure holding both characters. *mkEps* on $(b + \varepsilon) + \varepsilon$ gives $\mathit{Left}(\mathit{Right}(()))$ and injecting b fills that one empty leaf, leaving none for a, so $\mathit{Right}(a, b)$ is unrecoverable.

In Rust. The recursive definition of *mkEps* (5.2) is implemented directly in Rust, as shown in Figure 5.2, following the same cases for each expression form.

```
pub fn mk_eps(r: &Regex) -> ParseTree {
    match r {
        Regex::Eps          => ParseTree::Empty,
        Regex::Star(_)      => ParseTree::Star(Vec::new()),
        Regex::Seq(r1, r2) => ParseTree::Pair(
                               Box::new(mk_eps(r1)), Box::new(mk_eps(r2))),
        Regex::Alt(r1, r2) =>
            if nullable(r1)   {ParseTree::Left(Box::new(mk_eps(r1)))}
            else              {ParseTree::Right(Box::new(mk_eps(r2)))},
        Regex::Lit(c)       => panic!("mk_eps called on Lit('{}')", c),
        Regex::Phi          => panic!("mk_eps called on Phi"),
    }
}
```

Figure 5.2: `mk_eps`: construct a parse tree for the empty word.

The expression r is borrowed rather than consumed because the backward pass needs it again during injection. The alternation arm encodes disambiguation as a conditional, examining `r2` only when `nullable(r1)` is false. The panic arms enforce the precondition that *mkEps* applies only to nullable expressions, rejecting the non-nullable $\emptyset$ and literals.

5.1.2 Reinserting Characters via Injection

After *mkEps* constructed the parse tree v for the empty input, the reconstruction starts from this initial tree and proceeds backwards through the derivation sequence. Each application of *inj* takes the current tree, the corresponding derivative step, and its consumed character, returning a tree for the preceding expression. *inj* recursively follows the structure of the residual until it reaches the subexpression containing the literal that consumed the character. It reinserts the character there and then unwinds to rebuild the surrounding tree. After all characters have been reinserted, the parse tree for the original expression and input is recovered. The injection function is defined by the following rules [Sulzmann and Lu, 2014, Fig. 3]:

$$
\begin{aligned}
inj_{x\backslash x}() &= x && (\mathsf{I}_{\text{lit}}) \\
inj_{(r_1+r_2)\backslash x}(Left\ v_1) &= Left\ (inj_{r_1\backslash x}\ v_1) && (\mathsf{I}^L_+) \\
inj_{(r_1+r_2)\backslash x}(Right\ v_2) &= Right\ (inj_{r_2\backslash x}\ v_2) && (\mathsf{I}^R_+) \\
inj_{r^*\backslash x}(v, vs) &= (inj_{r\backslash x}\ v) : vs && (\mathsf{I}_*) \\
inj_{(r_1 r_2)\backslash x}(v_1, v_2) &= (inj_{r_1\backslash x}\ v_1,\ v_2) && (\mathsf{I}_{\text{seq}}),\ n(r_1) = \text{false} \\
inj_{(r_1 r_2)\backslash x}(Left(v_1, v_2)) &= (inj_{r_1\backslash x}\ v_1,\ v_2) && (\mathsf{I}^L_{\text{seq}}),\ n(r_1) = \text{true} \\
inj_{(r_1 r_2)\backslash x}(Right\ v_2) &= (mkEps_{r_1},\ inj_{r_2\backslash x}\ v_2) && (\mathsf{I}^R_{\text{seq}}),\ n(r_1) = \text{true}
\end{aligned}
\tag{5.3}
$$

Rules $(\mathsf{I}_{\text{lit}})$ and (I_*) invert their derivative rules directly, reconstructing the consumed character and the current iteration, the latter prepended to the remaining ones. Since the consumed character lies in exactly one branch of an alternation, the incoming tree is either $Left\ v_1$ or $Right\ v_2$. Rule (I^L_+) recurses into v_1 and rebuilds the *Left* node, and (I^R_+) recurses

into v_2 and rebuilds the *Right* node. The concatenation rules cover the three cases of the sequence derivative. If $n(r_1) = \text{false}$, r_1 cannot match ε and must consume x, so (I_{seq}) recurses into v_1. If $n(r_1) = \text{true}$, the derivative recorded which component consumed x as a tag: ($\mathsf{I}^L_{\text{seq}}$) handles an incoming $\mathit{Left}(v_1, v_2)$ tag and recurses into v_1, while ($\mathsf{I}^R_{\text{seq}}$) handles a *Right* v_2 tag by reconstructing r_1's empty match with mkEps_{r_1} and recursing into v_2. Since $\emptyset$ consumes no characters, *inj* is never applied with $\emptyset$ as the residual.

```
pub fn inject(r: &Regex, l: char, v: ParseTree) -> ParseTree {
    match r {
        Regex::Lit(c) => { assert!(*c == l);
            ParseTree::Char(l) }
        Regex::Alt(r1, r2) => match v {
            ParseTree::Left(v1) =>
                ParseTree::Left(Box::new(inject(r1, l, *v1))),
            ParseTree::Right(v2) =>
                ParseTree::Right(Box::new(inject(r2, l, *v2))),
            _ => panic!("inject/Alt: want Left|Right"),
        },
        Regex::Star(r1) => match v {
            ParseTree::Pair(v1, vs) => {
                let mut it = vec![inject(r1, l, *v1)];
                if let ParseTree::Star(rest) = *vs { it.extend(rest); }
                ParseTree::Star(it)
            }
            _ => panic!("inject/Star: want Pair"),
        },
        Regex::Seq(r1, r2) => if !nullable(r1) {
            match v {
                ParseTree::Pair(v1, v2) => ParseTree::Pair(
                    Box::new(inject(r1, l, *v1)), v2),
                _ => panic!("inject/Seq: want Pair"),
            }
        } else {
            match v {
                ParseTree::Left(vp) => match *vp {
                    ParseTree::Pair(v1, v2) => ParseTree::Pair(
                        Box::new(inject(r1, l, *v1)), v2),
                    o => panic!("inject/SeqL: want Pair, got {:?}", o),
                },
                ParseTree::Right(v2) => ParseTree::Pair(
                    Box::new(mk_eps(r1)),
                    Box::new(inject(r2, l, *v2))),
                _ => panic!("inject/Seq: want Left|Right"),
            }
        },
        Regex::Eps | Regex::Phi => panic!("inject called on Eps/Phi"),
    }
}
```

Figure 5.3: `inject`: reinsert a character into a parse tree.

In Rust. `inject(r, l, v)` takes three arguments because *inj* is indexed by both an expression and a character. Figure 5.3 shows the complete implementation.

The function matches on the expression `r` first and on the tree `v` second, following the order in which (5.3) indexes its rules. This order matters because a tree tag is interpreted relative to the expression that produced it. Under an alternation, `Left` selects the left disjunct, while under a nullable concatenation it indicates that the derivative continued inside the left operand. The `Lit` arm reconstructs a leaf after asserting that the consumed character matches, and the `Star` arm inverts (I_*), reinserting the character into the current iteration and prepending it to the remaining ones.

The expression `r` is borrowed and read only, while the tree `v` is consumed and decomposed. In the concatenation arms where the right operand was not derived, `v2` moves directly into the new pair, since (5.3) leaves it unchanged. The `Right` arm of concatenation is the only concatenation case that synthesizes content, calling `mk_eps(r1)` because the derivative skipped `r1` and the forward pass produced no tree for that half of the pair.

Calling `nullable(r1)` in the `Seq` arm recomputes a decision the forward pass already made when it chose between D_{seq} and $\mathsf{D}^n_{\text{seq}}$. However, carrying this as a flag at every step would require additional storage, while `nullable` is cheap to evaluate on demand.

5.1.3 Parser

The standard parser has two passes. The forward pass stores each residual and the backward pass applies *mkEps* then *inj* in reverse. The split is needed because *inj* requires the pre-character residual, which only the stored sequence provides:

$$\begin{aligned} &r_0 = r, && r_i = r_{i-1}\backslash x_i && (1 \le i \le n),\\ &v_n = \mathit{mkEps}(r_n), && v_i = \mathit{inj}_{r_{i-1}\backslash x_i}(x_i,\ v_{i+1}) && (0 \le i < n). \end{aligned} \tag{5.4}$$

Every r_i is kept because each injection step needs r_{i-1}; re-deriving it would duplicate the forward pass. The vector holds the entire sequence $r_0, \ldots, r_n$, not just the final residual.

In Rust. Figure 5.4 implements (5.4). The forward loop fills the `Vec expressions` with $r_0, \ldots, r_n$, the nullability check returns early if r_n is not nullable, and the backward loop walks the vector in reverse, applying `inject` to each residual and its character.

```
pub fn parse_deriv_std_loop(input: &str, r: &Regex) -> Option<ParseTree>{
    let chars: Vec<char> = input.chars().collect();
    let mut expressions = Vec::with_capacity(chars.len() + 1);
    expressions.push(r.clone());
    for &c in &chars {
        expressions.push(deriv(expressions.last().unwrap(), c));
    }
    if !nullable(expressions.last().unwrap()) { return None; }
    let mut tree = mk_eps(expressions.last().unwrap());
    for i in (0..chars.len()).rev() {
        tree = inject(&expressions[i], chars[i], tree);
    }
    Some(tree)
}
```

Figure 5.4: `parse_deriv_std_loop`: the standard parser.

5.1.4 Worked Example: Applying the Standard Parser

Apply the standard parser to the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ on input $\mathtt{ab}$.

Forward pass. There are two derivative steps, one per input symbol. After deriving $\mathtt{a}$, the left factor $\varepsilon + \varepsilon b$ is nullable, so the second derivation on $\mathtt{b}$ uses $\mathsf{D}^n_{\mathrm{seq}}$ rather than $\mathsf{D}_{\mathrm{seq}}$:

$$\begin{aligned} r_0 = (a + ab)(b + \varepsilon) &\xrightarrow[\mathsf{D}_{\mathrm{seq}}]{a} r_1 = (\varepsilon + \varepsilon b)(b + \varepsilon) \\ &\xrightarrow[\mathsf{D}^n_{\mathrm{seq}}]{b} r_2 = \underbrace{(\emptyset + (\emptyset b + \varepsilon))(b + \varepsilon)}_{X} + \underbrace{(\varepsilon + \emptyset)}_{Y}. \end{aligned}$$

The final expression $r_2 = X + Y$ is nullable, and both disjuncts accept the empty word. X records $ab \cdot \varepsilon$ (the first factor matched ab, the second ε); Y records $a \cdot b$ (the first matched a, the second b).

Backward pass: mkEps. The backward pass begins by building the parse tree for the empty word, $mkEps(r_2)$. Since $r_2 = X + Y$ and both disjuncts are nullable, E^L_+ selects the left one:

$$mkEps(r_2) = Left\Big(mkEps(X)\Big) \qquad (\mathsf{E}^L_+),$$

where $X = (\emptyset + (\emptyset b + \varepsilon))(b + \varepsilon)$. Since X is a concatenation, $\mathsf{E}_{\mathrm{seq}}$ combines the trees for its two factors:

$$mkEps(X) \xrightarrow{\mathsf{E}_{\mathrm{seq}}} \Big(mkEps(\emptyset + (\emptyset b + \varepsilon)),\ mkEps(b + \varepsilon)\Big).$$

The first factor selects ε by applying E^R_+ twice, since $\emptyset$ and $\emptyset b$ are not nullable; the second factor selects ε by E^R_+, since b is not nullable. These give

$$\begin{aligned} mkEps(\emptyset + (\emptyset b + \varepsilon)) &\xrightarrow{\mathsf{E}^R_+} Right\ mkEps(\emptyset b + \varepsilon) \\ &\xrightarrow{\mathsf{E}^R_+} Right\ Right\ mkEps(\varepsilon) \\ &\xrightarrow{\mathsf{E}_\varepsilon} Right\ Right\ () \end{aligned} \qquad \begin{aligned} mkEps(b + \varepsilon) &\xrightarrow{\mathsf{E}^R_+} Right\ mkEps(\varepsilon) \\ &\xrightarrow{\mathsf{E}_\varepsilon} Right\ () \end{aligned}$$

and hence

$$v_2 = mkEps(r_2) = Left\Big(Right\ Right\ (), Right\ ()\Big).$$

Backward pass: Inj. Reinsertion begins with $\mathtt{b}$ and maps v_2 to a tree for $r_1 = (\varepsilon + \varepsilon b)(b + \varepsilon)$. The *Left* tag of v_2 sends $\mathsf{I}^L_{\mathrm{seq}}$ into the first factor $\varepsilon + \varepsilon b$, leaving the second factor's tree $Right\ ()$ unchanged. Inside $\varepsilon + \varepsilon b$, I^R_+ selects the ab branch; since ε is nullable, $\mathsf{I}^R_{\mathrm{seq}}$ supplies the empty tree for ε before $\mathsf{I}_{\mathrm{lit}}$ reinserts $\mathtt{b}$:

$$\begin{aligned} inj_{r_1 \backslash \mathtt{b}}(v_2) &= inj_{r_1 \backslash \mathtt{b}}\Big(\underbrace{Left(Right\ Right\ (),\ Right\ ())}_{v_2}\Big) \\ &\xrightarrow{\mathsf{I}^L_{\mathrm{seq}}} \Big(inj_{(\varepsilon + \varepsilon b) \backslash \mathtt{b}}(Right\ Right\ ()),\ Right\ ()\Big) \\ &\xrightarrow{\mathsf{I}^R_+} \Big(Right\ inj_{\varepsilon b \backslash \mathtt{b}}(Right\ ()),\ Right\ ()\Big) \\ &\xrightarrow{\mathsf{I}^R_{\mathrm{seq}}} \Big(Right\ (mkEps(\varepsilon),\ inj_{b \backslash \mathtt{b}}()),\ Right\ ()\Big) \\ &\xrightarrow{\mathsf{E}_\varepsilon,\ \mathsf{I}_{\mathrm{lit}}} \underbrace{\Big(Right((), b),\ Right\ ()\Big)}_{v_1}. \end{aligned}$$

Reinsertion of $\mathtt{a}$ maps v_1 to a tree for $r_0 = (a + ab)(b + \varepsilon)$. The outer sequence's first factor $a+ab$ is not nullable, so I_{seq} fires directly on v_1's first component, leaving the second component $Right\,()$ untouched. That first component, $Right((), b)$, carries a $Right$ tag, so I_+^R selects ab. Inside $ab = Seq(a, b)$ the first factor a is again not nullable, so I_{seq} fires once more and recurses into a, where I_{lit} reinserts the character:

$$
\begin{aligned}
inj_{r_0\backslash\mathtt{a}}(v_1) = inj_{r_0\backslash\mathtt{a}}\Big(\underbrace{(Right((), b),\ Right\,())}_{v_1}\Big) \\
&\xrightarrow{\mathsf{I}_{\text{seq}}} \big(inj_{(a+ab)\backslash\mathtt{a}}(Right((), b)),\ Right\,()\big) \\
&\xrightarrow{\mathsf{I}_+^R} \big(Right\, inj_{ab\backslash\mathtt{a}}(((), b)),\ Right\,()\big) \\
&\xrightarrow{\mathsf{I}_{\text{seq}}} \big(Right\,(inj_{a\backslash\mathtt{a}}(()),\ b),\ Right\,()\big) \\
&\xrightarrow{\mathsf{I}_{\text{lit}}} \underbrace{\big(Right(a, b),\ Right\,()\big)}_{v_0}.
\end{aligned}
$$

The implementation stores r_0, r_1, and r_2 in its vector. One entry per character, plus the initial expression, kept for the backward pass to reconstruct from. The tree it returns is $(Right(a, b), Right())$, which flattens to $\mathtt{ab}$ and matches the input.

5.2 The Bit-Coded Derivative-Based Parser

The direct construction of Sulzmann and Lu [Sulzmann and Lu, 2014] reconstructs the tree only after the full input, since each injection step needs the preceding expression. Their bit-coded optimization records this information during a single pass instead.

Each subexpression carries the bit-sequence it has committed to so far, written $bs@r$. P and Q denote the annotated factors for $r_0 = (a + ab)(b + \varepsilon)$. Deriving $w = \mathtt{ab}$ gives:

$$
\begin{aligned}
ri_0 = [\,]@\Big(\underbrace{[0]@a \oplus [1]@(a\,b)}_{P} \cdot \underbrace{[0]@b \oplus [1]@\varepsilon}_{Q}\Big) \\
\xrightarrow{a} ri_1 = [\,]@\Big(\underbrace{[0]@\varepsilon \oplus [1]@b}_{P'} \cdot Q\Big) \\
\xrightarrow{b} ri_2 = [\,]@\Big([1]@Q \ \oplus\ [0, 0]@\varepsilon\Big).
\end{aligned}
\tag{5.5}
$$

Figure 5.5 shows this derivation together with the extraction step it feeds into:

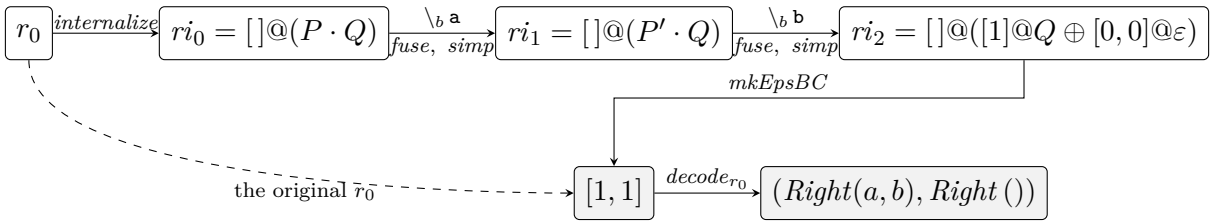


Figure 5.5: The bit-coded parser on the ambiguous example. *internalize* introduces the bits, derivative threads them forward, and *mkEpsBC* and $decode_{r_0}$ extract and read them against r_0.

The bit-coded pipeline. *internalize* tags each alternation branch with a bit that records its preference, producing $ri_0 = []@(P \cdot Q)$ with $P = [0]@a \oplus [1]@(a\,b)$ and $Q = [0]@b \oplus [1]@\varepsilon$, where 0 selects the left branch and 1 the right.

Deriving by `a` extracts P's leading a, giving P' while Q is untouched; deriving by `b` consumes the rest. Since P''s ε branch is nullable, both decompositions survive: in the first, the derivative continues inside P', which finishes and folds its bit 1 onto Q; in the second, Q matches `b` instead, and P''s completion bit fuses with Q's match into $[0,0]@\varepsilon$. This yields $ri_2 = []@([1]@Q \oplus [0,0]@\varepsilon)$.

Unlike the standard construction, the bit-coded construction requires simplification to prevent unbounded growth of annotated expressions. The function *simp*, from [Sulzmann and Lu, 2014, Fig. 6], discards dead branches and collapses finished subexpressions, while *fuse* preserves bits on surviving nodes, as when P collapses into P' and then onto Q.

Once `ab` is exhausted, *mkEpsBC* walks ri_2 the way *mkEps* walks a plain expression, taking the leftmost nullable branch at each remaining choice: $[1]@Q$ is nullable, so it contributes its 1, and inside Q the ε branch is nullable, contributing a second 1, giving $bits = [1,1]$. $decode_{r_0}$ then replays these two bits against the original r_0. The first 1 selects $a+ab$'s right branch and the second selects $b+\varepsilon$'s right branch, giving the tree $(Right(a,b), Right\,())$.

This shows that the bit-coded construction reconstructs the parse tree in a single step, while the direct construction reverses residuals step by step.

5.2.1 Annotated Expressions

Building parse-tree information during the forward pass extends the derivative computation with bit-sequence annotations, propagated by the bit-threaded derivative $ri\backslash_b l$. An annotated expression ri extends r by associating each non-$\emptyset$ node with a bit-sequence bs recording the reconstruction choices made during the forward pass.

$$ri ::= \emptyset \mid (bs@\varepsilon) \mid (bs@x) \mid (bs@ri_1 \oplus ri_2) \mid (bs@ri_1\,ri_2) \mid (bs@ri^*). \tag{5.6}$$

In Rust. Figure 5.6 defines `ARegex`, the annotated counterpart of `Regex`. Every variant except `Phi` carries a leading `Bits` field (cf. (5.6)). `Bits` is a persistent, `Rc`-backed cons list, not a `Vec<bool>`: prepending a prefix costs only the prefix, independent of the accumulated history. A `Vec<bool>` would copy the entire prefix on every derivative step, making an n-character run cost $O(n^2)$. the persistent representation keeps it $O(n)$.

```
pub enum ARegex {
    Phi,
    Eps(Bits), Lit(Bits, char),
    Alt(Bits, Box<ARegex>, Box<ARegex>),
    Seq(Bits, Box<ARegex>, Box<ARegex>),
    Star(Bits, Box<ARegex>),
}
```

Figure 5.6: The annotated-expression type extends `Regex` with a leading bit-sequence, `Bits`.

All following operations act on this `ARegex`. Since derivative and simplification steps remove nodes while the bits attached to them must survive, each operation needs a way to transfer those bits onto the node that remains. The helper *fuse* provides this transfer.

5.2.2 Fusing Bit-Lists

fuse, defined in [Sulzmann and Lu, 2014, Fig. 5], transfers bits from a node that is about to be removed onto the node that survives in its place. It prepends bs to the top node's annotation bs' without descending into its children, so outer reconstruction choices precede those already recorded in the subtree.

$$\begin{aligned} \mathit{fuse}(bs, \emptyset) &= \emptyset \\ \mathit{fuse}(bs, (bs'@\varepsilon)) &= (bs +\!\!+ bs')@\varepsilon \\ \mathit{fuse}(bs, (bs'@x)) &= (bs +\!\!+ bs')@x \\ \mathit{fuse}(bs, (bs'@ri_1 \oplus ri_2)) &= (bs +\!\!+ bs')@(ri_1 \oplus ri_2) \\ \mathit{fuse}(bs, (bs'@ri_1\, ri_2)) &= (bs +\!\!+ bs')@(ri_1\, ri_2) \\ \mathit{fuse}(bs, (bs'@ri^*)) &= (bs +\!\!+ bs')@ri^*. \end{aligned} \tag{5.7}$$

In Rust. Figure 5.7 implements the six rules of (5.7), one per variant. The helper `Bits::prepend` corresponds to the concatenation $+\!\!+$ and places `bs` before the node's existing bits:

```
pub fn fuse(bs: &[bool], ri: ARegex) -> ARegex {
    if bs.is_empty() { return ri; }
    match ri {
        ARegex::Phi => ARegex::Phi,
        ARegex::Eps(p) => ARegex::Eps(p.prepend(bs)),
        ARegex::Lit(p, c) => ARegex::Lit(p.prepend(bs), c),
        ARegex::Alt(p, r1, r2) => ARegex::Alt(p.prepend(bs), r1, r2),
        ARegex::Seq(p, r1, r2) => ARegex::Seq(p.prepend(bs), r1, r2),
        ARegex::Star(p, r) => ARegex::Star(p.prepend(bs), r),
    }
}

impl Bits {
    // prefix ++ self, O(prefix.len()) regardless of how long self
    // already is -- the existing chain is shared, not copied.
    pub fn prepend(&self, prefix: &[bool]) -> Bits {
        if prefix.is_empty() { return self.clone(); }
        Bits(Rc::new(Node::Prepend(Rc::from(prefix), self.0.clone())))
    }
}
```

Figure 5.7: `fuse`: prepend a bit-list to the annotation of the top node.

The `Phi` arm is the only case that returns without calling `prepend`, because `Phi` represents the empty language and carries no annotation slot, so there is no bit-list for `fuse` to prepend to. Every other arm rebuilds the top node with a combined annotation and moves its two children, or single child in the star case, unchanged. Since `fuse` prepends `bs` only to the top node's annotation, the bits carried by its children are not touched, and the operation rewrites exactly one node regardless of how large the subtree below it is.

The function is therefore non-recursive, and no subtree needs to be copied. The implementation realizes this directly, since `ri` is consumed by value and its untouched children

move into the rebuilt node without a clone. `Bits::prepend` wraps the node's existing chain in a new `Rc`, sharing rather than copying it: the allocation is proportional only to `bs`'s length, independent of how many bits the node already carries. This is why `Bits` replaces a plain `Vec<bool>` here: with a `Vec`, `prepend` would have to copy the existing content on every call, a cost proportional to the combined length of the two bit-lists – the node's own history can grow with every character consumed so far, which would make a single `fuse` call as expensive as the whole parse up to that point. An empty `bs` is the one case that still skips any work, since prepending an empty prefix returns the node's existing chain unchanged.

5.2.3 Internalizing Expressions

The forward pass operates on annotated expressions, but the input is a plain expression, so r must first be converted into its annotated counterpart ri. The function *internalize*, following [Sulzmann and Lu, 2014, Fig. 5], performs this conversion recursively, and it is the only step that introduces new bits:

$$\begin{aligned}
\mathit{internalize}(\emptyset) &= \emptyset \\
\mathit{internalize}(\varepsilon) &= [\,]@\varepsilon \\
\mathit{internalize}(x) &= [\,]@x \\
\mathit{internalize}(r_1 + r_2) &= [\,]@\Big(\mathit{fuse}([0], \mathit{internalize}(r_1)) \oplus \mathit{fuse}([1], \mathit{internalize}(r_2))\Big) \\
\mathit{internalize}(r_1\, r_2) &= [\,]@\Big(\mathit{internalize}(r_1)\ \mathit{internalize}(r_2)\Big) \\
\mathit{internalize}(r^*) &= [\,]@\mathit{internalize}(r)^*.
\end{aligned} \tag{5.8}$$

The tags [0] and [1] on the two branches of $r_1 + r_2$ mark the left and right branch respectively, so the decoder can later recover which one was taken. Every other case introduces no new bit and uses only the empty list $[\,]$. The two calls to *fuse* in (5.8) are the only points in the construction where new bits are introduced.

In Rust. Figure 5.8 directly maps the constructors of `Regex` to their annotated counterparts. Each recursive call applies `internalize` to the corresponding subexpression, preserving the structure of the original expression in the annotated representation.

```
pub fn internalize(r: &Regex) -> ARegex {
    match r {
        Regex::Phi => ARegex::Phi,
        Regex::Eps => ARegex::Eps(Bits::empty()),
        Regex::Lit(c) => ARegex::Lit(Bits::empty(), *c),
        Regex::Alt(r1, r2) => {
            let ri1 = fuse(&[false], internalize(r1));
            let ri2 = fuse(&[true], internalize(r2));
            ARegex::Alt(Bits::empty(), Box::new(ri1), Box::new(ri2))
        }
        Regex::Seq(r1, r2) => ARegex::Seq(Bits::empty(),
            Box::new(internalize(r1)), Box::new(internalize(r2))),
        Regex::Star(r1) => ARegex::Star(Bits::empty(),
            Box::new(internalize(r1))),
    }
}
```

Figure 5.8: `internalize` converts a `Regex` into an `ARegex`. Each alternation branch receives a tag recording its side, and the two `fuse` calls are the only points where new bits are introduced.

The `Alt` case calls `fuse` with a non-empty bit-list, tagging the left operand with `[false]` and the right with `[true]` before wrapping them in a `Alt` with an empty outer bit-list. Every later bit originates from one of these two tags and is propagated during derivation.

The `Phi`, `Eps`, `Lit`, `Seq`, and `Star` cases use an empty bit-list, since they introduce no disambiguation choice at this stage. Star iteration choices are instead determined during derivative computation.

5.2.4 Extracting Bits

mkEpsBC, from [Sulzmann and Lu, 2014, Fig. 5], is the bit-list counterpart of *mkEps*. It keeps the left preference and star closing bit, but extracts bits instead of building a tree. Each rule prepends the current node's bits to those returned recursively.

$$
\begin{aligned}
mkEpsBC_{(bs@\varepsilon)} &= bs \\
mkEpsBC_{(bs@ri^*)} &= bs +\!\!+ [1] \\
mkEpsBC_{(bs@ri_1\, ri_2)} &= bs +\!\!+ mkEpsBC_{ri_1} +\!\!+ mkEpsBC_{ri_2} \\
mkEpsBC_{(bs@ri_1 \oplus ri_2)} &= bs +\!\!+ \begin{cases} mkEpsBC_{ri_1} & \varepsilon \in L(ri_1) \\ mkEpsBC_{ri_2} & \text{otherwise.} \end{cases}
\end{aligned}
\tag{5.9}
$$

As with *mkEps*, the rules cover only the four nullable forms. $\emptyset$ and literals cannot match the empty word, so *mkEpsBC* is called only on nullable expressions and both are handled as preconditions.

In Rust. Figure 5.9 implements `mk_eps_bc` alongside `mk_eps` (Figure 5.2), with one difference. Where `mk_eps` builds a `ParseTree`, `mk_eps_bc` concatenates bit-lists, calling `bs.to_vec()` where `mk_eps` would borrow or clone: this is the point where a node's `Bits` chain (Section 5.2.1) is finally walked and materialized into a concrete `Vec<bool>`,

something `fuse` deliberately avoids doing on every derivative step. The `Star` arm also does not recurse, since a star matching the empty word uses zero iterations, so it only appends the closing `true` and never examines the child expression:

```
pub fn mk_eps_bc(ri: &ARegex) -> Vec<bool> {
    match ri {
        ARegex::Phi => panic!("mk_eps_bc called on Phi"),
        ARegex::Lit(_, c) => panic!("mk_eps_bc called on Lit('{}')", c),
        ARegex::Eps(bs) => bs.to_vec(),
        ARegex::Alt(bs, r1, r2) => {
            let mut result = bs.to_vec();
            if nullable_bc(r1) {
                result.extend(mk_eps_bc(r1));
            } else {
                result.extend(mk_eps_bc(r2));
            }
            result
        }
        ARegex::Seq(bs, r1, r2) => {
            let mut result = bs.to_vec();
            result.extend(mk_eps_bc(r1));
            result.extend(mk_eps_bc(r2));
            result
        }
        ARegex::Star(bs, _) => {
            let mut result = bs.to_vec();
            result.push(true);
            result
        }
    }
}
```

Figure 5.9: `mk_eps_bc` implements (5.9). `Phi` and `Lit` panic, since neither can be nullable.

5.2.5 Annotated Derivatives

$ri\backslash_b l$ has the same case structure as the plain derivative (3.6), but extended with bits threaded through the rules. Each rule corresponds to its unannotated counterpart:

$$
\begin{aligned}
\emptyset\backslash_b x &= \emptyset && (\mathsf{B}_\emptyset)\\
(bs@\varepsilon)\backslash_b x &= \emptyset && (\mathsf{B}_\varepsilon)\\
(bs@c)\backslash_b x &= \begin{cases} bs@\varepsilon & c = x \\ \emptyset & \text{otherwise} \end{cases} && (\mathsf{B}_{\text{lit}})\\
(bs@ri_1 \oplus ri_2)\backslash_b x &= (bs@(ri_1\backslash_b x) \oplus (ri_2\backslash_b x)) && (\mathsf{B}_+)\\
(bs@ri_1\ ri_2)\backslash_b x &= (bs@(ri_1\backslash_b x)\ ri_2) && (\mathsf{B}_{\text{seq}}),\ \varepsilon \notin L(ri_1)\\
(bs@ri_1\ ri_2)\backslash_b x &= bs@\Big(([\,]@(ri_1\backslash_b x)\ ri_2) \oplus \mathit{fuse}(\mathit{mkEpsBC}_{ri_1}, ri_2\backslash_b x)\Big) && (\mathsf{B}^n_{\text{seq}}),\ \varepsilon \in L(ri_1)\\
(bs@ri^*)\backslash_b x &= (bs@\mathit{fuse}([0], ri\backslash_b x)\ ([\,]@ri^*)) && (\mathsf{B}_*).
\end{aligned}
\tag{5.10}
$$

The rules B_{+}, B_{seq}, and B_{*} preserve the outer bs, since derivation changes the residual but not the choices already recorded above it.

The nullable sequence case $\mathsf{B}^{n}_{\text{seq}}$ is the exception. Since ri_1 is nullable, x may be matched by either factor, so the rule introduces a new alternation. Its left branch continues into ri_1, while its right branch recurses into ri_2, with the empty match's bits extracted by *mkEpsBC* and fused onto ri_2's derivative, so the choice inside ri_1 survives. This performs during the forward pass what $\mathsf{I}^{R}_{\text{seq}}$ performs during injection. The outer bs attaches to the new alternation, since the alternation as a whole is what $ri_1\ ri_2$ derived into.

In Rust. Figure 5.10 implements (5.10), following the case structure of plain `deriv` with bits threaded through the constructors. The base cases `Phi`, `Eps`, and `Lit` match plain `deriv`, except that `Lit` carries its bit-list onto the resulting `Eps`.

The `Alt` arm derives both operands and rebuilds the alternation with the outer `bs` on the wrapper, not pushed into either branch. This matches B_{+}, where the recorded choice belongs to the alternation itself and derivation does not change it, so the bit stays on the outer constructor until the decoder reads it.

The `Seq` arm reflects B_{seq} and $\mathsf{B}^{n}_{\text{seq}}$. In the non-nullable case, `r1` must consume the character, so only its derivative is needed and `r2` moves directly into the rebuilt sequence. In the nullable case, both alternatives are possible. The bits of `r1`'s empty match are extracted by `mk_eps_bc` *before* `r1` is consumed by `deriv_bc`, since after that call the original `r1` is gone. Those bits are then fused onto the derivative of `r2`, so the choice made inside `r1` reaches the surviving branch. The left branch is wrapped in a fresh `Seq` with an empty bit-list, since the outer `bs` moves to the new `Alt` wrapper, matching $\mathsf{B}^{n}_{\text{seq}}$. Two clones are therefore needed. `r2.clone()` provides one copy for the left branch and one for its own derivative, and `nullable_bc` borrows `r1` before `deriv_bc` consumes it.

The `Star` arm follows the same pattern as the nullable case of `Seq`. The body is needed both for its derivative and for the reconstructed star, so `r.clone()` provides the second copy, and the reconstructed star is wrapped in a fresh `Star` with an empty bit-list for the same reason as the left branch above. The leading `[false]` prepended to the derivative marks one more iteration, matching the outermost bit in B_{*}

```
pub fn deriv_bc(ri: ARegex, l: char) -> ARegex {
    match ri {
        // ...
        ARegex::Alt(bs, r1, r2) => {
            let d1 = deriv_bc(*r1, l);
            let d2 = deriv_bc(*r2, l);
            ARegex::Alt(bs, Box::new(d1), Box::new(d2))
        }
        ARegex::Seq(bs, r1, r2) => {
            if nullable_bc(&r1) {
                let eps_bits = mk_eps_bc(&r1);
                let d1 = deriv_bc(*r1, l);
                let d2 = deriv_bc(*r2.clone(), l);
                let left_branch = ARegex::Seq(Bits::empty(), Box::new(d1), r2);
                let right_branch = fuse(&eps_bits, d2);
                ARegex::Alt(bs, Box::new(left_branch), Box::new(right_branch))
            } else {
                let d1 = deriv_bc(*r1, l);
                ARegex::Seq(bs, Box::new(d1), r2)
            }
        }
        ARegex::Star(bs, r) => {
            let d = deriv_bc(*r.clone(), l);
            let fused = fuse(&[false], d);
            let new_star = ARegex::Star(Bits::empty(), r);
            ARegex::Seq(bs, Box::new(fused), Box::new(new_star))
        }
    }
}
```

Figure 5.10: Annotated derivative `deriv_bc`, implementing $\mathsf{B}_{\emptyset}$, B_{ε}, B_{lit}, B_{+}, $\mathsf{B}_{\text{seq}}/\mathsf{B}^{n}_{\text{seq}}$, B_{*}.

5.2.6 Decoding Bits

$decode_r$ reconstructs a `ParseTree` from the bit sequence of $mkEpsBC$, walking r with one bit per alternation to choose a branch. Its worker $decodeInner$ returns the current tree and the remaining bits, so the recursion continues at the correct position:

$$decode_r(bs) = v \quad \text{if } decodeInner_r(bs) = (v, [\,])$$

$$\begin{aligned}
decodeInner_{\varepsilon}(bs) &= (Empty,\ bs)\\
decodeInner_{x}(bs) &= (Char(x),\ bs)\\
decodeInner_{r_1+r_2}(0{:}bs) &= (Left(v),\ bs_1) \quad \text{if } decodeInner_{r_1}(bs) = (v, bs_1)\\
decodeInner_{r_1+r_2}(1{:}bs) &= (Right(v),\ bs_1) \quad \text{if } decodeInner_{r_2}(bs) = (v, bs_1)\\
decodeInner_{r_1 r_2}(bs) &= (Pair(v_1, v_2),\ bs_2)\\
&\qquad \text{if } decodeInner_{r_1}(bs) = (v_1, bs_1) \text{ and } decodeInner_{r_2}(bs_1) = (v_2, bs_2)\\
decodeInner_{r^*}(1{:}bs) &= (Star([\,]),\ bs)\\
decodeInner_{r^*}(0{:}bs) &= (Star(v{:}vs),\ bs_2)\\
&\qquad \text{if } decodeInner_{r}(bs) = (v, bs_1) \text{ and } decodeInner_{r^*}(bs_1) = (Star(vs), bs_2)
\end{aligned} \tag{5.11}$$

In Rust. Figure 5.11 shows `decode_inner`, the worker behind `decode`. The public entry point `decode` wraps it and asserts that no bits remain. This assertion is the side condition under which $decode_r$ is defined. The worker must consume the entire bit-string.

The worker consumes the `&[bool]` one bit per step using Rust's slice patterns: `[false, rest @ ..]` matches a bit-string whose first element is `false` and binds the remainder to `rest`, and `[true, rest @ ..]` does the same for `true`.

```
fn decode_inner<'a>(r: &Regex, bs: &'a [bool]) -> (ParseTree, &'a [bool]) {
    match r {
        Regex::Eps => (ParseTree::Empty, bs),
        Regex::Lit(c) => (ParseTree::Char(*c), bs),
        Regex::Alt(r1, r2) => match bs {
            [false, rest @ ..] => {
                let (v, remaining) = decode_inner(r1, rest);
                (ParseTree::Left(Box::new(v)), remaining)
            }
            [true, rest @ ..] => {
                let (v, remaining) = decode_inner(r2, rest);
                (ParseTree::Right(Box::new(v)), remaining)
            }
            [] => panic!("decode: unexpected end of bits at Alt"),
        },
        Regex::Seq(r1, r2) => {
            let (v1, after_r1) = decode_inner(r1, bs);
            let (v2, after_r2) = decode_inner(r2, after_r1);
            (ParseTree::Pair(Box::new(v1), Box::new(v2)), after_r2)
        }
        Regex::Star(r1) => {
            let mut iterations = Vec::new();
            let mut remaining = bs;
            loop {
                match remaining {
                    [true, rest @ ..] => {
                        remaining = rest;
                        break;
                    }
                    [false, rest @ ..] => {
                        let (v, after) = decode_inner(r1, rest);
                        iterations.push(v);
                        remaining = after;
                    }
                    [] => panic!("decode: unexpected end of bits inside Star"),
                }
            }
            (ParseTree::Star(iterations), remaining)
        }
        Regex::Phi => panic!("decode: called on Phi"),
    }
}
```

Figure 5.11: `decode_inner`: the worker behind `decode`, one arm per expression form.

The `Alt` arm matches the leading bit with `[false, rest @ ..]` or `[true, rest @ ..]` and recurses into the branch it names, returning `rest` as the remaining bits, since only the leading bit belongs to the alternation.

The `Seq` arm shows why the worker returns a pair. It recurses into the first factor, receives both a tree and the bits the first factor left unconsumed, and passes those bits to the second factor. The second factor then returns its own tree and the bits that follow it, which are the bits for any enclosing expression. Returning only the tree would leave the second recursion with no way to know where to start.

The `Star` arm loops, since the number of iterations is not known in advance and is itself encoded in the bits. Each leading `false` is matched by `[false, rest @ ..]` and corresponds to one iteration, so the arm recurses into the body and continues the loop from whatever bits the body left behind. The loop ends at the leading `true` matched by `[true, rest @ ..]`, which `mk_eps_bc` appended.

5.2.7 Parser

The bit-coded parser performs three steps on the input. First, *internalize* converts the pattern r into an annotated expression ri. Then, for each input character c, the parser takes the annotated derivative $ri\backslash_b c$ and simplifies it, replacing ri with $simp(ri\backslash_b c)$. After the last character, the parser checks whether the final ri is nullable. If it is, the parser extracts the bits with $mkEpsBC$ and decodes them against the original r:

$$\begin{aligned} ri_0 &= internalize(r), \\ ri_i &= simp(ri_{i-1}\backslash_b x_i) \quad (1 \le i \le n), \\ result &= \begin{cases} decode_r(mkEpsBC(ri_n)) & n(ri_n) \\ \text{no match} & \text{otherwise.} \end{cases} \end{aligned} \tag{5.12}$$

The loop holds only the current ri. Unlike the standard derivative parser, which keeps a growing vector of residuals for the backward reconstruction, the bit-coded parser needs no history. The bits record everything needed to decode, so the forward pass can discard each intermediate expression as soon as the next one is computed.

In Rust. Figure 5.12 implements (5.12).

```
pub fn parse_bitcoded_loop(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut ri = internalize(r);
    for l in input.chars() { ri = simp(deriv_bc(ri, l)); }
    if !nullable_bc(&ri) { return None; }
    Some(decode(r, &mk_eps_bc(&ri)))
}
```

Figure 5.12: `parse_bitcoded_loop`: the bit-coded parser implementation.

The loop matches the three cases of (5.12) one for one. `internalize` initialises `ri`, the `for` loop applies `simp(deriv_bc(ri, l))` to each character and the nullability check gates the final `decode(mk_eps_bc(ri))`.

5.2.8 Worked Example: Applying the Bitcoded Parser

Consider the same input $\mathtt{ab}$ and regular expression $r_0 = (a + ab)(b + \varepsilon)$ as in Section 5.1.4.

Internalize. Before consuming input, *internalize* tags each alternative with [0] for the left branch and [1] for the right branch:

$$\mathit{internalize}(r_0) = ri_0 = [\,]@\Big([\,]@([0]@a \oplus [1]@(a\,b)) \cdot [\,]@([0]@b \oplus [1]@\varepsilon)\Big). \tag{5.13}$$

Bit-Threaded Derivatives. The two input symbols are consumed by successive bit-threaded derivatives with *simp* applied after each step. The first derivative uses B_{seq}. After consuming $\mathtt{a}$, the first factor becomes nullable, so the second derivative uses $\mathsf{B}^n_{\text{seq}}$:

$$\begin{aligned}
ri_0 &\xrightarrow[\mathsf{B}_{\text{seq}}]{a} ri_1 = [\,]@\Big([\,]@([0]@\varepsilon \oplus [1]@b) \cdot [\,]@([0]@b \oplus [1]@\varepsilon)\Big) \\
ri_1 &\xrightarrow[\mathsf{B}^n_{\text{seq}}]{b} ri_2' = [\,]@\Big(\underbrace{[\,]@(\emptyset \oplus [1]@\varepsilon) \cdot [\,]@([0]@b \oplus [1]@\varepsilon)}_{X} \oplus \underbrace{[0]@\Big([0]@\varepsilon \oplus \emptyset\Big)}_{Y}\Big) \\
&\xrightarrow{\mathit{simp}} ri_2 = [\,]@\Big([1]@([0]@b \oplus [1]@\varepsilon) \oplus [0,0]@\varepsilon\Big).
\end{aligned} \tag{5.14}$$

Each disjunct of ri_2' has a dead branch. In X, the first factor's derivative by $\mathtt{b}$ is $\emptyset \oplus [1]@\varepsilon$: the [0] side cannot consume $\mathtt{b}$, the [1] side owes one. In Y, the second factor's derivative by $\mathtt{b}$ is $[0]@\varepsilon \oplus \emptyset$: the [0] side matches, the [1] side cannot. The outer [0] on Y is the bit that $\mathsf{B}^n_{\text{seq}}$ fused on from the first factor's empty match.

simp drops the dead branches. From X, the survivor is $[1]@\varepsilon$, and since ε contributes nothing to the concatenation, the [1] is carried onto the second factor, giving $[1]@([0]@b \oplus [1]@\varepsilon)$. From Y, the survivor is $[0]@\varepsilon$; the outer [0] is merged with the inner [0] and the resulting $[0,0]$ is carried onto the ε, giving $[0,0]@\varepsilon$. The two results are the disjuncts of ri_2, so both bits are recorded there and the tree can later be decoded from the bits alone.

Extracting the bits. Both disjuncts of ri_2 are nullable, so EB^L_+ selects the left one. Inside, b is not nullable while ε is, so EB^R_+ contributes its [1] tag:

$$\mathit{mkEpsBC}(ri_2) \xrightarrow{\mathsf{EB}^L_+} \mathit{mkEpsBC}\Big([1]@([0]@b \oplus [1]@\varepsilon)\Big) \xrightarrow{\mathsf{EB}^R_+} [1,1]. \tag{5.15}$$

Unlike *mkEps*, which constructs a partial parse tree, *mkEpsBC* extracts only the committed choices as a bit-string.

Decoding the bits. The decoder interprets $[1,1]$ against the original r_0: the first bit selects the right branch of $a + ab$ (producing ab), and the second selects the right branch of $b + \varepsilon$:

$$\mathit{decode}_{r_0}([1,1]) = \Big(\mathit{Right}(a,b), \mathit{Right}(\,)\Big) = v_0. \tag{5.16}$$

The decoder consumes the bits left to right, one per alternation it encounters, in the same order that *internalize* introduced them. Because *mkEpsBC* preserved that order when it fused and prepended bits, the first bit always belongs to the outermost alternation, and so on down.

Flattening gives $\mathit{flatten}(v_0) = a \cdot b \cdot \varepsilon = \mathtt{ab}$, confirming the parse. The bit-coded construction produces the same tree as the direct construction, but stores only ri_2, a single expression bounded by the pattern instead of one residual per character.

6 Partial-Derivative-Based Parser

Chapter 5 presented two derivative-based methods for constructing POSIX parse trees. The standard parser stores the derivation sequence and reconstructs the tree in a backward pass using *mkEps* and the injections. The bit-coded parser records the choices as bits during the forward pass and decodes them against the original expression at the end.

This chapter presents the partial-derivative alternative of Sulzmann and Lu [Sulzmann and Lu, 2012]. Instead of one residual per step, the partial derivative produces a set of states, each pairing a residual expression with the reconstruction information for one path through the original expression. The parser maintains this set as an NFA state set, and at the end walks it in generation order, taking the first accepting state. Because partial derivatives break apart the structure of the original expression, this walk yields the leftmost alternative at every choice rather than the longest one, and therefore a non-POSIX parse tree.

As in Chapter 5, two variants are presented. The standard parser attaches an injection closure to each state. The bit-coded parser replaces these closures with bit-sequences. Both compute the same parse tree.

6.1 The Standard Partial-Derivative Parser

Consider again $r_0 = (a + ab)(b + \varepsilon)$ with input $w = \mathtt{ab}$. Where *deriv* reduces r_0 to a single residual at each step, Antimirov's partial derivative *pderiv* reduces it to a *set* of states, one per way the next character can be consumed. Each state pairs a residual expression with an injection that rebuilds a parse tree for r_0 from a parse tree for that residual. The injection is attached to its state as soon as the state is created, rather than recovered afterwards from a stored sequence the way *inj* was for the standard derivative parser in Section 5.1.

Deriving r_0 by $\mathtt{a}$ produces two states, one for each way $\mathtt{a}$ can be consumed:

$$\begin{aligned} \mathit{pderiv}(r_0, \mathtt{a}) = \big\{ &(\ \varepsilon \cdot (b + \varepsilon), \ \lambda(_, v_2).\ (\mathit{Left}\, a, \ v_2) \ ), \\ &(\ (\varepsilon \cdot b)\,(b + \varepsilon), \ \lambda((v_1, v_2), v_2').\ (\mathit{Right}(a, v_2), \ v_2') \ ) \big\}. \end{aligned} \tag{6.1}$$

A state is a pair: a residual expression describing what is left to match, and an injection rebuilding a parse tree of the original expression from a parse tree of the residual. The two states above correspond to the two branches of the alternation $a + ab$ in the first factor. The second factor $b + \varepsilon$ is untouched in both and appears as the trailing component of each residual.

Each residual has the shape $r' \cdot (b + \varepsilon)$: the derivative of the first factor, followed by the untouched second factor, never simplified. The leading ε in each r' records that the consumed character matched a leading literal and left nothing behind in that position; it is not removed, because the injection below pattern-matches on exactly this pair shape.

The first state's residual $\varepsilon \cdot (b + \varepsilon)$ is a sequence, so its tree is a pair (v_1, v_2): v_1 is the tree of the leading ε, which is necessarily *Empty*, and v_2 is the tree of the untouched second factor $b + \varepsilon$. The closure returns $(\mathit{Left}(a), v_2)$. It discards v_1 because the tree of ε carries no information, and replaces the first position with $\mathit{Left}(a)$ to record that the left branch of the alternation $a + ab$ was taken. The second factor's tree v_2 is passed through unchanged.

The second state's residual $(\varepsilon \cdot b)\,(b + \varepsilon)$ is a sequence whose first component is itself a sequence, so its tree is a pair $((v_1, v_2), v_2')$: the inner pair (v_1, v_2) is the tree of $\varepsilon \cdot b$, with v_1 the tree of the leading ε and v_2 the tree of the trailing b; the outer v_2' is the tree of the untouched second factor. The closure returns $(\mathit{Right}(a, v_2), v_2')$. It rebuilds the consumed a (which the leading ε records) and the still-owed b (which v_2 records) as the pair (a, v_2), places that pair inside a right injection to record that the right branch ab of the alternation was taken, and passes the second factor's tree v_2' through unchanged. The binder v_1 is bound but not used in the body, for the same reason as the first state's `_`: the tree of ε carries no information.

Transitioning both states on the second character `b` gives a new state set:

$$\begin{aligned} \mathit{pderiv}(r_0, \mathtt{a}, \mathtt{b}) = \big\{&(\varepsilon,\ \lambda_.\ (\mathit{Left}\, a,\ \mathit{Left}\, b)\ ),\\ &(\varepsilon \cdot (b + \varepsilon),\ \lambda(_, v_2).\ (\mathit{Right}(a, b),\ v_2)\ )\big\}. \end{aligned} \tag{6.2}$$

The first state now has residual ε, meaning every input character has been consumed, and is accepting; the second has residual $\varepsilon \cdot (b + \varepsilon)$ – the second factor is untouched by `b`, since it was the first factor's leftover b that matched – which is also accepting. Selection walks the state set in generation order and returns the first accepting state. Applying its injection to $\mathit{mkEps}_{\varepsilon} = ()$ yields $(\mathit{Left}\, a, \mathit{Left}\, b)$, the leftmost alternative at every choice (Figure 6.1).

Two properties of the state set determine which tree is selected. First, the order of states in the set encodes the preference. When a state produces several successors, they are appended in the order the rules generate them, with the left alternative first, so the leftmost path appears earliest. Second, when two states share a residual, *nub* keeps the first and discards the later one with its injection, so the earlier path wins ties. Both mechanisms favour the leftmost alternative at every choice over the longest one, which is what distinguishes the tree this parser returns from the POSIX tree of Chapter 5.

6.1.1 Partial Derivatives with Injections

A partial derivative of an expression r with respect to a character x is not a single expression but a set of pairs. Each pair consists of a residual expression r' whose language is one component of the left quotient, together with an injection closure ι that maps a parse tree for r' back to a parse tree for r. Stripped of the injection, the residual side of this definition is exactly *pd*, the Antimirov partial derivative already used by the matcher of Section 3.3 [Antimirov, 1996], so *pderiv* reuses its rule names $\mathsf{P}_{\emptyset}, \mathsf{P}_{\varepsilon}, \mathsf{P}_{\mathrm{lit}}, \mathsf{P}_{+}, \mathsf{P}_{\mathrm{seq}}, \mathsf{P}^{n}_{\mathrm{seq}}, \mathsf{P}_{*}$

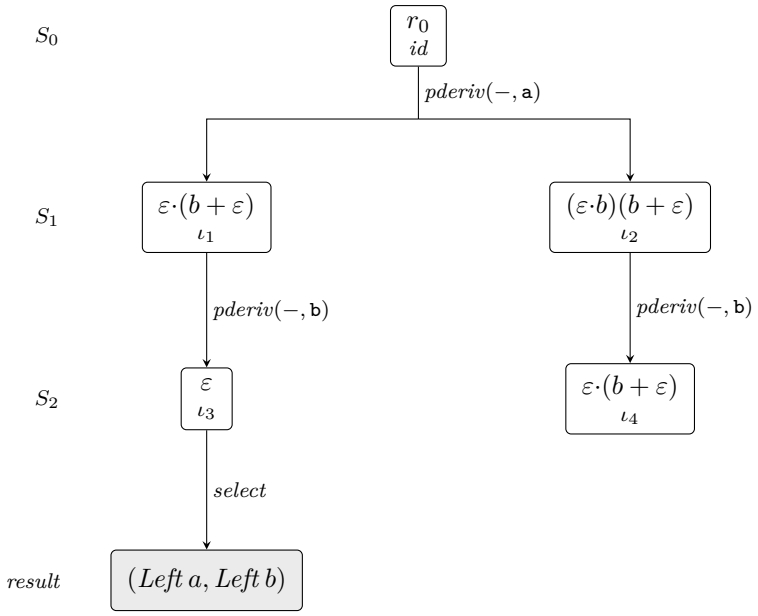


Figure 6.1: The standard partial-derivative parser on $r_0 = (a + ab)(b + \varepsilon)$ with input `ab`. Each box is a state (r, ι): a residual and an injection, both shown without any simplification. $pderiv(-, x)$ advances every state by one character; *select* takes the first accepting state and applies its injection to *mkEps* of its residual.

from Equation 3.14, each now additionally threading an injection alongside its residual. The idea of pairing a residual with a reconstruction function is, in turn, in the style of Sulzmann and Lu's pattern partial derivative [Sulzmann and Lu, 2012, Fig. 11], which pairs a residual *pattern* with a function building a variable-binding environment; here the same idea builds a *ParseTree* instead.

The rules that do not depend on whether their operand is nullable are:

$$
\begin{aligned}
pderiv(\emptyset, x) &= \{\} && (\mathsf{P}_\emptyset)\\
pderiv(\varepsilon, x) &= \{\} && (\mathsf{P}_\varepsilon)\\
pderiv(y, x) &= \begin{cases} \{(\varepsilon,\ \lambda_.\, y)\} & x = y \\ \{\} & x \neq y \end{cases} && (\mathsf{P}_{\text{lit}})\\
pderiv(r + s, x) &= \{\, (r',\ \lambda v.\, Left\,(\iota\, v)) \mid (r', \iota) \in pderiv(r, x) \,\}\\
&\quad \cup\ \{\, (s',\ \lambda v.\, Right\,(\iota\, v)) \mid (s', \iota) \in pderiv(s, x) \,\} && (\mathsf{P}_+)\\
pderiv(r^*, x) &= \{\, (r'\, r^*,\ \lambda(v, Star(vs)).\, Star([\iota\, v] +\!\!+ vs)) \mid (r', \iota) \in pderiv(r, x) \,\} && (\mathsf{P}_*)
\end{aligned}
\tag{6.3}
$$

exactly as in *pd*, with the recursive call's injection wrapped by *Left*, *Right*, or prepended to the iteration list. No rule is needed for $\emptyset$ or ε beyond their empty results, since neither can consume a character.

Concatenation alone must branch on $n(r)$, so it is shown separately:

$$
pderiv(r \cdot s, x) = \begin{cases}
\{\, (r'\, s,\ \lambda(v_1, v_2).\, (\iota\, v_1,\ v_2)) \mid (r', \iota) \in pderiv(r, x) \,\} & n(r) \quad (\mathsf{P}^n_{\text{seq}})\\
\cup\ \{\, (s',\ \lambda v.\, (mkEps_r,\ \iota\, v)) \mid (s', \iota) \in pderiv(s, x) \,\} & \\
\{\, (r'\, s,\ \lambda(v_1, v_2).\, (\iota\, v_1,\ v_2)) \mid (r', \iota) \in pderiv(r, x) \,\} & \neg\, n(r) \quad (\mathsf{P}_{\text{seq}})
\end{cases}
\tag{6.4}
$$

After expansion, each rule's result is deduplicated by residual, keeping the first occurrence; this is *nub*, applied wherever a transition can produce two states with the same residual, namely P_+, $\mathsf{P}^n_{\text{seq}}$, and P_*. P_{seq} has only one source of states and needs no deduplication of its own, since its single recursive call is already duplicate-free. This is the step that breaks apart the original expression: where the derivative of $r + s$ produces a single alternation, the partial derivative produces separate states for each branch, and the outer alternation disappears from the residual.

Rule (P_{seq}) – the only one present regardless of nullability – passes the outer pair through unchanged, with the recursive injection applied to the first component only. When r is nullable, rule ($\mathsf{P}^n_{\text{seq}}$) additionally unions in the states that skip r entirely, using $mkEps_r$ to synthesise its empty match for the first component. Rule (P_+) wraps the recursive injection in a *Left* or *Right* node, reinserting the alternation choice. Rule (P_*) reconstructs the current iteration by applying the recursive injection and prepending it to the remaining iterations.

Unlike *pd*, which applies the smart constructor *smartC* in P_{seq} and P_* to avoid building $\varepsilon \cdot s$ (Equation 3.15), *pderiv* never simplifies: $r' \cdot s$ above is built with the plain pairing constructor, even when $r' = \varepsilon$. A simplified residual would collapse exactly the pair-shaped node that the injection above pattern-matches on, so *pderiv* trades the smaller residuals *pd* produces for residuals whose shape the injection can always follow. This is the same tradeoff the standard derivative parser makes by never simplifying (Section 5.1), for the same reason: *inj* there pattern-matches on the derivative's structure just as ι does here.

In Rust. The recursive definition in Equations 6.3 and 6.4 is implemented directly in Rust, with two departures from a literal reading of the equations, both for performance rather than semantics.

First, `pderiv_tree` itself builds only *one* transition's closure, type `InjStep`, a reference-counted closure exactly as before. The accumulated injection a state carries across many characters – what the equations call ι once composition has happened several times – is a separate type, `Inj`, described in Section 6.1.2: a persistent list of `InjStep`s rather than a nested closure, so that applying or dropping a long-lived `Inj` does not recurse one native stack frame per character.

Second, the `Seq` and `Star` arms use `smart_seq` – the Rust counterpart of *smartC* used by `pderiv` in Figure 3.7 – wherever the equations build $\varepsilon \cdot s$, except in `Seq`'s nullable branch, which still pairs its continue-strand with the plain `Regex::seq` constructor. Collapsing $\varepsilon \cdot s$ to s changes the shape the next transition's closure must match against, so each arm that uses `smart_seq` branches on whether the residual collapsed (`r1_ == Regex::Eps`) and builds the closure for that shape specifically, rather than assuming a `Pair` is always present. Without this, two strands that independently derive to the same residual but reached it through different wrapper shapes would not dedup against each other, and the frontier can grow unboundedly on an ambiguous pattern even though the number of distinct residuals stays bounded.

```
pub(crate) type InjStep = Rc<dyn Fn(ParseTree) -> ParseTree>;

pub(crate) fn pderiv_tree(r: &Regex, x: char) -> Vec<(Regex, InjStep)> {
    match r {
        Regex::Eps | Regex::Phi => vec![],

        Regex::Lit(c) => {
            if *c == x {
                vec![(Regex::Eps,
                      Rc::new(move |_| ParseTree::Char(x))) as InjStep]
            } else { vec![] }
        }

        Regex::Alt(r1, r2) => {
            let mut out: Vec<(Regex, InjStep)> = pderiv_tree(r1, x)
                .into_iter()
                .map(|(r1_, inj1)| {
                    let f: InjStep = Rc::new(move |v| {
                        ParseTree::Left(Box::new(inj1(v)))
                    });
                    (r1_, f)
                })
                .collect();
            out.extend(pderiv_tree(r2, x).into_iter().map(|(r2_, inj2)| {
                let f: InjStep = Rc::new(move |v| {
                    ParseTree::Right(Box::new(inj2(v)))
                });
                (r2_, f)
            }));
            nub_tree(out)
        }

        Regex::Seq(r1, r2) => {
            if nullable(r1) {
                // Continue-strand: not smart-constructed, matching
                // the reference literally -- the skip-strand below
                // needs r1's own closure untouched to inject into.
                let mut out: Vec<(Regex, InjStep)> = pderiv_tree(r1, x)
                    .into_iter()
                    .map(|(r1_, inj1)| {
                        let r2c = (**r2).clone();
                        let f: InjStep = Rc::new(move |v| match v {
                            ParseTree::Pair(v1, v2) =>
                                ParseTree::Pair(Box::new(inj1(*v1)), v2),
                            _ => panic!("pderiv_tree/Seq: want Pair"),
                        });
                        (Regex::seq(r1_, r2c), f)
                    })
                    .collect();

                let v1_eps = mk_eps(r1);
                out.extend(pderiv_tree(r2, x).into_iter().map(|(r2_, inj2)| {
                    let v1c = v1_eps.clone();
                    let f: InjStep = Rc::new(move |v2| {
                        ParseTree::Pair(Box::new(v1c.clone()),
                                        Box::new(inj2(v2)))
                    });
```

The `Phi`, `Eps`, and `Lit` arms implement $\mathsf{P}_\emptyset$, P_ε, and P_{lit} directly, matching the same arms of `pderiv` in Figure 3.7, except that each returns a closure alongside its residual instead of nothing. The `Alt` arm implements P_+ and is where the partial derivative visibly breaks apart the expression: each branch produces its own states, tagged by *Left* or *Right* in the injection, then deduplicated by `nub_tree`.

The `Seq` arm's `if nullable(r1)` matches the case split between $\mathsf{P}^n_{\text{seq}}$ and P_{seq}. The nullable branch's continue-strand is built exactly as the equations describe, with the plain pairing constructor, since the skip-strand alongside it needs `r1`'s own closure to still expect a `Pair`; the nullable branch additionally computes `r1`'s empty tree once with `mk_eps` and captures a clone of it in every skip-strand closure, then deduplicates the combined list. The non-nullable branch, by contrast, does use `smart_seq`, with the closure split on whether the residual collapsed to `Eps`, as explained above.

The `Star` arm implements P_*, with the closure appending the reconstructed iteration to the star's list (left-to-right order, reversed once at the end by `select` rather than paying `insert(0, ..)`'s cost on every iteration), again split on whether `smart_seq` collapsed the residual.

6.1.2 State Sets and Selection

The parser maintains a state set of (residual, injection) configurations. For each input character, every configuration in the state set is transitioned with `pderiv_tree`, and the resulting configurations are collected into a new state set, replacing the old one – unlike the standard derivative parser, nothing from earlier state sets is retained once the next one has been computed. A configuration's parent injection is composed with the transition injection, so that at the end the state set's injections map residuals all the way back to the original expression.

Conceptually the composition is direct: when a parent configuration (r, inj_p) transitions to (r', inj_s), the new configuration's injection is $\lambda v.\, inj_p(inj_s(v))$. Composing this way as nested closures, however, makes both calling the result and dropping it recurse one native stack frame per character – fine at tens of thousands of characters, a real stack-overflow risk beyond that. The `Inj` type instead stores a configuration's accumulated steps as a persistent cons-list and applies them in an explicit loop:

```
enum InjNode { Identity, Step(InjStep, Rc<InjNode>) }
pub(crate) struct Inj(Rc<InjNode>);

impl Inj {
    pub(crate) fn then(&self, step: InjStep) -> Self {
        Inj(Rc::new(InjNode::Step(step, self.0.clone())))
    }
    pub(crate) fn apply(&self, v: ParseTree) -> ParseTree {
        let mut v = v;
        let mut cur = &self.0;
        loop {
            match &**cur {
                InjNode::Identity => return v,
                InjNode::Step(step, rest) => { v = step(v); cur = rest; }
```

```
            }
        }
    }
}
```

`then` adds one more character's step in O(1), just wrapping the existing chain in a new node, the same way `Bits::prepend` (Section 5.2.2) wraps an existing bit chain; `apply` walks the whole chain in one iterative pass, applying each step in the order it was added, with no recursive call regardless of how long the chain is. `Inj` also has its own iterative `Drop`, for the same reason `Bits`'s does: the derived recursive drop glue would walk the chain one stack frame per character too.

```
pub(super) fn step_frontier(
    frontier: Vec<(Regex, Inj)>, x: char
) -> Vec<(Regex, Inj)> {
    let mut out = Vec::new();
    for (r, inj_parent) in frontier {
        for (r_next, inj_step) in pderiv_tree(&r, x) {
            out.push((r_next, inj_parent.then(inj_step)));
        }
    }
    nub_tree(out)
}
```

Figure 6.3: `step_frontier`: one transition of the NFA state set on a single character. The function name retains the older "frontier" terminology; it computes the successor state set $\delta(S, x)$. The final `nub_tree` deduplicates *across* parent configurations, not just within one parent's own expansion (which `pderiv_tree` already does on its own output) – without it, two strands that independently derive to the same residual both survive, and the state set can double every character on an ambiguous pattern even though the number of distinct residuals stays bounded by the pattern's own size.

After the input is exhausted, selection walks the state set in priority order and returns the tree for the first accepting configuration:

```
pub(super) fn select(frontier: &[(Regex, Inj)]) -> Option<ParseTree> {
    for (r, inj) in frontier {
        if nullable(r) {
            return Some(reverse_stars(inj.apply(mk_eps(r))));
        }
    }
    None
}
```

Figure 6.4: `select`: return the tree for the first accepting configuration in the state set. `reverse_stars` undoes, in one O(total output size) pass, the left-to-right-via-push construction order `pderiv_tree`'s `Star` arm uses (Figure 6.2).

Because the state set lists configurations in the order the rules generated them, and because `nub_tree` keeps the first of each duplicate residual, selection favours the configuration that committed earliest to the leftmost alternative. This is the non-POSIX leftmost

policy. Unlike the standard derivative parser, there is no separate backward pass here: by the time `select` runs, every configuration's injection is already the full composition back to the original expression, so selection only has to apply it once.

6.1.3 Parser

The parser has a single pass: every input character advances the whole state set at once via *step_frontier*, and the final state set is handed directly to *select*:

$$S_0 = \{(r, id)\}, \qquad S_i = step_frontier(S_{i-1}, x_i) \;\; (1 \le i \le n), \qquad result = select(S_n). \tag{6.5}$$

Unlike the standard derivative parser's two-pass driver (Equation 5.4), no backward pass is needed here: each configuration in S_i already carries its injection composed all the way back to r_0 (Section 6.1.2), so *select* only ever applies one injection, to the single accepting configuration it finds in S_n.

In Rust. Figure 6.5 implements (6.5).

```
pub fn parse_pderiv_std(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier: Vec<(Regex, Inj)> =
        vec![(r.clone(), Inj::identity())];
    for c in input.chars() {
        frontier = step_frontier(frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    select(&frontier)
}
```

Figure 6.5: The standard partial-derivative parser.

The loop computes $S_0, \ldots, S_n$ in place, overwriting `frontier` on every iteration rather than appending to a growing `Vec` the way `parse_deriv_std_loop` (Figure 5.4) does – nothing from an earlier state set is retained once the next one exists. The early return on an empty frontier corresponds to $S_i = \{\}$: no configuration can consume the current character, so the input cannot be matched. The final `select` returns `None` if no configuration in S_n is accepting, and otherwise the tree for the first one.

6.1.4 Worked Example: Standard Partial-Derivative Parser

Recall the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ on input `ab`.

Forward pass. The initial state set holds a single configuration:

$$S_0 = \big[(r_0,\ id)\big].$$

Transitioning with respect to `a`:

$$pderiv(r_0, \mathtt{a}) = pderiv\Big(Alt(a, ab), \mathtt{a}\Big) \cdot Alt(b, \varepsilon).$$

Inside the first factor:

$$\begin{aligned} \mathit{pderiv}(a, \mathtt{a}) &= \Big[(\varepsilon,\ \lambda_.\, a)\Big] \\ \mathit{pderiv}(ab, \mathtt{a}) &= \Big[(\mathit{Seq}(\varepsilon, b),\ \lambda(v_1, v_2).\, (a, v_2))\Big]. \end{aligned}$$

After the alternation rule wraps each state with *Left* or *Right*:

$$\begin{aligned} \mathit{pderiv}(a + ab, \mathtt{a}) = \Big[&(\varepsilon,\ \lambda v.\, \mathit{Left}\,(\lambda_.\, a)(v)), \\ &(\mathit{Seq}(\varepsilon, b),\ \lambda(v_1, v_2).\, \mathit{Right}\,(a, v_2))\Big]. \end{aligned}$$

The outer sequence rule combines this with the untouched second factor:

$$\begin{aligned} S_1 = \Big[&(\mathit{Seq}(\varepsilon, b + \varepsilon),\ \lambda(v_1, v_2).\, (\mathit{Left}\, a,\ v_2)), \\ &(\mathit{Seq}(\mathit{Seq}(\varepsilon, b), b + \varepsilon),\ \lambda((v_1, v_2), v_2').\, (\mathit{Right}(a, v_2),\ v_2'))\Big]. \end{aligned}$$

Transitioning with respect to $\mathtt{b}$ steps both configurations. For the first, the left factor ε is nullable, so the sequence rule produces the state that jumps directly into the second factor:

$$\mathit{pderiv}(b + \varepsilon, \mathtt{b}) = \Big[(\varepsilon,\ \lambda v.\, \mathit{Left}\, b)\Big].$$

The second factor's injection inserts the character at the leftmost position of the pair:

$$\mathit{inj}_1(\mathit{Left}\, b) = (\mathit{Left}\, a,\ \mathit{Left}\, b).$$

For the second configuration, the leading $\mathit{Seq}(\varepsilon, b)$ is not nullable (ε is, but b is not), so rule ($\mathsf{P}_{\mathrm{seq}}$) fires and the derivative continues inside b:

$$\mathit{pderiv}(\mathit{Seq}(\varepsilon, b), \mathtt{b}) = \Big[(\varepsilon,\ \lambda_.\, ((), b))\Big],$$

and the outer sequence rule rebuilds the pair *without* simplifying the leading ε away, giving the second configuration's residual $\varepsilon \cdot (b + \varepsilon)$, not $b + \varepsilon$. The two resulting state-set entries are:

$$\begin{aligned} S_2 = \Big[&(\varepsilon,\ \lambda_.\, (\mathit{Left}\, a,\ \mathit{Left}\, b)), \\ &(\varepsilon \cdot (b + \varepsilon),\ \lambda(_, v_2).\, (\mathit{Right}(a, b),\ v_2))\Big]. \end{aligned}$$

Selection. The first configuration's residual is ε, which is nullable, and selection returns immediately:

$$\mathit{select}(S_2) = (\lambda_.\, (\mathit{Left}\, a, \mathit{Left}\, b))(\mathit{mkEps}_{\varepsilon}) = (\mathit{Left}\, a, \mathit{Left}\, b).$$

Flattening confirms the result:

$$\mathit{flatten}\Big((\mathit{Left}\, a, \mathit{Left}\, b)\Big) = \mathtt{ab}.$$

The second configuration is never inspected: its residual $\varepsilon \cdot (b + \varepsilon)$ is also nullable, and would likewise decode to a valid tree, but generation order already placed the first configuration ahead of it, and *select* stops at the first match.

This is the non-POSIX parse tree. The derivative-based parser from Chapter 5 returns $(\mathit{Right}(a, b), \mathit{Right}())$ for the same input, the POSIX tree. The two trees agree on which word they spell out, but differ on how the input is decomposed: the non-POSIX tree commits to the shorter first alternative a in the first factor, while the POSIX tree commits to the longer alternative ab.

6.2 The Bit-Coded Partial-Derivative Parser

The standard partial-derivative parser carries a closure per configuration, which accumulates the reconstruction path during the forward pass. The bit-coded variant replaces closures with bit-sequences: each configuration's annotation records the choices made so far, and decoding the final bit-sequence against the original expression yields the parse tree.

Transitioning $w = \texttt{ab}$ from $r_0 = (a + ab)(b + \varepsilon)$ gives:

$$\begin{aligned} S_0 &= \Big[(r_0,\ [\,])\Big] \\ &\xrightarrow{\texttt{a}} S_1 = \Big[(b + \varepsilon,\ [\mathit{false}]),\ (b\,(b + \varepsilon),\ [\mathit{true}])\Big] \\ &\xrightarrow{\texttt{b}} S_2 = \Big[(\varepsilon,\ [\mathit{false}, \mathit{false}]),\ (b + \varepsilon,\ [\mathit{true}])\Big]. \end{aligned} \tag{6.6}$$

The first configuration of S_1 records that the character `a` was matched by the left branch of the first factor's alternation. The second configuration records that it was matched by the right branch, leaving the residual b to be consumed later. After the second character, only the first configuration is accepting, and its bits $[\mathit{false}, \mathit{false}]$ decode to $(\mathit{Left}\, a, \mathit{Left}\, b)$.

Figure 6.6 shows this transition together with the selection and decode steps.

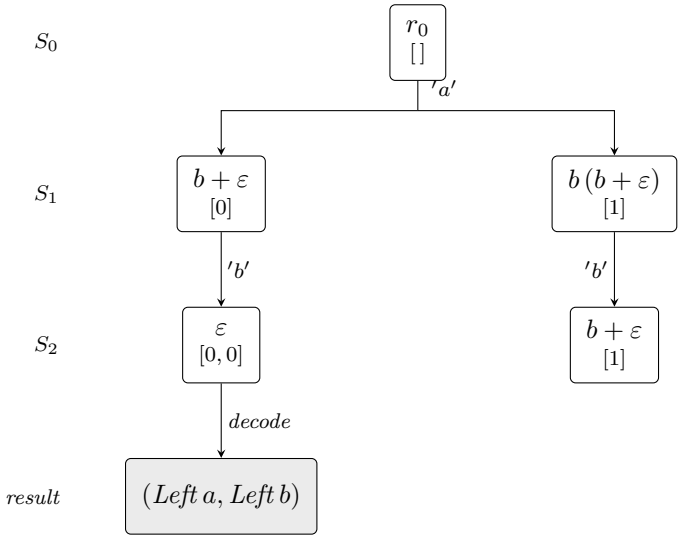


Figure 6.6: Overview of the bit-coded partial-derivative parser on the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ with input `ab`. Each box is a configuration: a residual expression and a bit-sequence, written 0 for *false* and 1 for *true*. Selection takes the first accepting configuration in S_2 and appends *mkEpsBC* of its residual, yielding the bit-string $[0, 0]$, which decode_{r_0} reads against the original expression.

The bit-coded parser keeps the same state-set structure as the standard one. Only the configuration's annotation changes, from a closure to a sequence of bits, and the final step changes from applying an injection to calling *decode*.

6.2.1 Bit-Coded Partial Derivatives

The bit-coded partial derivative $\mathit{pderivBC}(r, x)$ returns a list of (residual, bit-sequence) pairs. It follows the same case structure as (**??**), with one difference: where the standard

version builds an injection closure, the bit-coded version prepends a fixed bit-sequence to the configuration's existing bits.

The bit conventions are inherited from Chapter 5.2. Each alternation contributes a leading *false* for the left branch or a leading *true* for the right branch. Each star iteration contributes a leading *false*, and each star termination contributes a *true*. Concatenation contributes no bits of its own.

$$
\begin{aligned}
\mathit{pderivBC}(\phi, x) &= \emptyset \\
\mathit{pderivBC}(\varepsilon, x) &= \emptyset \\
\mathit{pderivBC}(c, x) &= \begin{cases} \{(\varepsilon,\, [\,])\} & c = x \\ \emptyset & \text{otherwise} \end{cases} \\
\mathit{pderivBC}(r_1 + r_2, x) &= \{\, (r_1',\, [\mathit{false}] +\!\!+\, bs) \mid (r_1', bs) \in \mathit{pderivBC}(r_1, x) \,\} \\
&\quad \cup \{\, (r_2',\, [\mathit{true}] +\!\!+\, bs) \mid (r_2', bs) \in \mathit{pderivBC}(r_2, x) \,\} \\
\mathit{pderivBC}(r_1\, r_2, x) &= \{\, (\mathit{seq}(r_1', r_2),\, bs) \mid (r_1', bs) \in \mathit{pderivBC}(r_1, x) \,\} \\
&\quad \cup \begin{cases} \{\, (r_2',\, \mathit{mkEpsBC}_{r_1} +\!\!+\, bs) \mid (r_2', bs) \in \mathit{pderivBC}(r_2, x) \,\} & n(r_1) \\ \emptyset & \text{otherwise} \end{cases} \\
\mathit{pderivBC}(r^*, x) &= \{\, (\mathit{seq}(r', r^*),\, [\mathit{false}] +\!\!+\, bs) \mid (r', bs) \in \mathit{pderivBC}(r, x) \,\}.
\end{aligned}
\tag{6.7}
$$

As in the standard version, the result is deduplicated by residual, keeping the first occurrence.

The bit-coded sequence rule is the counterpart of the injection rule in the standard version. When the first factor is nullable, the bits recorded inside its empty match are extracted with $\mathit{mkEpsBC}_{r_1}$ and prepended to the second factor's bits, so the choice made inside the first factor survives in the surviving configuration.

In Rust. Figure 6.7 implements (6.7) directly. The base cases and the alternation case mirror the standard version with bit-lists in place of closures. The sequence case inlines both the non-nullable and nullable paths. The star case prepends the leading *false* that marks one more iteration.

```
pub fn pderiv_bc(r: &Regex, x: char) -> Vec<(Regex, Vec<bool>)> {
    match r {
        Regex::Phi | Regex::Eps => vec![],

        Regex::Lit(y) => {
            if *y == x { vec![(Regex::Eps, vec![])] } else { vec![] }
        }

        Regex::Alt(r1, r2) => {
            let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
                .into_iter()
                .map(|(r1_, mut bs)| {
                    let mut b = vec![false]; b.append(&mut bs);
                    (r1_, b)
                })
                .collect();
            out.extend(pderiv_bc(r2, x).into_iter().map(|(r2_, mut bs)| {
                let mut b = vec![true]; b.append(&mut bs);
                (r2_, b)
            }));
            nub2(out)
        }

        Regex::Seq(r1, r2) => {
            if nullable(r1) {
                let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
                    .into_iter()
                    .map(|(r1_, bs)| (Regex::seq(r1_, *r2.clone()), bs))
                    .collect();

                let eps_bits = mk_eps_bits(r1);
                out.extend(pderiv_bc(r2, x).into_iter().map(|(r2_, bs)| {
                    let mut b = eps_bits.clone(); b.extend(bs);
                    (r2_, b)
                }));
                nub2(out)
            } else {
                pderiv_bc(r1, x)
                    .into_iter()
                    .map(|(r1_, bs)| (smart_seq(r1_, r2), bs))
                    .collect()
            }
        }

        Regex::Star(r1) => {
            let out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
                .into_iter()
                .map(|(r1_, mut bs)| {
                    let mut b = vec![false]; b.append(&mut bs);
                    (smart_seq(r1_, &Regex::star(*r1.clone())), b)
                })
                .collect();
            nub2(out)
        }
    }
}
```

Figure 6.7: `pderiv_bc`: one bit-coded partial-derivative transition

The `Seq` arm keeps the outer sequence constructor in the nullable case, matching the standard version's behaviour. It uses `smart_seq` only in the non-nullable case, where the derivative of the first factor is guaranteed to be a proper expression and no outer pair needs to be preserved. This asymmetry mirrors the paper's reference implementation and is needed so that the residual's shape stays compatible with the decoder.

6.2.2 Extracting Bits

When a configuration's residual is nullable, its accumulated bits must be extended with the bits of the residual's empty match. This is the role of $mkEpsBC$, which mirrors $mkEps$ from Chapter 5 but produces a bit-sequence instead of a parse tree.

$$\begin{aligned} mkEpsBC_{\varepsilon} &= [\,] \\ mkEpsBC_{r^*} &= [true] \\ mkEpsBC_{r_1 r_2} &= mkEpsBC_{r_1} +\!\!+ mkEpsBC_{r_2} \\ mkEpsBC_{r_1 + r_2} &= \begin{cases} [false] +\!\!+ mkEpsBC_{r_1} & n(r_1) \\ [true] +\!\!+ mkEpsBC_{r_2} & \text{otherwise.} \end{cases} \end{aligned} \tag{6.8}$$

The alternation rule reproduces the left preference of $mkEps$, emitting *false* when the left branch is nullable and *true* otherwise. The star rule emits a single terminating *true*, since a star matching the empty word uses zero iterations.

```
pub fn mk_eps_bits(r: &Regex) -> Vec<bool> {
    match r {
        Regex::Phi => panic!("mk_eps_bits called on Phi"),
        Regex::Lit(c) => panic!("mk_eps_bits called on Lit('{}')", c),
        Regex::Eps => Vec::new(),
        Regex::Alt(r1, r2) => {
            if nullable(r1) {
                let mut bits = vec![false];
                bits.extend(mk_eps_bits(r1));
                bits
            } else {
                let mut bits = vec![true];
                bits.extend(mk_eps_bits(r2));
                bits
            }
        }
        Regex::Seq(r1, r2) => {
            let mut bits = mk_eps_bits(r1);
            bits.extend(mk_eps_bits(r2));
            bits
        }
        Regex::Star(_) => vec![true],
    }
}
```

Figure 6.8: `mk_eps_bits`: the bit-string analogue of `mk_eps` for plain `Regex`.

The `Phi` and `Lit` arms panic, matching the precondition that *mkEpsBC* is applied only to nullable expressions. Neither empty language nor a literal can match the empty word.

6.2.3 State Sets and Selection

State-set transition for the bit-coded parser mirrors the standard version, with bit-list concatenation replacing closure composition. Each configuration's bits accumulate across the whole input so far, so a plain `Vec<bool>` clone-then-extend on every transition is O(current length) per step, $O(n^2)$ over n characters – the same cost `Bits`/`Inj` avoid elsewhere in this and the previous chapter. `AppendBits` is the append-optimized mirror of `Bits` (Section 5.2.1): it stores the reverse of the real sequence internally, so appending a suffix becomes prepending its reverse, reusing `Bits::prepend`'s O(suffix length) guarantee, and materializes the sequence back into a plain `Vec<bool>` only once, when the bits are actually consumed:

```
pub(super) fn step_frontier(
    frontier: &[(Regex, AppendBits)], x: char
) -> Vec<(Regex, AppendBits)> {
    let mut out = Vec::new();
    for (r, bits) in frontier {
        for (r_next, extra) in pderiv_bc(r, x) {
            out.push((r_next, bits.append(&extra)));
        }
    }
    nub2(out)
}
```

Figure 6.9: `step_frontier` (bit-coded): as with the standard parser's version, `nub2` deduplicates across the whole combined state set, not just within one configuration's own expansion (which `pderiv_bc` already does via its own `nub2` call).

Selection walks the state set in priority order and returns the bits of the first accepting configuration, extended with *mkEpsBC* of the residual:

```
pub(super) fn select_bits(
    frontier: &[(Regex, AppendBits)]
) -> Option<Vec<bool>> {
    for (r, bits) in frontier {
        if nullable(r) {
            return Some(bits.append(&mk_eps_bits(r)).to_vec());
        }
    }
    None
}
```

The resulting bit-sequence is interpreted by the shared `decode` function from Chapter 5.2, which walks the original expression and consumes one bit per alternation and per star iteration.

6.2.4 Parser

The parser composes `step_frontier` over the input, checks that the state set is non-empty, and calls `select_bits` followed by `decode`.

```
pub fn parse_pderiv_bc(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier = vec![(r.clone(), AppendBits::empty())];
    for c in input.chars() {
        frontier = step_frontier(&frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    let bits = select_bits(&frontier)?;
    Some(decode(r, &bits))
}
```

Figure 6.10: The bit-coded partial-derivative parser.

The driver has the same structure as the standard one. Only the configuration type and the final step differ: bits and `decode` in place of closures and injection application.

6.2.5 Worked Example: Ambiguous Expression

Consider again the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ on input `ab`.

Forward pass. The initial state set holds a single configuration with empty bits:

$$S_0 = \Big[(r_0,\ [\,])\Big].$$

Transitioning by `a`, the first factor is not nullable, so *pderivBC* recurses into it:

$$\begin{aligned} \mathit{pderivBC}(a + ab, \mathtt{a}) &= \mathit{pderivBC}(a, \mathtt{a}).\mathit{map}(+[\mathit{false}]) \cup \mathit{pderivBC}(ab, \mathtt{a}).\mathit{map}(+[\mathit{true}]) \\ &= \Big[(\varepsilon,\ [\mathit{false}]),\ (b,\ [\mathit{true}])\Big]. \end{aligned}$$

The sequence rule then places the second factor after each derivative:

$$S_1 = \Big[(b + \varepsilon,\ [\mathit{false}]),\ (b\,(b + \varepsilon),\ [\mathit{true}])\Big].$$

Transitioning by `b`, both configurations are stepped. The first configuration reaches the second factor directly:

$$\mathit{pderivBC}(b + \varepsilon, \mathtt{b}) = \Big[(\varepsilon,\ [\mathit{false}])\Big],$$

giving bits $[\mathit{false}, \mathit{false}]$. The second configuration's derivative comes from b inside the leading sequence:

$$\mathit{pderivBC}(b, \mathtt{b}) = \Big[(\varepsilon,\ [\,])\Big],$$

and the outer sequence rule reconstructs the residual $b + \varepsilon$ with the bits unchanged:

$$S_2 = \Big[(\varepsilon,\ [\mathit{false}, \mathit{false}]),\ (b + \varepsilon,\ [\mathit{true}])\Big].$$

Selection and decoding. The first configuration is accepting, so *selectBits* returns its bits extended with $mkEpsBC_{\varepsilon} = [\,]$:

$$selectBits(S_2) = [false, false].$$

Decoding these bits against r_0:

$$decode_{r_0}\Big([false, false]\Big) = \Big(Left(a), Left(b)\Big).$$

The first *false* selects the left branch of the first factor's alternation, committing to the shorter alternative a. The second *false* selects the left branch of the second factor's alternation, committing to the shorter alternative b. The resulting tree is the non-POSIX parse tree, identical to the one produced by the standard partial-derivative parser.

Flattening validates the result:

$$flatten\Big((Left(a), Left(b))\Big) = \texttt{ab}.$$

Both partial-derivative parsers therefore produce the same tree on this example, and both differ from the derivative-based parsers of Chapter 5, which commit to the longer first alternative and return $(Right(a, b), Right())$.

7 Translation

The five parsers of Chapters 5 and 6 operate on the six-constructor `Regex` of Chapter 2. Real patterns may be more complex and contain shorthand character classes, bounded repetition, alternation over ranges, and a PCRE flag suffix, none of which the `Regex` can represent. This chapter introduces `ExtPat`, an AST that represents a surface syntax that allows for such richer patterns, and shows how pattern strings are first parsed into this `ExtPat` and subsequently translated into the internal `Regex`.

7.1 Parsing: String to ExtPat

Take the pattern string `[a-c]+\d{2}`. A recursive-descent parser turns it into the intermediate tree `ExtPat`, as shown in Figure 7.1. Precedence is fixed by the call chain, from loosest to tightest, with alternation and concatenation being left-associated and postfix quantifiers binding most tightly. Concatenation splits the pattern into `([a-c]+)(\d{2})`, and the postfix turns them into `Plus` over a class and a bounded repetition of a shorthand:

```
Concat [
    Plus(Any(['a', 'b', 'c'])),
    Bound(Escape('d'), 2, Some(2))
]
```

Figure 7.1: The `ExtPat` produced from `[a-c]+\d{2}`.

7.2 Translate: ExtPat to Regex

`translate` then turns the `ExtPat` into the `Regex` value, that can be consumed by the parsing algorithms of the previous chapters:

- `Any(['a','b','c'])` expands over the working alphabet to a 3-leaf `Alt` chain.
- `Plus` is rewritten structurally as `Seq(r, Star(r))`.
- `Escape('d')` expands to a 10-leaf `Alt` chain, with one leaf for each digit.
- The two `Concat` layers are translated into nested `Seq` nodes, with the resulting structure folded into a single `Seq` chain.

The resulting `Regex` has 54 nodes (Figure 7.2). The two digit expansions account for 38 of them, with 20 digit leaves and 18 `Alt` nodes, here abbreviated as `D`:

```
D = Alt(Alt(Alt(Alt(Alt(Alt(Alt(Alt(Alt(0,1),2),3),4),5),6),7),8),9)
```

The remaining 16 come from the two `[a-c]` expansions, the `Plus` rewrite, the unrolling, and the surrounding `Seq` nodes.

```
Seq(
   Seq(
      Alt(Alt(a, b), c),
      Star(Alt(Alt(a, b), c))
   ),
   Seq(
      Seq(Eps, D),
      D
   )
)
```

Figure 7.2: The full `Regex` produced by translating the `ExtPat`. `D` abbreviates the nine-deep, left-nested `Alt` chain over the digits `0`–`9`.

7.3 The Surface Syntax

Rejected constructs: The example pattern is accepted, but not all are. A construct is rejected whenever it cannot be translated without changing what the pattern denotes.

Full-string matching: A translated pattern matches the whole input or nothing. There is no search padding, no substring search, and no anchor handling.

A 98-character working alphabet: Shorthand and negated classes enumerate a fixed set of 98 characters, the 95 printable ASCII characters `0x20`–`0x7E` together with tab, newline, and carriage return. The shorthands expand to `\d` (digit) `0`–`9`, `\w` (word) `a`–`z`, `A`–`Z`, `0`–`9`, and underscore, and `\s` (whitespace) space, tab, newline, and carriage return.

PCRE flags: Rule sets commonly encode matching options in a `/PATTERN/FLAGS` wrapper such as `/abc/i`, so the suffix must be interpreted alongside the pattern. Each flag falls into one of three groups (Table 7.1).

Flag	Group	Effect
`i`	honoured	case-fold literal letters
`s`	honoured	dot matches newline
`m`	rejected	anchors match at newlines
`x`	rejected	whitespace insignificant
`R`	ignored	search positioning
`A`	ignored	anchor selection
`E`	ignored	dollar tuning
`U`, `H`, `P`, `C`, ...	ignored	buffer or normalisation selection

Table 7.1: PCRE flags by group. Honoured flags are part of the pattern's meaning and are reproduced by a structural rewrite on `ExtPat`. Some flags are rejected outright. Ignored flags choose a buffer region or search position and do not affect the denoted language.

7.3.1 Accepted Constructs

Table 7.2 summarizes the surface syntax handled by the frontend, its representation in `ExtPat`, and the corresponding `Regex` translation for supported constructs.

Surface syntax	`ExtPat`	Translates to	Note
Structural			
ε, empty branch of `a|/(|b)`	`Eps`, `Empty`	`Eps`	
$\emptyset$	`Never`	`Phi`	
literal `a`	`Char`	`Lit('a')`	full Unicode `char`
`ab`	`Concat`	`Seq(a, b)`	left-folded
`a|b|c`	`Or`	`Alt(Alt(a, b), c)`	left-folded
`(...)`, `(?:...)`, `(?<n>...)`, `(?P<n>...)`	`Group`, `GroupNonMarking`	body	wrapper name dropped
`\n`, `\t`, `\r`	`Char`	`Lit(ch)`	
`\xHH`	`Char`	`Lit(ch)`	one byte
`\0` outside a class	`Char`	`Lit(ch)`	octal, clamped to 255
`\0`–`\7` inside `[...]`	class member	`Any`/`NoneOf` member	octal, clamped to 255
Quantifier			
`r?`	`Opt`	`Alt(r, Eps)`	
`r+`	`Plus`	`Seq(r, Star(r))`	
`r*`	`Star`	`Star(r)`	
`r{m,n}`	`Bound`	`m` copies and `n-m` optional	$m,n \leq 1000$
`r{m,}`	`Bound`	`m` copies and a `Star`	$m \leq 1000$
`r*?`, `r+?`, `r??`, `r{m,n}?`	as above	as above	lazy marker dropped
Class (alphabet expansion)			
`[abc]`, `[a-z]`	`Any`	`Alt` chain, left-folded	
`[^abc]`	`NoneOf`	`Alt` chain over complement	alphabet minus listed
`.`	`Dot`	`Alt` chain, 98 leaves	97 under `/.../` without `s`
`\d`, `\w`, `\s`	`Escape`	`Alt` chain, 10, 63, 4 leaves	
`\D`, `\W`, `\S`	`Escape`	`Alt` chain, 88, 35, 94 leaves	

Table 7.2: Surface syntax, grouped by translation mechanism, together with the `ExtPat` each produces and the `Regex` it translates into. Leaf counts refer to the 98-character alphabet.

`ExtPat::Escape` covers `\d`, `\w`, `\s`, their capitalised forms, and escaped punctuation such as `\.`. Control, hexadecimal, and octal escapes are represented as `ExtPat::Char`. Capitalised shorthands and negated classes denote the working alphabet minus the positive set. Literals can be any Unicode `char` and become `Regex::Lit` values. Thus, `é` matches itself, but `\W` cannot match it. Shorthands expand over the working alphabet; escaped punctuation becomes one literal, so `\.` matches only a period. Within character classes, shorthands expand inline and cannot serve as range endpoints.

7.3.2 Rejected Constructs

Backreferences and lookarounds are rejected as non-regular constructs, while other regular forms, such as anchors and POSIX classes, are excluded from this work's scope (Table 7.3).

Construct	Example	Reason
Rejected at parse		
backreference	`(a)\1`, `(?P=n)`, `\1`–`\9`	not regular, no `ExtPat`
lookaround	`(?=`, `(?!`, `(?<=`, `(?<!`	not regular, no `ExtPat`
POSIX class	`[[:alpha:]]`	would be read as literal characters
Rejected at translation		
anchor	`^`, `$`	position-dependent
word boundary	`\b`, `\B`	position-dependent
bound above 1000	`a{1001}`	unrolling prohibitive
Rejected at flag check		
flag `m`	`/abc/m`	would make anchors newline-aware
flag `x`	`/a b/x`	would make whitespace insignificant

Table 7.3: Surface forms the parser refuses, grouped by the stage at which refusal happens.

7.4 Translation Sizes

The example in Figure 7.2 translates to 54 nodes. Table 7.4 shows the contribution of individual character classes. A class with n members produces n literals and $n - 1$ `Alt` nodes, for $2n - 1$ nodes in total. Negated classes are complemented against the 98-character working alphabet before expansion. The dot construct expands to the full working alphabet when `s` is set, or to the alphabet excluding newline otherwise.

Pattern	Members	`Alt` nodes	Total nodes
`\d`	10	9	19
`\D`	88	87	175
`\w`	63	62	125
`\W`	35	34	69
`\s`	4	3	7
`\S`	94	93	187
`[^a]`	97	96	193
`.`	98	97	195

Table 7.4: Node counts for alphabet expansion over the 98-character working alphabet.

Translated pattern size depends on both structural and alphabet expansion, as shown by the effects of repetition and case folding in Table 7.5. Repetition increases the size by cloning the repeated expression, while case folding replaces each letter with an `Alt` between its uppercase and lowercase forms, adding two nodes per letter.

Structural expansion only		*Including alphabet expansion*	
Pattern	Nodes	Pattern	Nodes
`a*`	2	`\d+`	40
`a?`	3	`\d{2}`	41
`/a/i`	3	`[a-z]`	51
`a+`	4	`[a-c]+\d{2}`	54
`abc`	5	`\d{2,}`	62
`a{2}`	5	`\d{2,4}`	87
`ab{2}`	7	`[a-z]+`	104
`a{2,}`	8	`\w+`	252
`a{2,4}`	15	`\S+`	376

Table 7.5: Translated size in `Regex` nodes for representative patterns.

Bounds are also a source of growth. `Regex` has no representation for `ExtPat::Bound`, so `r{m,n}` becomes m exact copies plus $n - m$ optional ones, and `r{m,}` becomes m copies plus a `Star`. Both clone the body, so size grows with the bound.

However, the implementation imposes limits to keep translated patterns manageable. Unrolling clones the body once per repetition, and `translate` recurses once per unrolled copy, so an unbounded repeat count risks a stack overflow during translation alone. Bounded repetitions whose `m` or `n` exceeds 1,000 are therefore rejected. This limits individual repetitions but not total size, since nested bounds and multiple classes compound. Dataset preparation therefore caps the translated `Regex` at 20,000 nodes and its estimated nesting depth at 2,000. The measurements taken to justify these limits can be seen in the following evaluation chapter.

8 Evaluation

This chapter evaluates the derivative and partial-derivative parsers developed in Chapters 5 and 6. Using two datasets and a common set of measurement criteria, it compares the candidate parsers in terms of stack depth, execution time, memory consumption, and translation cost, and assesses how far they agree on membership decisions and parse-tree shape. The resulting findings are then interpreted and discussed.

All measurements were taken on an AMD Ryzen 5 5600 six-core processor with 16 GB of RAM, running Pop!_OS 24.04 LTS on an AMD B550 chipset.

8.1 Candidates

All five candidates share the same interface. Each parser candidate, denoted by p, takes an input string and a translated `Regex` and returns either a `ParseTree` or no match:

$$p :: (\mathit{String}, \mathit{Regex}) \to \mathit{Option}\langle \mathit{ParseTree} \rangle.$$

The input (w, r) is called an *entry*, where w is an input string and r is a translated `Regex`. The candidates fall into two families according to their parse-tree selection strategy.

POSIX family. These three parsers select the POSIX parse tree.

- `deriv_std_rec`, the standard derivative parser in recursive form (Section 5.1.3).
- `deriv_std_loop`, the standard derivative parser in iterative form, holding the residual sequence in a `Vec` rather than a call stack.
- `deriv_bc`, the bit-coded derivative parser (Section 5.2.7).

Non-POSIX family. These two parsers select the leftmost parse tree.

- `pderiv_std`, the standard partial-derivative parser (Section 6.1.3).
- `pderiv_bc`, the bit-coded partial-derivative parser (Section 6.2.4).

All five parsers are expected to agree on membership. An entry must either be accepted or rejected by every candidate. Their parse trees, however, need not have the same shape. Within each family, the parsers are expected to agree on both membership and tree shape, since they implement the same parse-tree selection strategy. Across families, the selected trees may differ for ambiguous patterns.

8.2 Evaluation Criteria

The evaluation addresses *performance*, the time, memory, and translation cost of each candidate, and *correctness*, whether the five candidates agree on membership and parse-tree shape. First, a stack-depth comparison of the two forms of the standard derivative parser determines whether the iterative optimization is worth adopting.

8.2.1 Measurement Setup

Each entry runs in its own worker subprocess for timing, memory, stack-depth, and parse-agreement measurements. This allows hung or crashed calls to be terminated and their memory reclaimed; stable Rust does not support terminating individual threads.

- *Measured*: the worker exited zero and reported a result line.
- *Timed out*: the worker exceeded the per-entry wall-clock deadline, two seconds by default, and was killed by the parent.
- *Crashed*: the worker exited non-zero or was killed by a signal, covering both a stack overflow and an out-of-memory kill when the optional memory ceiling is set.

Timing, memory, and parse-agreement workers run the parser call on a custom 256 MiB stack. All candidates recurse over the pattern structure, so the default stack overflows on deeply nested patterns before the measured input sizes are reached. The larger stack is intended to prevent failures that reflect stack space rather than the parser.

Each timing and memory run produces two outputs. A summary table of aggregate candidate measurements and a per-entry file for comparing candidates on individual entries.

8.2.2 Timing

Each entry (r, w) is timed once, without warm-up or repetition. Translation is measured separately, and the worker's wall clock covers only the parser call $p(r, w)$.

For `deriv_std_rec`, `deriv_std_loop`, and `pderiv_std`, the entire call is reported as *derive*. For `deriv_bc` and `pderiv_bc`, the call is divided into *encode*, constructing the initial annotated state before any character of w is consumed; *derive*, processing the input character by character; and *decode*, converting the final bit-string into a `ParseTree`.

Timing results use four statistics, aggregated differently for the two datasets:

- *Mean* is a macro-average over patterns of the real corpus, so a pattern with 200 entries and one with 2 contribute equally, and a flat mean for the synthetic set.
- *Median*, *99th percentile*, and *maximum* are pooled over individual entries and describe the single worst entry rather than a typical pattern.

All four statistics represent lower bounds on the corresponding execution times, since timed-out and crashed entries are excluded rather than counted at their timeout values.

A separate *wall* column measures the benchmark's entire loop over one source pattern. It therefore exceeds *entries* $\times$ *mean* by the per-entry process-spawn and pipe-round-trip overhead excluded from the individual timings. It captures the total cost of reproducing a pattern's results, harness included, but is not suitable for comparisons between candidates.

8.2.3 Memory

Memory is measured by a counting allocator inside the worker. The baseline is recorded immediately before the parser call, so only allocations made during $p(r, w)$ are counted. Three measures are distinguished:

- *Total bytes*: all memory allocated during the call, including allocations later freed.
- *Average bytes*: total bytes divided by the number of completed entries.
- *Max peak*: the highest byte count within a single call, measuring peak consumption.

Total and peak can differ substantially. Many small, short-lived allocations give a high total but a low peak, while a single large, long-lived allocation gives a moderate total but a high peak. Reporting both therefore separates allocation activity from the memory footprint of a call. The average uses the same averaging bases as the timing measurements. Total and peak are pooled directly over entries.

The counting allocator sees heap allocations only, not the 256 MiB worker stack, so parsers dominated by recursion rather than allocation may appear inexpensive here. Stack depth is evaluated separately for the two forms of the standard derivative parser

8.2.4 Translation

Translation cost depends only on the source pattern, so each distinct pattern is translated once rather than once per entry. It runs in the main process without a worker subprocess, because the frontend enforces three resource limits:

- *Bound cap*: bounds the numeric value of any repetition bound.
- *Depth cap*: bounds the estimated depth of the translated pattern.
- *Node cap*: bounds the node count of the result.

Together the three ensure a translation measurement cannot hang or allocate unboundedly the way a parser call can.

Translation cost is characterised by four measurements:

- *Time*: the wall-clock duration of the translation call.
- *Total bytes*: every allocation made during translation.
- *Peak bytes*: the highest live-byte high-water mark during the call.
- *Nodes*: the node count of the resulting `Regex`.

The counting allocator is reset before each pattern is translated and read immediately after the call returns, so subsequent node counting and debug rendering are excluded. Mean, median, 99th percentile, and maximum are pooled directly over patterns. Since each pattern contributes exactly one measurement, no macro-averaging is required. Rejected and panicked patterns produce no measurement, so all four are lower bounds.

8.2.5 Stack Depth

`deriv_std_rec` and `deriv_std_loop` compute identical results but maintain the residual sequence differently. The recursive implementation stores it on the call stack, one frame per input character, whereas the iterative implementation stores it in an explicit `Vec`. To compare their stack requirements, both run with the operating system's default thread stack with no memory cap.

The maximum surviving input length is found by a doubling search. Starting from a small input length, the size is doubled after each successful probe until the worker no longer completes. Each probe runs in its own worker subprocess, so a stack overflow terminates only that process. The per-probe timeout is raised from the default two seconds to fifteen, so slow derivatives have time to exhaust the stack. With no memory cap, a signal-killed worker almost always indicates stack overflow rather than an out-of-memory kill.

The result is a bracket between the last length that completed and the first that did not, with the outcome of the latter probe. A survived-to-crash bracket means stack exhaustion limited the input length, whereas a survived-to-timeout bracket means derivation cost limited it first. The latter are kept as negative controls, identifying patterns whose derivation becomes prohibitively expensive before their stack does.

The same doubling search also checks the other two translation caps, the *bound cap* and the *depth cap*, against where their own mechanism exhausts an explicit 8 MiB stack matching the Linux default. Bound unrolling recurses once per unit of a bound's open part, syntactic nesting once per level of `(?:...)` grouping; each cap was chosen independently and is checked against, not derived from, where its mechanism actually crashes.

8.2.6 Parse Agreement

Parse agreement is evaluated on entries with a known expected membership result, written (r, w, e), where r is the translated `Regex`, w the input string, and $e \in \{true, false\}$ the expected result.

Each candidate processes the same entry in a separate worker subprocess, and the results are evaluated along three dimensions:

- *Timeout or crash*: recorded if a candidate exceeds the default 2 s deadline or its worker is killed by a signal.
- *Membership*: each candidate's result against e. Any disagreement is a bug.
- *Shape*: compared only when $e = true$ and all candidates return a parse tree. The three POSIX parsers must agree with one another, as must the two non-POSIX parsers. Agreement across the two families is assessed separately but not required.

The results summarise how many entries were evaluated and how the candidates performed on membership and parse-tree agreement.

For each dataset, the summary separates entries that timed out or crashed from those that could be compared, and reports *match agreement*, the number of entries on which all candidates returned the expected result. Among successful matches, *shape agree* counts entries where the two families select the same tree and *shape disagree* those where their trees differ.

8.3 Dataset Preparation

The evaluation uses entries pairing a pattern r with an input string w, drawn from two datasets, as shown in the following Table 8.1.

Dataset	Authored	Duplicate	Rejected	Accepted	Unanchored	Fully-Anchored	Inputs
S2ELab	762	7	177	578	115	463	11 395
Synthetic	113	0	0	113	0	113	887

Table 8.1: Benchmark scope. *Authored* = *Duplicate* + *Rejected* + *Accepted*, and *Accepted* = *Full-Anchored* + *Unanchored*. Only the *Full-Anchored* patterns are benchmarked after stripping, and the *Inputs* column counts entries for those patterns only.

8.3.1 S2ELab

Siddiq et al. [2024] contains 762 human-authored patterns, each with a natural-language description and labeled matching and non-matching inputs.

The source patterns follow the usual convention for `^` and `$`. A top-level anchor around the whole pattern marks a full-string match, and no anchor leaves the pattern open for substring search. Anchors are not supported, as described in Section 7.3.2, but a fully anchored pattern can still be used by stripping them. Under full-string matching the pattern already consumes the entire input, so the anchors add nothing, and the stripped pattern matches exactly the language the anchored pattern intended.

The benchmarks use the 463 anchor-stripped patterns. The 115 unanchored patterns are parseable but excluded because they denote substring searches, whereas the parsers match full strings only. The remaining 177 patterns are rejected by the frontend. Table 8.2 summarises the rejection causes.

Cause	S2ELab patterns
Anchor (non-strippable)	106
Lookaround	43
Word boundary	23
Backreference	3
Other (node cap, syntax errors)	2
Total	177

Table 8.2: S2ELab patterns rejected by the frontend, broken down by cause.

Pipeline. A date pattern like `^(0[1-9]|1[0-2])\/...$` is processed the following:

1. **Extract.** The pattern is kept with the match and non-match lists.
2. **Deduplicate.** The pattern is byte-distinct and no input string appears double.
3. **Translate.** The anchors are stripped and the remainder is checked to translate.
4. **Check labels.** Each input string is checked against the translated pattern. The input string `"11/31/2002"` is accepted because the day group allows `31` regardless of month, validating the pattern's semantics, not real-world correctness.
5. **Write.** The surviving pairs are written to the corpus.

8.3.2 Synthetic Stress Set

The synthetic set was created with 113 patterns authored for controlled measurement of asymptotic behaviour. The real corpus does not stress any single cost mechanism at scale, so the synthetic set is intended to cover that. All 113 were written by hand against the full-string model, so every one is a full-string match by design and passes translation as-is.

Each row defines a pattern and an input as functions of input length n or pattern size p or both. A *sweep* is a sequence of values for one such parameter, with a measurement taken at each value in turn. Two sample rows show the difference:

Row	Swept parameter	Resulting pattern	Resulting input
P01 ($n = 10$)	n	`(a+)+`	`a` repeated 10 times
P01 ($n = 1000$)	n	`(a+)+`	`a` repeated 1000 times
E36 ($p = 100$)	p	100-way word alternation	`w000001`
E36 ($p = 1000$)	p	1000-way word alternation	`w000001`

Table 8.3: Two sample rows at two sweep values each. Row P01 fixes the pattern and sweeps n. Row E36 sweeps p, so the pattern grows while the input stays fixed.

Each row's ID carries a letter marking which half of the dataset defines it, independent of the A–L category below: `P` for the hand-authored pathological patterns proper, `E` for the baseline-and-edge-case patterns, including the handful, like `E36`, generated from a template by substituting p.

Most rows follow the P01 shape, sweeping n over repeated characters. A smaller number follow the E36 shape, sweeping p by substituting into the pattern template before parsing.

The 113 patterns are grouped into 12 categories (Table 8.4), each isolating one complexity mechanism. Each category's sweep is calibrated so that the measured cost crosses from microseconds to failure within the category's own range.

Cat.	Name	Mechanism	Patterns
A	Nested quantifiers	Quantified group inside another quantifier	16
B	Overlapping alternation	Alternatives sharing a prefix under a quantifier	15
C	Bounded rep over star	Bounded repetition wrapping a star or plus body, unrolled at translate time	7
D	Anchored-failure style	Must fully consume input, near-miss tail forces full enumeration	4
E	Prefix-overlap alternation	One alternative is a strict prefix of another	8
F	Alternation-star + literal	Alternation star or plus then a literal that fails late	7
H	Many adjacent stars/groups	Product of unbounded groups in sequence	4
I	Trivial pattern, large input	Simple pattern, very long input	10
J	Large pattern, trivial input	Thousands of literals or groups, short input	5
K	Large pattern, large input	Both axes at once	4
L	Trivial pattern, trivial input	Baseline, sanity check	33
Total			113

Table 8.4: The synthetic-set categories. Categories I, J, K, and L form a 2×2 matrix of pattern size against input size. The remaining seven categories stress a genuine blow-up mechanism.

8.4 Performance Analysis

This section compares the candidates on both datasets in terms of stack depth, execution time, memory consumption, and translation cost. Each subsection presents the real-corpus results before the synthetic-set results, with extreme synthetic cases discussed separately.

8.4.1 Stack Depth

The recursive and iterative forms run side by side on the operating system's default stack, with no memory cap, under the doubling search of Section 8.2.5. Two rows are probed. The input is `a` repeated n times in both, and the sweep grows n.

Row	Pattern	Derivative step	Parser	Last n	Outcome	Elapsed
P01	`(a+)+`	expensive (nested quantifier)	`rec`	16	timeout	15.709 s
P01	`(a+)+`	expensive (nested quantifier)	`loop`	16	timeout	15.799 s
E51	`x`	cheap (fixed residual)	`rec`	32 768	crash	1.234 s
E51	`x`	cheap (fixed residual)	`loop`	1 048 576	cap	0.160 s

Table 8.5: Doubling-search stack-depth results. *Derivative step* categorizes the pattern's derivative cost. *Outcome* reports what stopped the sweep: *timeout* at the 15 s per-probe cap, *crash* on a stack overflow, or *cap* on reaching the sweep maximum without failing.

P01, beside E51, is a second negative control. Its nested quantifier makes the derivative

step expensive, so both forms hit the 15 s cap at $n = 16$ before either stack nears its limit, confirming that stack depth is reached only when derivation stays cheap.

E51 is such a row. Every entry mismatches on the first character, so the derivative collapses to a tiny fixed residual that never grows. The recursive form crashes with a genuine stack overflow, its bracket [32 768; 65 536]. The iterative form completes every probe up to the 2 000 000 sweep cap, its bracket [1 048 576; 2 000 000] bounded by the sweep length, not by a failure.

The iterative form is therefore worth adopting. Even on a pattern with a trivially cheap derivative step, the recursive form fails at tens of thousands of characters, whereas the iterative form handles every length this sweep reaches.

8.4.2 Timing

Timing is reported per candidate on both datasets under the isolation, phase split, and averaging bases of Section 8.2.2.

S2ELab

Parser	Timed out	Mean (ms)	Median (ms)	Max (ms)
`deriv_std_rec`	74	7.718	0.212	1 849.185
`deriv_std_loop`	72	7.430	0.230	1 900.578
`deriv_bc`	0	1.699	0.323	295.244
`pderiv_std`	0	0.616	0.097	103.749
`pderiv_bc`	0	0.625	0.100	100.992

Table 8.6: Corpus timing over the default scope of 463 patterns and 11 395 entries per parser. Timeouts are excluded from the mean, median, and maximum, so those columns are lower bounds on the standard variants' true cost.

The two standard variants are the slowest by a wide margin, roughly 4–4.5× the mean of `deriv_bc` and over an order of magnitude past both partial-derivative variants. That gap is measured before the 72 and 74 entries each times out on, so the true gap is larger. Within the two faster groups, `pderiv_std` is marginally faster than `pderiv_bc` on this corpus, 0.616 against 0.625 ms mean.

Parser	Matches	Match time (s)	Non-matches	Non-match time (s)
`deriv_std_rec`	5 797	74.725	5 524	9.657
`deriv_std_loop`	5 799	70.782	5 524	9.877
`deriv_bc`	5 869	14.642	5 526	5.062
`pderiv_std`	5 869	5.708	5 526	1.505
`pderiv_bc`	5 869	5.776	5 526	1.544

Table 8.7: Total wall time by verified outcome, same run as Table 8.6.

Every parser's total is dominated by the matching side, by roughly 3× to 8× depending on the parser, since a non-match can be rejected early while a match forces the parser

to construct a tree. `deriv_std_loop`'s match-side total of 70.782 s sits measurably below `deriv_std_rec`'s 74.725 s, while their non-match totals are almost identical. The recursion-versus-iteration difference is therefore a cost of the matching path specifically.

Synthetic Stress Set

		`std_rec`		`std_loop`		`deriv_bc`		`pderiv_std`		`pderiv_bc`	
Cat.	Entries	mean	to%	mean	to%	mean	to%	mean	to%	mean	to%
A	160	219.0	69%	183.2	70%	159.5	1%	16.9	0%	15.3	0%
B	150	162.0	63%	160.5	63%	99.1	4%	97.7	0%	94.4	0%
C	56	87.0	57%	86.8	57%	243.7	25%	212.7	18%	218.8	18%
D	40	248.8	20%	243.6	20%	137.2	0%	203.9	15%	195.7	15%
E	80	293.0	56%	279.4	56%	79.1	0%	61.2	0%	56.5	0%
F	70	384.4	66%	358.8	66%	42.7	40%	101.0	3%	103.8	3%
H	28	225.5	64%	224.2	64%	266.7	7%	214.9	14%	213.7	14%

Table 8.8: Mean per-entry time in ms over completed entries, and the share of entries that hit the two-second cap, one routine pass per category at each category's routine values. The mean column excludes timed-out entries, so a low mean next to a high timeout share describes only the easy entries that survived.

Categories A, B, and E confirm the design intent. Both standard variants time out on 56 to 70% of entries while `deriv_bc` and both partial-derivative variants stay under 5%. Category F is the routine category where `deriv_bc` itself times out at a real rate, 40% against under 4% for the partial-derivative pair, and is calibrated to show the F-family mechanism defeating bit-coding. Category C's timeout rate is close to uniform across all five parsers, consistent with C's cost being dominated by bound unrolling at translation time, a cost every parser pays identically before its derivative loop runs.

Category H is the one category where the ranking inverts. `deriv_bc` times out less often at 7% than either partial-derivative variant at 14%. H places several unbounded groups adjacently rather than nested, which is the frontier-of-residuals shape the partial-derivative construction is usually best at, and bit-coding does not suffer from that shape as much as the standard partial derivative does.

Representative Patterns

Eight rows are swept individually past their category's routine values, into the millions of characters or thousands of pattern repetitions. These are the representative patterns. The first establishes the method, and the remaining seven follow in the same form.

The method-establishing pattern. Figure 8.1 plots wall time against input length for `(a|b|ab)*` on `a` repeated n times, the reproduction of the reference paper's Figure 7(a). The two standard derivative parsers are absent because both crash before the first plotted point. `deriv_bc` and the two partial-derivative variants stay roughly linear on a log-log axis across four orders of magnitude, the signature of a linear-time parser under growing input.

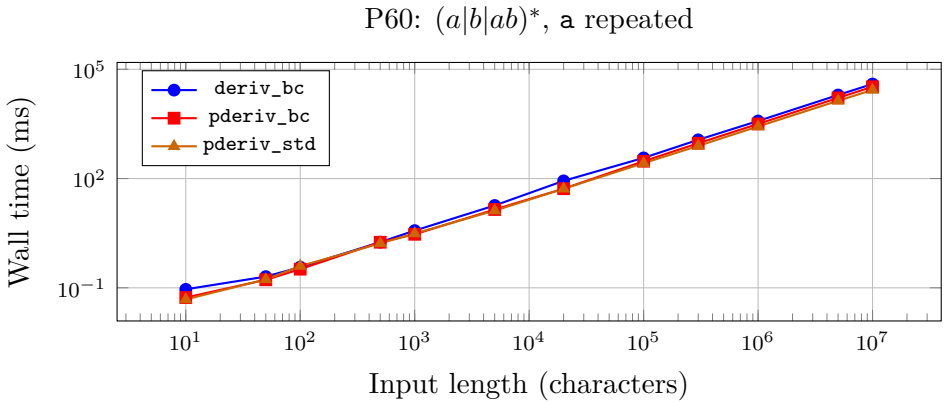


Figure 8.1: Wall time on P60, `(a|b|ab)*` against `a` repeated, every swept n plotted on a log-log axis. `deriv_bc` is directly comparable to the reference implementation's bit-coded parser. The two standard derivative parsers have no curve because both crash before the first plotted point.

Where the standard variants break. Table 8.9 reports, for each representative row, the largest swept n both standard variants still complete, the smallest swept n at which both crash, and the largest n `deriv_bc` completes before it times out.

		Standard (`rec` & `loop`)		`deriv_bc`	
Row	Pattern	Last OK	First crash	Last OK	First timeout
P60	`(a|b|ab)*`, `a` repeated	5 000	20 000	10 000 000	—
P61	`(a|b|ab)*`, `ab` repeated	15	100	5 000 000	—
P20	`(a|b|ab)+`	16	100	10 000 000	—
P01	`(a+)+`	20	50	50 000	200 000
P09	`(a|aa)+`	25	100	5 000 000	—
P25	`(a|b)*c`	2 000	3 500	10 000	30 000
P27	`([ab]+)*c`	10	15	10 000	30 000
P28	`([ab]*)*c`	10	15	10 000	30 000

Table 8.9: Match-side crash and timeout thresholds. The standard columns cover both `deriv_std_rec` and `deriv_std_loop`, since both crash at the identical swept n on every row. A dash means `deriv_bc` completed every swept n without timing out.

The two standard variants crash, not time out, at every threshold. Both hit the memory cap before the wall-clock cap, at the same swept n on every row, with near-identical total and average allocation up to that point. Recursion depth is therefore not the mechanism here. The stack-depth comparison of Section 8.4.1 showed the two variants diverge sharply on stack depth alone, with the iterative form surviving 32 times further than the recursive form. If these crashes were call-stack depth, the two variants would not die at the same input length. They do, so the mechanism is the shared two-pass retention of every intermediate derivative, not the control-flow shape.

The E family. Figure 8.2 plots the two remaining prefix-overlap rows. P61 uses the same expression as P60 over `ab` repeated, and P20 replaces the star with a plus. On all three E rows the two standard variants crash within the first few characters, and the three survivors stay roughly linear. Within the survivors the ranking differs between rows. On P60 `pderiv_std` is fastest throughout, while on P61 and P20 `pderiv_bc` overtakes it. The input shape decides which partial-derivative variant wins, not the input length.

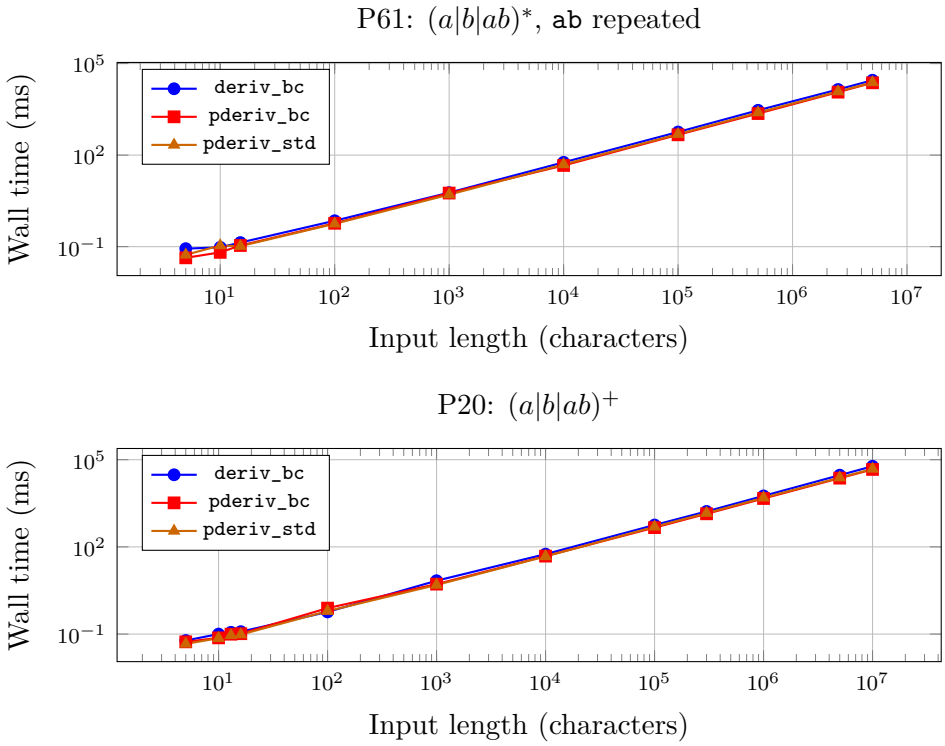


Figure 8.2: Wall time on P61 and P20, every swept n on a log-log axis. On P61 the two partial-derivative variants flip rank relative to P60, with `pderiv_bc` now faster throughout. On P20 `pderiv_bc` is also lower-peak than `pderiv_std`.

Nested quantifiers. Figure 8.3 plots the two nested-quantifier rows. P01 is `(a+)+`, the most cited example of catastrophic backtracking under conventional engines. P09 is `(a|aa)+`, its overlapping-alternation analogue. Both are the family where the two standard variants crash first, at $n = 50$ and $n = 100$, far below the E family's thresholds. `deriv_bc` is the only survivor that eventually times out on P01, at $n = 200\,000$. Both partial-derivative variants complete the sweep on both rows.

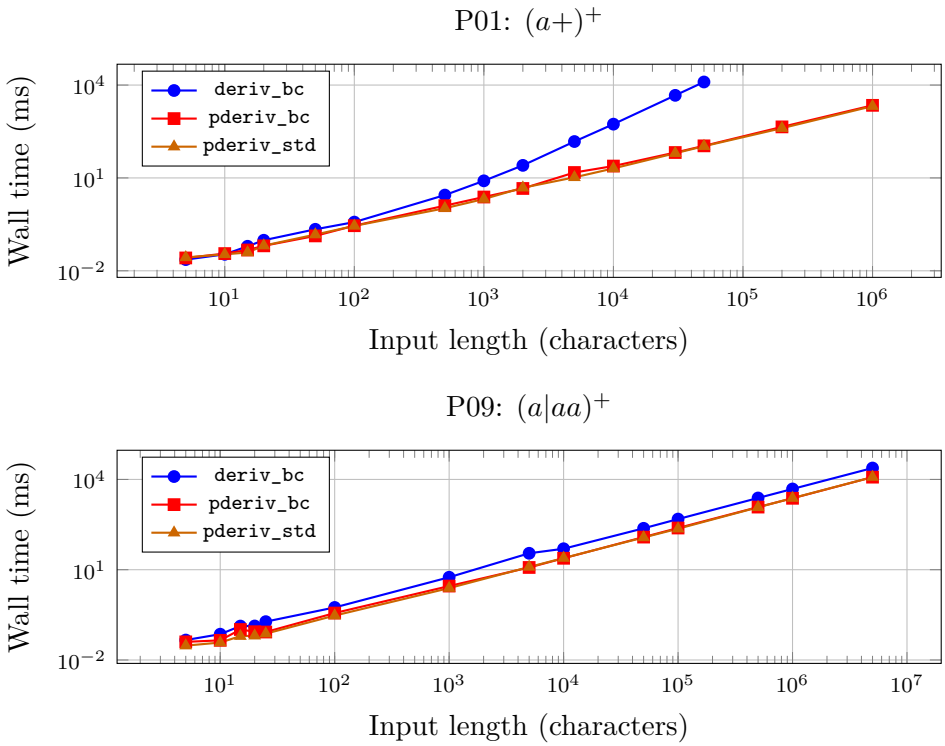


Figure 8.3: Wall time on P01 and P09, every swept n on a log-log axis. `deriv_bc` is the one parser that eventually times out on P01, at $n = 200\,000$, short of the partial-derivative pair's full sweep. On P09 all three survivors complete the whole sweep. The two standard variants crash from $n = 50$ on P01 and from $n = 100$ on P09.

The F family. Figure 8.4 plots the three alternation-star-then-literal rows, each of shape `(alt)*c` with a literal tail that fails late. All three share `deriv_bc` timing out at $n = 10\,000$, and the two standard variants crashing already at $n = 15$ on P27 and P28. The two partial-derivative variants complete every sweep. This is the family where `deriv_bc` is weakest relative to the partial-derivative pair, and the only family where the pair is not merely cheaper but qualitatively more robust.

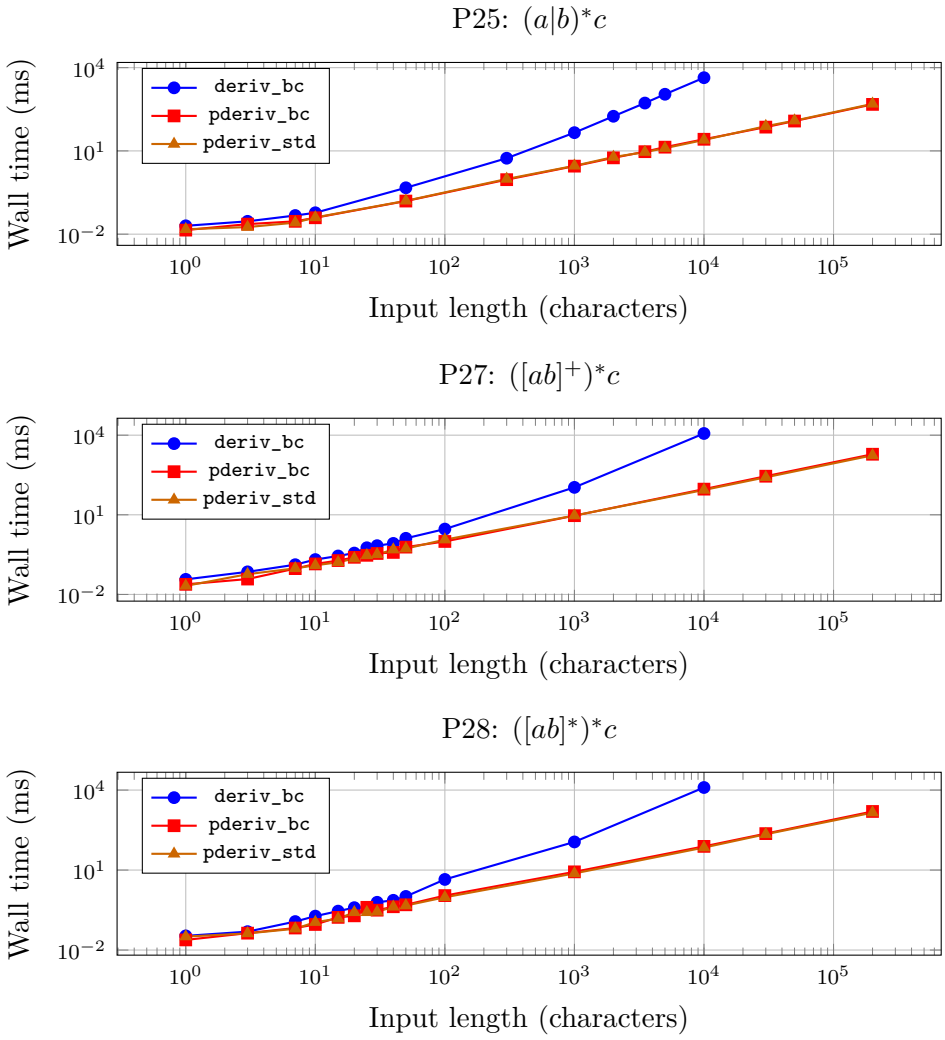


Figure 8.4: Wall time on P25, P27 and P28, every swept n on a log-log axis. `deriv_bc` eventually times out on all three, its line stopping at $n = 10\,000$. P27 and P28 additionally show the two standard variants crashing already at $n = 15$, far earlier than P25's $n = 3\,500$.

The overall picture. Across the eight representative rows the standard variants crash first everywhere, between $n = 15$ and $n = 5\,000$, and never complete a full sweep. `deriv_bc` completes every swept length on six of the eight rows, from thousands of characters past the crash point on P27 and P28 up to ten million on P60 and P20. The two exceptions are P01 and the F family, where `deriv_bc` eventually times out between one and two orders of magnitude past where the standard variants already crashed. Both partial-derivative variants complete every sweep on every row and never time out.

8.4.3 Memory

Memory is reported per candidate on both datasets under the isolation, counting-allocator setup, and averaging bases of Section 8.2.3.

S2ELab

Parser	Total	Average per entry	Max peak
`deriv_std_rec`	29.19 GB	2.671 MB	582.4 MB
`deriv_std_loop`	30.70 GB	2.820 MB	737.6 MB
`deriv_bc`	15.94 GB	1.372 MB	5.1 MB
`pderiv_std`	4.70 GB	0.403 MB	2.5 MB
`pderiv_bc`	4.80 GB	0.412 MB	3.8 MB

Table 8.10: Heap allocation over the default scope of 463 patterns and 11 395 entries. *Total* sums every allocation including churn and measures allocator pressure. *Max peak* is the highest live-byte high-water mark on any single entry and measures the footprint a deployment would see.

The gap between constructions is starkest in peak allocation. The two standard variants peak at 582.4 and 737.6 MB on their single worst entry, against 5.1 MB for `deriv_bc`, a factor of 114 to 144×. The standard parser retains its full derivation sequence for the backward pass, while `deriv_bc` carries only a flat bit vector. The same pattern holds in total and average bytes, though less sharply, roughly 1.9× and 2×.

The partial-derivative pair behaves the opposite way from what the growth curves alone would suggest. `pderiv_std` is lower than `pderiv_bc` on every column here, 0.403 against 0.412 MB average and 2.5 against 3.8 MB peak. That is the reverse of the timing gap on the same corpus. At the corpus's short input lengths the partial-derivative frontier is already small regardless of construction, so `pderiv_bc`'s bit-tracking machinery is pure overhead. The asymptotic bound-state argument is about how the two scale as input grows, not which is smaller at short inputs.

Growth on canonical expressions. Two demos drive the parsers on two canonical expressions and report live-state size per input character. For the two standard forms the figure is the node count of the current residual. For the two partial-derivative forms it is the total node count across the frontier.

	a^* on a^n			$(a+(b+ab))^*$ on $(ab)^n$		
Parser	n=1	n=8	n=16	n=1	n=8	n=16
`deriv_std`	4	39	79	16	539	9179
`deriv_bc`	2	2	2	12	8	8
`pderiv_std`	2	2	2	18	128	2048
`pderiv_bc`	2	2	2	18	128	2048

Table 8.11: Live-state size per parser at three input lengths.

On `a*`, the `deriv_std` column grows linearly in n while the other three stay bounded. The two-pass structure keeps every intermediate expression alive until the backward pass, so its state is a function of input length. Bit-coded and partial-derivative variants carry no history. On `(a+(b+ab))*`, `deriv_std` reaches 9179 nodes at $n = 16$ and keeps growing, while `deriv_bc` stays between 8 and 12 nodes forever, because the simplification step dedups branches by structural shape alone and collapses the same-shape branches this pattern produces at every step. The two partial-derivative variants are identical at every step, 2048 nodes at $n = 16$, since they build residuals with the same construction. Their

live state is the frontier of residuals rather than a single expression, bounded by the pattern's own size, well above `deriv_bc`'s fully-collapsed total but far below `deriv_std`'s.

Synthetic Stress Set

		`std_rec`		`std_loop`		`deriv_bc`		`pderiv_std`		`pderiv_bc`	
Cat.	Entries	avg MB	to%	avg MB	to%	avg MB	to%	avg MB	to%	avg MB	to%
A	160	74.3	70%	74.3	70%	186.9	1%	15.5	0%	14.5	0%
B	150	61.8	63%	61.8	63%	93.6	4%	70.5	0%	71.5	0%
C	56	33.1	57%	33.1	57%	145.8	25%	122.9	18%	139.9	18%
D	40	96.0	20%	96.0	20%	79.7	0%	121.6	15%	115.8	15%
E	80	114.8	56%	114.8	56%	106.3	0%	60.1	0%	59.5	0%
F	70	158.0	66%	158.0	66%	32.4	40%	88.0	3%	94.3	3%
H	28	77.9	64%	77.9	64%	232.3	7%	131.5	14%	135.2	14%

Table 8.12: Average total bytes per completed entry in MB, and the share of entries that hit the two-second cap, one routine pass per category at each category's routine values. The average excludes timed-out entries, so a low average next to a high timeout share describes only the easy entries that survived.

The two standard forms' per-entry averages are identical to three significant figures in every category, the same finding as the representative sweeps below. Whichever entries each variant completes before timing out cost essentially the same memory, which is a property of what both constructions retain, not of which one is recursive. `deriv_bc` is the most memory-hungry survivor in A, C, and H despite timing out far less often than the standard pair, consistent with the general point that bit-coding's retained history is not free, only cheaper than the standard construction's. Category F again stands out. `deriv_bc`'s average, 32.4 MB, is the lowest of the five parsers in that category, but only because 40% of its entries time out before accumulating the larger allocations the partial-derivative pair's surviving entries show, 88.0 and 94.3 MB.

Representative Patterns

The eight representative rows are reported by mechanism family. Tables 8.13 through 8.15 give each row's total across its full sweep, its average cumulative allocation per entry, and its peak live-byte high-water mark on any single entry. The two standard forms are reported once per row, since their totals agree to within 0.01% of each other on every row measured.

Row	Parser	Total	Average per entry	Max peak
P60	`deriv_std_rec/loop`	13.87 GB	1.156 GB	5.70 GB
P60	`deriv_bc`	169.26 GB	7.05 GB	2.64 GB
P60	`pderiv_std`	94.74 GB	3.95 GB	2.48 GB
P60	`pderiv_bc`	101.86 GB	4.24 GB	1.64 GB
P61	`deriv_std_rec/loop`	0.27 GB	45.6 MB	140.0 MB
P61	`deriv_bc`	134.06 GB	6.70 GB	1.70 GB
P61	`pderiv_std`	84.73 GB	4.24 GB	2.48 GB
P61	`pderiv_bc`	88.99 GB	4.45 GB	1.64 GB
P20	`deriv_std_rec/loop`	0.60 GB	75.6 MB	280.0 MB
P20	`deriv_bc`	271.24 GB	11.30 GB	3.40 GB
P20	`pderiv_std`	171.44 GB	7.14 GB	4.97 GB
P20	`pderiv_bc`	180.05 GB	7.50 GB	3.27 GB

Table 8.13: Memory across the three E-family representative rows.

Row	Parser	Total	Average per entry	Max peak
P01	`deriv_std_rec/loop`	1.48 GB	185.1 MB	805.3 MB
P01	`deriv_bc`	50.36 GB	1.94 GB	7.9 MB
P01	`pderiv_std`	6.47 GB	215.5 MB	153.2 MB
P01	`pderiv_bc`	6.51 GB	217.0 MB	98.2 MB
P09	`deriv_std_rec/loop`	0.58 GB	57.6 MB	284.3 MB
P09	`deriv_bc`	97.96 GB	3.50 GB	845.7 MB
P09	`pderiv_std`	37.63 GB	1.34 GB	961.3 MB
P09	`pderiv_bc`	38.00 GB	1.36 GB	698.1 MB

Table 8.14: Memory across the two nested-quantifier representative rows.

Row	Parser	Total	Average per entry	Max peak
P25	`deriv_std_rec/loop`	6.61 GB	413.0 MB	2.11 GB
P25	`deriv_bc`	8.65 GB	393.0 MB	4.73 MB
P25	`pderiv_std`	1.92 GB	68.5 MB	86.2 MB
P25	`pderiv_bc`	2.06 GB	73.5 MB	52.0 MB
P27	`deriv_std_rec/loop`	3.17 GB	396.2 MB	1.41 GB
P27	`deriv_bc`	14.79 GB	568.8 MB	4.81 MB
P27	`pderiv_std`	4.60 GB	153.3 MB	99.0 MB
P27	`pderiv_bc`	4.60 GB	153.4 MB	52.0 MB
P28	`deriv_std_rec/loop`	2.76 GB	345.1 MB	1.21 GB
P28	`deriv_bc`	14.75 GB	567.2 MB	4.81 MB
P28	`pderiv_std`	4.08 GB	135.9 MB	99.0 MB
P28	`pderiv_bc`	4.19 GB	139.6 MB	52.0 MB

Table 8.15: Memory across the three F-family representative rows.

Two things are worth being explicit about. First, the standard forms' totals in every table are small *because* most of their swept entries crashed rather than completed. The figure is a sum over the few entries that survived, not evidence the standard construction is cheap at this scale. Second, the *total* column is cumulative allocation churn across the whole sweep, including memory allocated and freed many times over. It is not memory held

at once. P20's `deriv_bc` total of 271 GB against a peak of only 3.40 GB illustrates the gap directly. Ten million characters of `(a|b|ab)+`, parsed by a construction that rebuilds its bit-coded tree at every one of millions of derivative steps without memoizing across them, allocates and discards an enormous amount of short-lived state while never holding more than a few gigabytes live at once. The same row's entire routine-scope corpus total, 15.94 GB across all 11 395 real entries, is smaller than one adversarial 10-million-character input's churn. That is the point of the synthetic set, to find costs a realistic corpus never approaches.

Peak memory on the surviving entries is what bounds deployability. `deriv_bc` is consistently the cheapest full-tree-building construction at this scale, 1.64 to 3.40 GB peak across the eight rows, below both partial-derivative variants on the E family and below `pderiv_std` on P20 by a wide margin, 3.40 against 4.97 GB. On the F family its peak drops under 5 MB on every row, because the derivative sequence collapses to a single residual rather than a frontier of residuals, so the bit-coded representation never holds more than a handful of nodes live at once. On P01 its peak is 7.9 MB outright, since the pattern's ambiguity is modest and the single bit-vector representation never tracks more than a handful of live candidates.

Within the two partial-derivative forms, bit-coding's effect on peak memory depends on family in the same way its effect on timing does. On the E family `pderiv_bc` peaks lower than `pderiv_std` on every row, and on the A and B rows it is lower on P01 (98.2 against 153.2 MB) and on P09 (698.1 against 961.3 MB). On the F family `pderiv_bc` peaks lower on every row too, 52.0 against 86.2 to 99.0 MB. Bit-coding's peak-memory advantage for the partial-derivative construction is therefore more consistent than its timing advantage, which flipped between rows.

8.4.4 Translation

Translation is reported per distinct pattern on both datasets under the in-process setup and averaging bases of Section 8.2.4. Because translation cost depends only on the pattern, the counts below are per pattern rather than per entry.

S2ELab

Patterns	Mean (ms)	Max (ms)	Mean nodes	Max nodes
463	0.015	0.384	502.5	18 707

Table 8.16: Translation cost and output size over the 463 distinct patterns in the default scope.

Translation on the real corpus is cheap at 0.015 ms mean. Even its single largest pattern, 18 707 translated nodes, translates in well under a millisecond, so no real-corpus pattern comes close to the node cap.

Synthetic Stress Set

Cat.	Name	Rows swept	Rejected	Mean (ms)	Max nodes
A	Nested quantifiers	16	0	0.001	84
B	Overlapping alternation	15	0	0.003	389
C	Bounded rep over star	7	0	0.120	9 881
D	Anchored-failure style	20	0	0.007	905
E	Prefix-overlap alternation	8	0	0.002	40
F	Alternation-star + literal	7	0	0.006	756
H	Many adjacent stars/groups	4	0	0.076	3 889
I	Trivial pattern, large input	10	0	0.027	4 001
J	Large pattern, trivial input	17	4	0.390	15 999
K	Large pattern, large input	4	1	0.153	8 400
L	Trivial pattern, trivial input	33	0	0.003	388

Table 8.17: Translation cost over the synthetic set's routine values. *Rejected* counts rows whose translated result exceeds the node cap at the swept pattern size and therefore produce no measurement.

Category C is the most translation-expensive category by far, mean 0.120 ms and 9 881 max nodes, consistent with its being built around bound unrolling. Categories J and K show directly what the parser benches could only show indirectly as uniform crash counts. Four of J's 17 swept rows and one of K's 4 reject outright at translation, at the top of each category's sweep. A rejected row produces no entry for any of the five parsers, which is why the uniform per-parser crash counts in those two categories are a translation artifact, not a parser-timing result.

Translation Caps

The three translation caps of Section 8.2.4 are grounded by the doubling search described there. Table 8.18 reports the crash point of the two recursions behind the bound and depth caps, on the default 8 MiB stack, alongside the cap the frontend enforces.

Mechanism	Survives to	Crashes between	Cap
Bound unrolling	51 200	51 200–102 400	1 000
Syntactic nesting	6 400	6 400–12 800	2 000

Table 8.18: The crash point of each mechanism on the default 8 MiB stack, alongside the cap the frontend enforces. The caps sit far below the crash points, for the reasons in the text.

Measuring the syntactic-nesting row exposed a real gap in the depth estimate: a group, capturing or not, previously added nothing to the count, so deep grouping was not actually caught before translation, regardless of how deeply nested it was. Fixed to add one level per group, the same way other wrapping constructs already do. The fix changes no outcome on the real corpus: `dataset_prepare` on the S2ELab corpus gives the same 578 accepted, 572 usable, 13 957-input corpus before and after, since no real rule nests groups anywhere near this depth. The synthetic set is not immune, though: category J's E39 row sweeps a pattern of p sequential capturing groups, using the category's own $p \in \{100, 500, 1000, 2000\}$ since the row defines no sweep of its own. The fix raises the estimate enough that $p = 1000$ now also exceeds the depth cap and is rejected before translation – previously only $p = 2000$ was, already flagged as borderline when that sweep was chosen – two fewer entries than before the fix (Section 8.4.5).

The caps sit far below the crash points, and the gap is deliberate. The crash test measures one bound's open part on a trivial one-node body, whereas a real pattern can nest bounds, so the effective recursion multiplies, and it can unroll bounds over multi-node subtrees, so each frame is larger. The depth and node caps constrain the total structure further. The crash measurements therefore establish an upper bound on what is safe, and the caps are set well inside it rather than at the edge of what the test showed.

Real usage is far below either cap in any case. Of the 762 authored patterns, 714 parse, 442 contain at least one bounded repetition, and the largest single bound value anywhere in the corpus is 67, with median 2 and 99th percentile 16. No real-corpus rule comes close to the bound cap.

The third cap, the node cap, is sized differently: node count has no comparable recursion cliff to check against, so it is set by coverage instead. The largest real-corpus pattern translates to 18 707 nodes, so the 20 000 cap covers every pattern this evaluation needed to represent with headroom to spare.

8.4.5 Parse Agreement

Parse agreement is reported per dataset under the measurement of Section 8.2.6. Membership is the precondition for anything shape can say, so both are reported together. S2ELab's entry count below, 11 395, is the same 463-pattern, full-match-only default scope used by Timing and Memory (Section 8.3.1), not the full 572-pattern, 13 957-input prepared corpus.

Dataset	Entries	Timeouts	Crashes	Compared	Match agr.	Shape agr.	Shape dis.
S2ELab	11 395	73	0	5 798	100.0%	99.8% (5 784)	0.2% (14)
Synthetic	887	451	1	207	100.0%	89.9% (186)	10.1% (21)

Table 8.19: Parse agreement over both datasets. *Compared* is the number of expected matches where no candidate timed out or crashed and both families were internally consistent, so a single POSIX tree and a single non-POSIX tree exist to compare. The two shape columns are counted over *Compared*, not over *Entries*.

Membership agreement is complete on both datasets. Every completed entry had every candidate return the expected result, so no membership bug was found, and neither are the timeouts: the 73 S2ELab timeouts are real patterns too costly for the slower candidates within the budget, while the synthetic set's 451 timeouts and single crash are deliberately engineered for the same purpose, to push candidates past what they can finish.

The 14 shape disagreements on the real corpus represent 0.2% of the compared matches. Both families agreed with themselves on every such entry, so the disagreements are purely the POSIX-versus-non-POSIX policy difference and not a within-family inconsistency. The synthetic set's 21 disagreements, out of only 207 compared matches, put its rate at 10.1%, roughly 42× the real corpus's rate. Grouping the synthetic counts by category shows where those 21 disagreements come from.

Category	Pattern	Entries	Timeouts	Compared	Shape agr.	Shape dis.
A	`(a+)+`	160	112	27	100.0% (27)	0.0% (0)
B	`(a|aa)+`	150	95	28	35.7% (10)	64.3% (18)
C	`(.*a){1,10}`	56	32	12	100.0% (12)	0.0% (0)
D	`(a?){N}a{N}`	40	12	14	100.0% (14)	0.0% (0)
E	`(a|b|ab)+`	80	45	16	87.5% (14)	12.5% (2)
F	`([ab]+)*c`	70	46	12	91.7% (11)	8.3% (1)
H	`(\w+\s{0,1})*`	28	18	4	100.0% (4)	0.0% (0)
I	`a+`	60	44	3	100.0% (3)	0.0% (0)
J	`(?:a{1,10}){1,100}`	24	4	10	100.0% (10)	0.0% (0)
K	`(?:ab|cd|ef|gh)+`	30	24	3	100.0% (3)	0.0% (0)
L	`[a-z]+`	189	19	78	100.0% (78)	0.0% (0)

Table 8.20: Parse agreement by category over the synthetic set. *Pattern* is one representative row per category; Table 8.4 names the mechanism each category stresses. Every completed entry had every candidate return the expected result, so the membership column is omitted.

Category B alone accounts for 18 of the 21 disagreements, 64.3% of its own 28 compared. B is built around alternatives sharing a prefix under a quantifier, the most reliable way to construct genuine ambiguity on purpose. Categories E and F, also alternation-heavy by design, contribute smaller shares at 12.5% and 8.3%. Every other category shows zero. The disagreements therefore concentrate exactly where the synthetic set was designed to create them, and are absent wherever a pattern leaves only one decomposition.

Representative disagreements

S2ELab. The loosely-formatted-number pattern

```
^[-+]?([0-9]{1,3}[,]?)?([0-9]{3}[,]?)*[.]?[0-9]*$
```

on input `"9000"` produces two trees that flatten to the same word, shown in Figure 8.5. Every constructor name is as it appears in the parser output, with two abbreviations for space. `<9>` stands for `Right(Char('9'))` and `<0>` for the nine-deep `Left` injection the digit-class encoding produces around `Char('0')`.

```
POSIX (deriv_bc):
  Pair(Pair(Pair(Pair(Right(Empty),
    Left(Pair(Pair(Pair(Empty, <9>),
                Left(Empty)), Right(Empty)))),
      Star([Pair(Pair(Pair(Pair(Empty, <0>), <0>), <0>),
                 Right(Empty))])),
    Right(Empty)), Star([]))

Non-POSIX (pderiv_bc):
  Pair(Pair(Pair(Pair(Right(Empty),
    Left(Pair(Pair(Pair(Empty, <9>),
                Right(Pair(<0>, Right(Pair(<0>, Empty))))),
            Right(Empty)))),
      Star([])), Right(Empty)), Star([<0>]))
```

Figure 8.5: The two parse trees for the loosely-formatted-number pattern on `"9000"`: the POSIX tree (`deriv_bc`) against the non-POSIX tree (`pderiv_bc`).

The two trees agree on every constructor up to the two optional copies of the one-to-three-digit group. The POSIX tree takes `Left(Empty)` on both, declining the extras, so that group matches only `"9"` and the `([0-9]{3}[_,]?)*` star that follows does the rest by matching `"000"` in a single iteration. The non-POSIX tree takes `Right(Pair(<0>, Right(Pair(<0>, Empty))))` on both, so that group matches `"900"`, the star becomes empty, and the trailing `[0-9]*` catches the remaining `"0"`.

Synthetic. Row P61, the pattern `(a|b|ab)*` on input `"ab"`, is the reference paper's running example, and here it appears at the smallest input that makes its ambiguity visible, shown in Figure 8.6.

```
POSIX (deriv_bc):
  Star([Right(Pair(Char('a'), Char('b')))])

Non-POSIX (pderiv_bc):
  Star([Left(Char('a')), Right(Left(Char('b')))])
```

Figure 8.6: The two parse trees for `(a|b|ab)*` on `"ab"`, the reference paper's running example of POSIX-versus-greedy disagreement.

The POSIX tree commits to a single iteration that matches `"ab"` via the right branch of the alternation. The non-POSIX tree commits to two iterations, first `"a"` then `"b"`, each via the left branch. Both flatten to `"ab"`. The disagreement is structural, not about which word was matched, and it is exactly the case the reference paper uses to introduce the POSIX-versus-greedy distinction.

8.5 Discussion of Results

8.5.1 Cost of the standard construction

The two standard derivative parsers are consistently the slowest and memory-heaviest. This follows directly from the two-pass structure of Chapter 5: every intermediate expression is stored until the backward pass completes. On `a*` input of length n, state size grows linearly in n, while the bit-coded and partial-derivative parsers stay constant.

On the full corpus the two standard variants are close in mean time (7.718 ms against 7.430 ms) but the outcome split shows where the difference lives. On matching entries, `deriv_std_loop`'s total (70.782 s) is measurably below `deriv_std_rec`'s (74.725 s), while their non-match totals are almost the same. The recursion-versus-iteration difference is specifically a cost of the matching path, confirmed directly at Figure 7 scale and beyond (Table 8.9): the two variants crash at identical input lengths and near-identical memory figures on every representative row, which rules out recursion depth as the mechanism there and leaves the two-pass retention both variants share as the only remaining explanation. Stack depth specifically is where the two variants do diverge sharply (Section 8.4.1): `deriv_std_loop` survives 32 times further than `deriv_std_rec` on a pattern engineered to isolate that one cost alone, with the memory cap removed. The two findings are not in tension. They describe two different failure modes, and which one a given pattern hits first depends on whether its derivative sequence or its character count is the thing actually growing.

8.5.2 Cost of bit-coding

For the standard-derivative family, bit-coding wins unconditionally. `deriv_bc`'s mean time (1.699 ms) is roughly 4–4.5× below either standard variant's, and its peak allocation (5.1 MB) is about 114–144× below theirs. The trade-off is that the forward pass cannot simplify away redundant branches without losing reconstruction information. The retention is not wasted: it is exactly what makes single-pass reconstruction possible, and `deriv_bc`'s decode step is close to free (well under a tenth of a millisecond mean).

The representative sweeps push this further: `deriv_bc` survives every swept input length on six of the eight representative rows where both standard variants crash outright, at scales ranging from thousands of characters past the crash point (P27/P28) to ten million (P60, P20). Section 8.4.3's caveat applies directly here: `deriv_bc`'s cumulative allocation at that scale is large (up to 271 GB of total churn on P20's full sweep), but its peak live footprint stays in the low gigabytes throughout, the distinction between allocator pressure and actual memory held that motivates reporting both figures separately (Section 8.2.3).

8.5.3 Cost of partial derivatives

The two partial-derivative parsers are the fastest and most memory-light on the full S2ELab corpus, clearly ahead of either derivative-based variant. They split the residual into a set of states rather than combining into one expression, which keeps each state small and avoids the alternation growth the standard derivative parser suffers.

Within the pair, bit-coding's advantage is not unconditional the way it is for the standard-derivative family, and which variant wins depends on more than scale. On the full corpus, `pderiv_std` is marginally ahead on every figure: mean time (0.616 against 0.625 ms), average bytes (0.403 against 0.412 MB), and peak bytes (2.5 against 3.8 MB). At Figure 7 scale, the ranking splits by row. On P60 (`(a|b|ab)*` over 'a'-repeated input), `pderiv_std` stays faster at every point, with a stable gap that does not narrow with input length. On P61 (the same pattern over 'ab'-repeated input) the ranking flips: `pderiv_bc` is faster at every point, with a smaller but equally consistent gap. On P20 (`(a|b|ab)+`), `pderiv_bc` is both faster and lower-peak than `pderiv_std` (3.27 against 4.97 GB peak over the full sweep). The finding is that the input shape changes which variant wins, not just how large the gap is. Bit-coding's cost for the partial-derivative construction is a fixed per-step overhead, and whether it pays for itself depends on the pattern and input together, not on scale alone.

The price of partial derivatives, independent of which variant, is that they produce a non-POSIX tree. This is a specification difference, not a deficiency, and it is why the two families are compared separately in Section 8.4.5.

8.5.4 Threats to validity

Corpus scope. S2ELab is one dataset's own domain and convention. A corpus from a different domain might exhibit different acceptance rates and timing profiles. Timing, Memory, and Parse Agreement all default to the 463 stripped patterns (Section 8.3.1). A different, defensible choice (`BENCH_MATCH_INTENT=both`) would fold in the remaining 115 patterns and could shift absolute numbers, though not the ordering among parsers, which does not depend on which patterns are included.

The pathological set. Each of its 113 rows isolates one complexity mechanism at a controlled scale. It does not represent the distribution of pattern shapes a real author would write. Most categories sweep one parameter in isolation, so a pattern combining two stress mechanisms at once is not directly represented.

Measurement model. Memory benches measure allocation counts, not wall-clock memory pressure. The evaluator runs on a single machine, so the wall-clock figures are CPU- and cache-specific. A *crashed* entry's recorded reason is "killed by signal," not a confirmed diagnosis of stack overflow specifically (Section 8.2.1); Section 8.4.1 is the one measurement in this chapter that removes the memory cap and so can attribute a crash to stack overflow directly, and it is the basis for the claim, elsewhere in this chapter, that the representative-row crashes are a shared-retention effect rather than a recursion-depth one.

9 Related Work

10 Conclusion

This thesis implemented and evaluated four regular-expression parsers over a shared core language and frontend. Two follow Brzozowski derivatives and two follow Antimirov partial derivatives (Chapters 5 and 6), each family with a two-pass injection variant and a bit-coded variant. All four operate on the same six-constructor `Regex` from Chapter 7, measured on the two datasets of Chapter 8.

The measurements separate the four parsers into two groups. Partial-derivative parsers are fastest and lightest on the real corpus. Derivative parsers are slowest and heaviest, since both retain every intermediate expression until the backward pass, so their live state grows with input length.

Within the derivative family, bit-coding wins unconditionally. `deriv_bc` carries bounded state where both `deriv_std` variants carry the whole derivation sequence, so it is faster and lighter at every scale measured. On multi-million-character `(a|b|ab)*` inputs at the scale of Sulzmann and Lu [2014]'s own benchmark, both standard variants fail outright at every input length tested, by two independent mechanisms, stack overflow for the recursive variant and heap exhaustion for the iterative one, while `deriv_bc` completes the full ten-million-character range in under a minute.

Within the partial-derivative family the same claim does not hold. `pderiv_std` and `pderiv_bc` trade leading position with input structure rather than size. `pderiv_std` is marginally faster on the real corpus and on `(a|b|ab)*` over `a`-only inputs, while `pderiv_bc` is consistently faster on the same pattern over alternating `ab` inputs. The choice between them is therefore per-pattern and per-input.

Construction therefore matters more than variant. A POSIX parser that scales to long inputs has one answer, `deriv_bc`, the only POSIX variant with bounded state. A non-POSIX parser must be chosen by measurement. Where the input is short and a full POSIX tree is not needed, `deriv_std_loop` remains usable on code-complexity grounds, and `deriv_std_rec` stays closest to the published algorithm.

All five parsers correctly classify every case in both datasets. POSIX and non-POSIX tree shapes diverge only on ambiguous patterns and reflect a policy difference, not an implementation error.

Three directions remain open. The bit-coded representation is compatible with incremental parsing, since the bit vector that reconstructs a tree can equally well serve as the state from which a subsequent input character is consumed. The frontend's uniform 1000 bound cap (Section 7.4) could be replaced with a per-pattern cost analysis. The remaining multi-million-character sweeps in Section 8.4.2 await their configured scale.

Bibliography

Valentin Antimirov. Partial derivatives of regular expressions and finite automaton constructions. *Theoretical Computer Science*, 155(2):291–319, 1996. doi: 10.1016/0304-3975(95)00182-4.

Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4): 481–494, 1964. doi: 10.1145/321239.321249.

Russ Cox. Regular expression matching can be simple and fast (but is slow in Java, Perl, PHP, Python, Ruby, ...). https://swtch.com/~rsc/regexp/regexp1.html, 2007. Accessed: 2026-08-05.

Mohammed Latif Siddiq, Jiahao Zhang, Lindsay Roney, and Joanna C. S. Santos. RegexEval dataset. https://github.com/s2e-lab/RegexEval, 2024.

Martin Sulzmann. Programming paradigms: Regular expressions. https://sulzmann.github.io/ProgrammingParadigms/pp-regular-expressions.html, 2026. Accessed: 2026-08-08.

Martin Sulzmann and Kenny Zhuo Ming Lu. Regular expression sub-matching using partial derivatives. In *PPDP*, pages 79–90, 2012. doi: 10.1145/2370776.2370788.

Martin Sulzmann and Kenny Zhuo Ming Lu. Posix regular expression parsing with derivatives. In *Functional and Logic Programming (FLOPS 2014)*, volume 8475. Springer, 2014.

The Rust Project Developers. Using `Box<T>` to point to data on the heap. The Rust Programming Language, 2026a. URL https://doc.rust-lang.org/book/ch15-01-box.html. Accessed: 2026-09-11.

The Rust Project Developers. Types. The Rust Reference, 2026b. URL https://doc.rust-lang.org/reference/types.html. Accessed: 2026-09-11.